

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ CPU
THEO KIẾN TRÚC RISC-V 32 BIT

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

GVHD: ThS. LÊ MINH THÀNH
SVTH: NGUYỄN THANH QUÂN

MSSV: 22119219

BÙI ĐỨC HUY

MSSV: 22119179

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ CPU
THEO KIẾN TRÚC RISC-V 32 BIT**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

GVHD: ThS. LÊ MINH THÀNH

SVTH: NGUYỄN THANH QUÂN

MSSV: 22119219

BÙI ĐỨC HUY

MSSV: 22119179



TP. Hồ Chí Minh, ngày 23 tháng 06 năm 2026

NHIỆM VỤ ĐỒ ÁN

Họ và tên sinh viên: Nguyễn Thanh Quân - **MSSV:** 22119219

Bùi Đức Huy - MSSV: 22119179

Ngành: Công nghệ kỹ thuật máy tính

Lớp: 221191B

Giảng viên hướng dẫn: ThS. Lê Minh Thành

Ngày nhận đề tài: 14/03/2026

Ngày nộp đề tài: 23/06/2026

1. Tên Đề tài: Thiết kế CPU theo kiến trúc RISC-V 32 bit

2. Các số liệu, tài liệu ban đầu: kiến thức cơ bản về các môn Thiết kế hệ thống và vi mạch tích hợp, Thiết kế mạch tích hợp VLSI, Kỹ thuật số,...

3. Nội dung thực hiện đề tài

Dựa trên các yêu cầu và định hướng đã đề ra, nhóm đã lựa chọn triển khai đề tài “Thiết kế CPU theo kiến trúc RISC-V 32 bit”. Quá trình thực hiện sử dụng ngôn ngữ Verilog để mô tả phần cứng, kết hợp với phần mềm Vivado và QuestaSim nhằm mô phỏng và phân tích dạng sóng. Thông qua việc thiết kế và kiểm thử hệ thống, nhóm có cơ sở để đánh giá khả năng hoạt động, đồng thời nhận diện rõ các điểm mạnh và hạn chế của thiết kế. Từ những kết quả đạt được, một số định hướng phát triển tiếp theo cũng được đề xuất nhằm nâng cao chất lượng và tính hoàn thiện của đề tài.

TRƯỞNG BỘ MÔN

(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Thiết kế CPU theo kiến trúc RISC-V 32 bit

Sinh viên: + Nguyễn Thanh Quân

MSSV: 22119219

+ Bùi Đức Huy

MSSV: 22119179

Hướng dẫn: ThS. Lê Minh Thành

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Hai SV có tinh thần, thái độ học tập, nghiên cứu tốt, đạt yêu cầu tốt nghiệp.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt yêu cầu và được bảo vệ.

Tp. HCM, ngày 22 tháng 06 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

Lưu
Lê Minh Thành

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Lời đầu tiên, nhóm thực hiện khóa luận xin gửi lời cảm ơn chân thành nhất đến thầy Lê Minh Thành, thầy đã hướng dẫn nhóm một cách tận tình nhất, những góp ý và đánh giá của thầy qua các buổi báo cáo đều giúp cho nhóm có thể nhìn ra những khuyết điểm, vượt qua các khó khăn về mặt kỹ thuật, nội dung và cả hình thức của bài báo cáo.

Nhóm thực hiện xin gửi lời cảm ơn đến quý thầy cô trong khoa Điện – Điện tử của Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh vì đã giúp cho nhóm có được những kiến thức từ cơ bản đến nâng cao, tạo tiền đề cho việc thực hiện khóa luận này. Do kiến thức, kinh nghiệm còn hạn hẹp, nội dung của khóa luận này có thể còn nhiều thiếu sót, nhóm xin kính mong nhận được những lời góp ý, nhận xét đến từ quý thầy cô để có thể hoàn thiện đề tài hơn.

Sau cùng, em gửi lời chúc đến quý Thầy/Cô thật nhiều sức khỏe, luôn tràn đầy nhiệt huyết và thành công trong sự nghiệp.

Nhóm thực hiện đề tài

Nguyễn Thanh Quân

Bùi Đức Huy

TÓM TẮT

Trong bối cảnh công nghệ vi xử lý ngày càng phát triển mạnh mẽ, kiến trúc tập lệnh mở RISC-V đã và đang khẳng định vị thế của mình như một nền tảng thiết kế vi xử lý linh hoạt, minh bạch và hiệu quả. Không bị ràng buộc bởi bản quyền như x86 hay ARM, RISC-V mở ra cơ hội lớn cho cộng đồng nghiên cứu, giáo dục và các doanh nghiệp muốn tự chủ trong thiết kế phần cứng.

Đề tài "Thiết kế CPU theo kiến trúc RISC-V 32 bit" được thực hiện nhằm nghiên cứu và hiện thực hóa một bộ xử lý hoàn chỉnh hỗ trợ cả phép toán số nguyên lẫn số thực dấu phẩy động trên kiến trúc RISC-V 32 bit. Đây là một bài toán kỹ thuật số có tính ứng dụng cao, đòi hỏi sự hiểu biết sâu về tổ chức bộ xử lý, thiết kế đường ống, xử lý xung đột và đặc tả tập lệnh chuẩn RISC-V.

Báo cáo được trình bày theo cấu trúc từ tổng quan kiến trúc, thiết kế chi tiết các khối chức năng, triển khai đường ống đến kiểm thử và mô phỏng. Quá trình kiểm thử mô phỏng bao gồm kiểm thử riêng lẻ từng khối chức năng, kiểm thử toàn hệ thống và kiểm thử trên môi trường UVM nhằm đánh giá tính đúng đắn của thiết kế.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	ii
MỤC LỤC.....	iii
DANH MỤC HÌNH ẢNH.....	vii
DANH MỤC BẢNG.....	x
CÁC TỪ VIẾT TẮT.....	xii
CHƯƠNG 1: TỔNG QUAN.....	1
1.1. GIỚI THIỆU	1
1.2. MỤC TIÊU	2
1.3. GIỚI HẠN	2
1.4. BỘ CỤC ĐỒ ÁN	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	4
2.1. KIẾN TRÚC RISC-V VÀ TẬP LỆNH RV32IF.....	4
2.1.1. Tổng quan kiến trúc RISC-V.....	4
2.1.2. Tập lệnh số nguyên RV32I	5
2.1.2.1. Định dạng lệnh	5
2.1.2.2. Tập thanh ghi số nguyên	5
2.1.2.3. Nhóm lệnh tính toán.....	6
2.1.2.4. Nhóm lệnh điều khiển luồng.....	7
2.1.3. Phân mở rộng số thực RV32F	9

2.1.3.1. Biểu diễn số thực theo chuẩn IEEE 754	9
2.1.3.2. Các nhóm lệnh RV32F	10
2.1.3.3. Tập thanh ghi số thực.....	11
2.2. KIẾN TRÚC ĐƯỜNG ỐNG 5 TẦNG.....	11
2.2.1. Tổng quan về kỹ thuật đường ống.....	11
2.2.2. Đường ống 5 tầng.....	12
2.2.3. Ưu và nhược điểm đường ống 5 tầng.....	12
2.2.4. Xung đột trong đường ống	13
2.2.4.1. Xung đột dữ liệu	13
2.2.4.2. Xung đột điều khiển.....	15
2.3. PHƯƠNG PHÁP XÁC MINH BẰNG MÔI TRƯỜNG UVM.....	16
2.3.1. Các thành phần trong UVM	16
2.3.2. Trình tự vận hành trong UVM.....	18
CHƯƠNG 3: THIẾT KẾ.....	21
3.1. YÊU CẦU CỦA HỆ THỐNG	21
3.2. SƠ ĐỒ KHỐI HỆ THỐNG	22
3.3. THIẾT KẾ CHI TIẾT CỦA HỆ THỐNG.....	24
3.3.1. Khối nạp lệnh	24
3.3.2. Khối giải mã	26
3.3.3. Khối thực thi.....	33
3.3.4. Khối truy cập bộ nhớ.....	37

3.3.5. Khối ghi lại	39
3.3.6. Khối xử lý xung đột.....	41
3.4. THIẾT KẾ MÔI TRƯỜNG KIỂM THỬ	43
3.4.1. Tổng quan kiến trúc môi trường UVM	43
3.4.2. Thiết kế khối sequence và khối sequencer	44
3.4.3. Thiết kế khối driver và khối monitor.....	45
3.4.4. Thiết kế khối interface.....	45
3.4.5. Thiết kế khối scoreboard	46
CHƯƠNG 4: KẾT QUẢ.....	47
4.1. KẾT QUẢ MÔ PHỎNG TỪNG KHỐI	47
4.1.1. Khối nạp lệnh	47
4.1.2. Khối giải mã	48
4.1.3. Khối thực thi.....	52
4.1.4. Khối truy cập bộ nhớ.....	57
4.1.5. Khối ghi lại.....	59
4.1.6. Khối xử lý xung đột.....	60
4.2. KẾT QUẢ MÔ PHỎNG TOÀN HỆ THỐNG	62
4.2.1. Bài kiểm tra số 1	62
4.2.2. Bài kiểm tra số 2.....	65
4.2.3. Bài kiểm tra số 3.....	70
4.3 Kết quả kiểm tra bằng môi trường UVM	73

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	77
5.1. KẾT LUẬN.....	77
5.2. HƯỚNG PHÁT TRIỂN.....	77
TÀI LIỆU THAM KHẢO	78

DANH MỤC HÌNH ẢNH

Hình 2.1: Cấu trúc lệnh loại R	6
Hình 2.2: Cấu trúc lệnh loại I.....	7
Hình 2.3: Cấu trúc lệnh loại B	8
Hình 2.4: Cấu trúc lệnh loại S	9
Hình 2.5: Thực thi tuần tự không có đường ống.....	12
Hình 2.6: Thực thi song song có đường ống.....	12
Hình 2.7: Kiến trúc tổng quan của môi trường UVM.....	18
Hình 2.8: Trình tự vận hành trong UVM	19
Hình 3.1: Trình tự thực thi của bộ vi xử lý đơn chu kỳ.....	22
Hình 3.2: Sơ đồ khối của bộ xử lý đơn chu kỳ [1].....	22
Hình 3.3: Tất cả các giai đoạn trong bộ vi xử lý kiến trúc đường ống	23
Hình 3.4: Cơ chế hỗ trợ chuyển tiếp tránh xung đột [1]	24
Hình 3.5: Sơ đồ khối tầng nạp lệnh.....	25
Hình 3.6: Sơ đồ khối chi tiết của khối giải mã.....	27
Hình 3.7: Sơ đồ khối của khối điều khiển.....	30
Hình 3.8: Sơ đồ khối tập thanh ghi số nguyên.....	32
Hình 3.9: Sơ đồ khối bộ giải mã FPU	32
Hình 3.10: Sơ đồ khối của khối thực thi	34
Hình 3.11: Sơ đồ khối của khối truy cập bộ nhớ	37
Hình 3.12: Sơ đồ khối của khối ghi lại	39

Hình 3.13: Sơ đồ khối của khối xử lý xung đột	41
Hình 3.14: Kiến trúc môi trường UVM	43
Hình 4.1: Dạng sóng của khối nạp lệnh.....	47
Hình 4.2: PCSrcE được kích hoạt và giá trị các thanh ghi	48
Hình 4.3: Dạng sóng mô phỏng chức năng của khối giải mã từ 0ns đến 100ns	48
Hình 4.4: Dạng sóng mô phỏng chức năng của khối giải mã sau 100ns	49
Hình 4.5: Kết quả mô phỏng của tầng giải mã từ 35ns đến 95ns	49
Hình 4.6: Kết quả mô phỏng của tầng giải mã từ 95ns đến 115ns	50
Hình 4.7: Kết quả mô phỏng của tầng giải mã từ 115ns đến 145ns	51
Hình 4.8: Kết quả mô phỏng của tầng giải mã từ 145ns đến 185ns	51
Hình 4.9: Kết quả mô phỏng của tầng giải mã từ 185ns đến sau 215ns	52
Hình 4.10: Kết quả dạng sóng mô phỏng của tầng thực thi.....	53
Hình 4.11: Kết quả dạng sóng mô phỏng của tầng thực thi từ 15ns đến 35ns.....	54
Hình 4.12: Kết quả dạng sóng mô phỏng của tầng thực thi từ 36ns đến 66ns.....	55
Hình 4.13: Kết quả dạng sóng mô phỏng của tầng thực thi từ 85ns đến 135ns.....	56
Hình 4.14: Kết quả mô phỏng của khối thực thi từ 145ns đến 185ns.....	57
Hình 4.15: Kết quả mô phỏng chức năng của tầng truy cập bộ nhớ.....	58
Hình 4.16: Kết quả mô phỏng của tầng ghi lại	59
Hình 4.17: Kết quả mô phỏng khối xử lý xung đột	61
Hình 4.18: Dạng sóng thực thi các lệnh của bài kiểm tra số 1 từ 0ns đến 40ns	63
Hình 4.19: Dạng sóng thực thi các lệnh của bài kiểm tra số 1 từ 50ns đến 90ns	63

Hình 4.20: Dạng sóng thực thi các lệnh của bài kiểm tra số 1 từ 90ns đến 130ns ...	64
Hình 4.21: Dạng sóng thực thi các lệnh của bài kiểm tra số 1 từ 130ns đến 150ns .	65
Hình 4.22: Dạng sóng thực thi các lệnh của bài kiểm tra số 2 từ 0ns đến 40ns	67
Hình 4.23: Dạng sóng thực thi các lệnh của bài kiểm tra số 2 từ 50ns đến 90ns	67
Hình 4.24: Dạng sóng thực thi các lệnh của bài kiểm tra số 2 từ 90ns đến 130ns ...	68
Hình 4.25: Dạng sóng thực thi các lệnh của bài kiểm tra số 2 từ 140ns đến 180ns .	69
Hình 4.26: Dạng sóng thực thi các lệnh của bài kiểm tra số 2 từ 180ns đến 220ns .	70
Hình 4.27: Dạng sóng thực thi các lệnh của bài kiểm tra số 3 từ 0ns đến 40ns	71
Hình 4.28: Dạng sóng thực thi các lệnh của bài kiểm tra số 3 từ 50ns đến 90ns	72
Hình 4.29: Dạng sóng thực thi các lệnh của bài kiểm tra số 3 từ 70ns đến 90ns	72
Hình 4.30: Các bài kiểm tra được thực hiện trong môi trường UVM	74
Hình 4.31: Kết quả tổng hợp từ các bài kiểm tra	74
Hình 4.32: Kết quả độ bao phủ từ QuestaSim	76

DANH MỤC BẢNG

Bảng 2.1: Chức năng của từng trường trong lệnh loại R	7
Bảng 2.2: Chức năng của từng trường trong lệnh loại I	7
Bảng 2.3: Chức năng của từng trường trong lệnh loại B	8
Bảng 2.4: Chức năng của từng trường trong lệnh S.....	9
Bảng 3.1: Các tín hiệu của khối nạp lệnh	26
Bảng 3.2: Mô tả các tín hiệu của khối giải mã.....	27
Bảng 3.3: Bảng giải mã mã lệnh	30
Bảng 3.4: Bảng tổng hợp mã lệnh ALU	31
Bảng 3.5: Bảng mở rộng giá trị tức thì	31
Bảng 3.6: Bảng giải mã lệnh FPU.....	33
Bảng 3.7: Nhóm tín hiệu đầu vào dữ liệu	35
Bảng 3.8: Nhóm tín hiệu đầu vào điều khiển.....	35
Bảng 3.9: Các tín hiệu nội bộ.....	36
Bảng 3.10: Tín hiệu của khối truy cập bộ nhớ.....	38
Bảng 3.11: Tín hiệu của khối ghi lại	40
Bảng 3.12: Tín hiệu của khối xử lý xung đột.....	42
Bảng 3.13: Các sequence được thiết kế cho việc kiểm thử.....	44
Bảng 3.14: Các tín hiệu trong interface	45
Bảng 4.1: Bảng mã bài kiểm tra số 1.....	62
Bảng 4.2: Bảng mã bài kiểm tra số 2	66

Bảng 4.3: Bảng mã các lệnh của bài kiểm tra số 3	70
Bảng 4 4: Bảng sai số giữa kết quả kỳ vọng và kết quả thực tế.....	73
Bảng 4.5: Tổng hợp kết quả kiểm tra từ môi trường UVM	75

CÁC TỪ VIẾT TẮT

STT	Từ viết tắt	Tên đầy đủ
1	AHB	Advanced High-performance Bus
2	AI	Artificial Intelligence
3	ALU	Arithmetic Logic Unit
4	APB	Advanced Peripheral Bus
5	CPU	Central Processing Unit
6	DUT	Design Under Test
7	DZ	Divide by Zero
8	EX	Execute
9	FPGA	Field Programmable Gate Array
10	FPU	Floating-Point Unit
11	I2C	Inter-Integrated Circuit
12	ID	Instruction Decode
13	IEEE	Institute of Electrical and Electronics Engineers
14	IF	Instruction Fetch
15	IoT	Internet of Things

16	MEM	Memory Access
17	MIPS	Microprocessor without Interlocked Pipelined Stages
18	NaN	Not a Number
19	NOP	No Operation
20	OF	Overflow
21	PC	Program Counter
22	RISC-V	Reduced Instruction Set Computer – Five
23	RTL	Register Transfer Level
24	RV32F	RISC-V 32 bit Floating-Point Extension
25	RV32I	RISC-V 32 bit Base Integer Instruction Set
26	RV32M	RISC-V 32 bit Integer Multiplication and Division Extension
27	SPI	Serial Peripheral Interface
28	UART	Universal Asynchronous Receiver-Transmitter
29	UF	Underflow
30	UVM	Universal Verification Methodology
31	WB	Write Back

CHƯƠNG 1

TỔNG QUAN

1.1. GIỚI THIỆU

Ngày nay, vi xử lý đóng vai trò quan trọng trong hầu hết các sản phẩm điện tử hiện đại, từ máy tính, điện thoại thông minh cho đến các hệ thống nhúng. Ngoài ra, sự phát triển mạnh mẽ của trí tuệ nhân tạo và Internet vạn vật đang thúc đẩy nhu cầu ngày càng cao về các vi xử lý hiệu năng cao.

Trong giai đoạn này, RISC-V nổi lên như một kiến trúc tập lệnh mở và có khả năng mở rộng được phát triển tại Đại học Berkeley là một lựa chọn lý tưởng cho nghiên cứu nhờ tính mở, không phải trả phí bản quyền như các kiến trúc thương mại x86 của Intel hay ARM, RISC-V cho phép tự do tùy chỉnh và mở rộng tập lệnh theo nhu cầu ứng dụng cụ thể [1]. Nhờ tính linh hoạt đó, RISC-V đã nhanh chóng trở thành nền tảng phổ biến cho các dự án nghiên cứu học thuật lẫn ứng dụng công nghiệp trên toàn thế giới.

Kiến trúc RISC-V được ứng dụng rộng rãi trong nhiều lĩnh vực, từ các thiết bị đeo tay, IoT và thiết bị gia dụng thông minh yêu cầu tiêu thụ điện năng thấp, cho đến điện thoại thông minh, hệ thống ô tô, điện toán hiệu năng cao và trung tâm dữ liệu; thậm chí độ tin cậy và bảo mật cao của RISC-V còn khiến kiến trúc này trở thành lựa chọn phù hợp cho các ứng dụng đặc thù trong lĩnh vực hàng không vũ trụ và chính phủ [3].

Về mặt kỹ thuật, ưu thế cốt lõi của RISC-V đến từ khả năng triển khai kiến trúc đường ống 5 tầng một cách hiệu quả, cho phép nhiều lệnh được thực thi chồng chéo nhau trong cùng một chu kỳ đồng hồ, từ đó tăng đáng kể hiệu năng xử lý mà không cần nâng tần số xung nhịp. Điều này đã được minh chứng qua kết quả so sánh giữa hai cấu hình hiện thực hóa trên cùng nền tảng FPGA: bộ vi xử lý RV32I đơn chu kỳ đạt tần số tối đa 65.4MHz, trong khi phiên bản đường ống 5 tầng đạt 117.74MHz tương đương mức tăng gần 1.8 lần thông qua phân tầng [4]. Điều này cho thấy rằng kiến trúc đường ống không chỉ nâng cao thông lượng xử lý mà còn cải thiện đáng kể hiệu năng tổng thể của hệ thống.

Đi cùng với sự phát triển mạnh mẽ đó, RV32I là tập lệnh 32 bit đơn giản được sử dụng phổ biến, đủ để xây dựng một bộ vi xử lý hoàn chỉnh cho các hệ thống nhúng

nhỏ gọn. Tuy nhiên, với các ứng dụng đòi hỏi tính toán số thực trong lĩnh vực xử lý tín hiệu, đồ họa và điều khiển, tập lệnh mở rộng RV32F được tích hợp bổ sung, cung cấp đơn vị dấu phẩy động (FPU) hỗ trợ các phép toán độ chính xác đơn theo chuẩn IEEE 754 [5], điều mà RV32I không thể đáp ứng. Việc xây dựng một môi trường nghiên cứu và kiểm thử cho RV32IF là điều cực kỳ cần thiết. Phương pháp UVM là chuẩn kiểm thử được lựa chọn trong công nghiệp và được áp dụng rộng rãi trong thiết kế vi mạch, nhờ khả năng xây dựng môi trường kiểm thử tự động, tái sử dụng cao và đáng tin cậy, giúp xác minh tính đúng đắn của toàn bộ kiến trúc RV32IF một cách hiệu quả [4].

Vì vậy, nhóm thực hiện khóa luận sẽ thực hiện thiết kế bộ vi xử lý 32 bit theo kiến trúc RISC-V, áp dụng mô hình đường ống 5 tầng cho tập lệnh RV32I, đồng thời triển khai phần mở rộng RV32F để xử lý dấu phẩy động theo chuẩn IEEE 754. Ngoài ra, đề tài còn xây dựng môi trường xác minh có cấu trúc theo phương pháp UVM thay cho kiểm thử thủ công. Kết quả dự kiến đạt được là một bộ vi xử lý RISC-V 32 bit có khả năng thực thi đúng cả tập lệnh số nguyên và dấu phẩy động, được xác minh đầy đủ chức năng cụ thể trên môi trường mô phỏng.

1.2. MỤC TIÊU

Đề tài hướng tới mục tiêu thiết kế một bộ vi xử lý 32 bit theo kiến trúc RISC-V mã nguồn mở áp dụng kỹ thuật đường ống 5 tầng để có thể tối ưu hóa hiệu năng thực thi lệnh liên tục và hỗ trợ tập lệnh RV32IF.

Bên cạnh việc thiết kế, đề tài còn hướng đến việc xây dựng một môi trường xác minh chức năng theo phương pháp UVM để đảm bảo thiết kế hoạt động chính xác.

1.3. GIỚI HẠN

Đề tài tập trung vào thiết kế và xác minh chức năng bộ vi xử lý 32 bit dựa trên kiến trúc RISC-V. Thiết kế được mô tả ở mức RTL bằng ngôn ngữ Verilog và được xác minh với môi trường UVM và ngôn ngữ SystemVerilog thông qua phần mềm QuestaSim. Để đảm bảo phạm vi nghiên cứu của đề tài và phù hợp với khả năng của nhóm, các lệnh được lựa chọn chủ yếu là những lệnh phổ biến, đại diện cho các nhóm chức năng cơ bản như tính toán số nguyên, truy cập bộ nhớ, rẽ nhánh và xử lý số

thực. Bên cạnh đó, thiết kế bổ sung một tập con của phần mở rộng RV32M gồm các lệnh mul, div nhằm mở rộng khả năng tính toán số nguyên của khối ALU.

Bộ vi xử lý được thiết kế với đầy đủ 32 thanh ghi số nguyên theo đặc tả RV32I, trong đó x0 được cố định bằng 0 và 32 thanh ghi dấu phẩy động 32 bit hoàn toàn tách biệt theo phần mở rộng RV32F.

Đề tài chỉ tập trung ở mức thiết kế và mô phỏng chức năng, chưa thực hiện các bước thiết kế vật lý. Kiến trúc CPU được giới hạn ở mô hình đường ống 5 tầng cơ bản, chưa triển khai các kỹ thuật tối ưu nâng cao như thực thi ngoài thứ tự hoặc bộ nhớ đệm nhiều tầng.

1.4. BỐ CỤC ĐỀ TÀI

Chương 1: Tổng quan. Giới thiệu về vai trò, xu hướng của bộ vi xử lý trong thời đại ngày nay cùng với ứng dụng, yêu cầu và giải pháp. Trình bày nội dung nghiên cứu, mục tiêu, giới hạn và bố cục đề tài.

Chương 2: Cơ sở lý thuyết. Tìm hiểu về bộ vi xử lý, kiến trúc RISC-V và trình bày các nội dung có liên quan như tập lệnh RV32I, RV32F và UVM.

Chương 3: Thiết kế. Xác định yêu cầu của hệ thống, vẽ sơ đồ khối và thiết kế chi tiết của từng khối chức năng và cuối cùng là thiết kế môi trường xác minh UVM.

Chương 4: Kết quả. Chạy mô phỏng chương trình, trình bày kết quả đã đạt được sau đó đánh giá thiết kế dựa trên kết quả đó, từ đó đưa ra nhận xét về độ chính xác của kết quả.

Chương 5: Kết luận và hướng phát triển. Đưa ra kết luận về kết quả đã đạt được và hạn chế trong đề tài sẽ giải quyết như thế nào từ đó đưa ra hướng phát triển đề tài.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1. KIẾN TRÚC RISC-V VÀ TẬP LỆNH RV32IF

2.1.1. Tổng quan kiến trúc RISC-V

RISC-V là một kiến trúc tập lệnh mã nguồn mở ban đầu được phát triển tại Đại học Berkeley. Khác với các kiến trúc độc quyền như x86 của Intel hay ARM, RISC-V được thiết kế với triết lý đơn giản, hiện đại và hoàn toàn mở, không yêu cầu giấy phép sử dụng. RISC-V ra đời nhằm giải quyết những hạn chế của các kiến trúc trước đó, đặc biệt là MIPS. Mặc dù MIPS đã phục vụ tốt trong giáo dục và nghiên cứu, nhưng việc tìm kiếm các máy tính thực tế để chạy chương trình MIPS ngày càng khó khăn do MIPS chủ yếu được sử dụng trong các thiết bị nhúng không hiển thị tập lệnh. RISC-V được tạo ra như một giải pháp thay thế thanh lịch, đơn giản và hiện đại cho những gì tập lệnh nên có vào năm 2017 [1]. Tính mở của RISC-V đã tạo ra một cơ hội thú vị cho cả công nghiệp và giáo dục. Các trình mô phỏng, trình biên dịch, trình gỡ lỗi đều dễ dàng có sẵn, thậm chí cả cách triển khai kiến trúc được viết bằng ngôn ngữ mô tả phần cứng. Điều này cho phép người học không chỉ nghiên cứu các thiết kế của kiến trúc này mà còn có thể sửa đổi chúng và trải qua quy trình triển khai để hiểu tác động của những thay đổi giả định đối với hiệu suất, kích thước chip và năng lượng. Tính đến thời điểm này, hơn 40 công ty đã tham gia vào RISC-V với tư cách là nhà đầu tư (và con số này đã vượt 300 công ty theo ấn bản năm 2020 của tài liệu [1]). Danh sách nhà tài trợ bao gồm hầu hết các công ty lớn ngoại trừ ARM và Intel, như AMD, Google, Hewlett Packard Enterprise, IBM, Microsoft, NVIDIA, Oracle, Qualcomm, Alibaba, Amazon, Samsung và Western Digital [1].

Kiến trúc RISC-V hiện nay đang khẳng định vị thế là một nền tảng linh hoạt với tiềm năng ứng dụng sâu rộng trong nhiều phân khúc công nghệ nhờ vào đặc tính mã nguồn mở và khả năng tùy biến cao. Trong lĩnh vực trí tuệ nhân tạo và Internet vạn vật, kiến trúc này đã trở thành lựa chọn ưu việt cho các thiết kế tối ưu hóa năng lượng như cảm biến thông minh hay các bộ xử lý biên. Minh chứng điển hình là việc NVIDIA tích hợp hàng chục lõi vi mô dựa trên kiến trúc RISC-V vào các bộ tăng tốc AI của mình, trong đó mỗi lõi được tối ưu hóa chuyên biệt cho các tác vụ như thu

nhận dữ liệu, lập lịch và suy luận [2]. Bên cạnh đó, với sự ủng hộ từ các tập đoàn công nghệ hàng đầu như NVIDIA và Alibaba, kiến trúc này đang mở rộng mạnh mẽ sang lĩnh vực điện toán hiệu năng cao và trí tuệ nhân tạo, cho phép các nhà thiết kế xây dựng các bộ tăng tốc chuyên dụng để xử lý hiệu quả các khối lượng công việc phức tạp.

Tóm lại, bằng cách kế thừa triết lý tập lệnh đơn giản hóa, RISC-V mang đến hiệu năng tối ưu trên mỗi đơn vị năng lượng tiêu thụ, đặc biệt phù hợp với hệ thống nhúng và thiết bị di động. Tập lệnh độ dài cố định cùng logic giải mã gọn nhẹ tạo nền tảng lý tưởng cho kiến trúc đường ống đa tầng, cho phép phần lớn lệnh cơ bản hoàn thành trong một chu kỳ xung nhịp. Việc tinh giản khâu giải mã không chỉ thu hẹp diện tích chiếm dụng trên chip mà còn nhường chỗ cho các thanh ghi đa năng hoặc khối xử lý chuyên dụng. Chính sự kết hợp giữa hiệu quả phần cứng và tính linh hoạt thiết kế đã định vị RISC-V như một lựa chọn kiến trúc chiến lược cho thế hệ vi xử lý tiếp theo.

2.1.2. Tập lệnh số nguyên RV32I

2.1.2.1. Định dạng lệnh

Kiến trúc tập lệnh RISC-V được xây dựng trên triết lý tối giản nhưng linh hoạt, thông qua cấu trúc phân lớp và tính mô-đun rõ ràng. Thay vì đóng gói cố định như các kiến trúc truyền thống, kiến trúc này tách biệt tập lệnh cơ sở (I) khỏi các phần mở rộng tùy chọn như M, A, F, D, cho phép nhà thiết kế chỉ tích hợp những khối chức năng thực sự cần thiết, từ đó tối ưu diện tích chip và công suất tiêu thụ.

Ở cấp độ mã hóa, tất cả lệnh trong biến thể RV32I đều có độ rộng cố định 32 bit, tạo nền tảng cho quá trình nạp và giải mã ổn định. RISC-V định nghĩa sáu định dạng lệnh gồm R, I, S, B, U và J. Mỗi định dạng mang cấu trúc trường dữ liệu riêng biệt nhưng đều tuân theo nguyên tắc chung: các trường quan trọng như chỉ số thanh ghi và mã lệnh được giữ cố định vị trí xuyên suốt, giúp đơn giản hóa đáng kể logic giải mã phần cứng.

2.1.2.2. Tập thanh ghi số nguyên

Trong kiến trúc RV32I, tập thanh ghi số nguyên là đơn vị lưu trữ trung tâm cho toán hạng, kết quả các phép toán và địa chỉ truy cập bộ nhớ. Cấu trúc này gồm 32 thanh ghi 32 bit, ký hiệu từ x0 đến x31. Điểm đặc biệt nhất là thanh ghi x0, được nói

cứng với giá trị 0 thay vì hoạt động như một thanh ghi đa năng thông thường. Thiết kế này vừa đơn giản hóa các lệnh gán và khởi tạo, vừa tiết kiệm diện tích phần cứng. Các thanh ghi còn lại từ x1 đến x31 được sử dụng để lưu trữ toán hạng, kết quả tính toán, địa chỉ và các giá trị tạm thời trong quá trình thực thi chương trình.

Về mặt hoạt động, mỗi chu kỳ xung nhịp cho phép đọc đồng thời hai thanh ghi và ghi vào một thanh ghi. Thao tác đọc diễn ra tức thời theo kiểu tổ hợp, không phụ thuộc xung nhịp, trong khi ghi được đồng bộ theo cạnh dương của xung nhịp để đảm bảo tính ổn định.

Khi mở rộng với các phần bổ sung như F, D hay V, kiến trúc RISC-V có thể bổ sung thêm 32 thanh ghi dấu phẩy động và 32 thanh ghi vector, nâng tổng số lên 96 thanh ghi mà không làm thay đổi cấu trúc tập lệnh cơ sở.

2.1.2.3. Nhóm lệnh tính toán

Nhóm lệnh tính toán đóng vai trò thực thi các phép toán số học và logic cơ bản trên tập thanh ghi số nguyên, sử dụng hai định dạng chính là lệnh loại R (thao tác giữa hai thanh ghi) và lệnh loại I (thao tác giữa một thanh ghi và một hằng số trực tiếp).

Định dạng lệnh loại R dùng cho các phép toán số học logic giữa hai thanh ghi, phổ biến trong lệnh ADD, SUB, AND, OR, SLL, SRL, SRA.

funct7	rs2	rs1	funct3	rd	opcode
--------	-----	-----	--------	----	--------

Hình 2.1: Cấu trúc lệnh loại R

Bảng 2.1: Chức năng của từng trường trong lệnh loại R

Trường	Số bit	Vị trí	Ý nghĩa
funct7	7	31–25	Xác định biến thể phép toán, phân biệt ADD/SUB/SRA...
rs2	5	24–20	Thanh ghi nguồn thứ hai
rs1	5	19–15	Thanh ghi nguồn thứ nhất
funct3	3	14–12	Mã thao tác phụ, phân biệt trong nhóm opcode
rd	5	11–7	Thanh ghi đích, lưu kết quả
opcode	7	6–0	Loại thao tác, luôn là 0110011 cho lệnh loại R

Định dạng lệnh loại I dùng khi cần thao tác với một giá trị hằng hoặc khi tải dữ liệu từ bộ nhớ, các lệnh điển hình là ADDI, LW, JALR.

imm	rs1	funct3	rd	opcode
-----	-----	--------	----	--------

Hình 2.2: Cấu trúc lệnh loại I**Bảng 2.2:** Chức năng của từng trường trong lệnh loại I

Trường	Số bit	Vị trí	Ý nghĩa
imm	12	31–20	Số tức thời, dùng cho phép toán/số liệu
rs1	5	19–15	Thanh ghi nguồn thứ nhất
funct3	3	14–12	Chỉ định kiểu phép toán hoặc loại lệnh
rd	5	11–7	Thanh ghi đích, lưu kết quả
opcode	7	6–0	Mã loại lệnh

2.1.2.4. Nhóm lệnh điều khiển luồng

Nhóm lệnh điều khiển luồng cho phép thay đổi giá trị của bộ đếm chương trình, từ đó hiện thực hóa các cấu trúc điều kiện, vòng lặp và lời gọi hàm. RISC-V phân chia nhóm này thành hai loại chính: nhánh có điều kiện và nhảy không điều kiện. Các lệnh nhánh có điều kiện (lệnh loại B) như BEQ, BNE, BLT thực hiện so sánh trực

tiếp giá trị giữa hai thanh ghi nguồn để quyết định việc nhảy đến một địa chỉ tương đối. Cơ chế so sánh trực tiếp này giúp loại bỏ nhu cầu về các cờ trạng thái phức tạp, từ đó tối ưu hóa diện tích thiết kế phần cứng.

imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
---------	-----------	-----	-----	--------	----------	---------	--------

Hình 2.3: Cấu trúc lệnh loại B

Bảng 2.3: Chức năng của từng trường trong lệnh loại B

Trường	Số bit	Vị trí	Ý nghĩa
imm[12]	1	31	Bit thứ 12 của immediate
imm[10:5]	6	30-25	Các bit 10-5 của immediate
rs2	5	24-20	Thanh ghi nguồn thứ 2 để so sánh
rs1	5	19-15	Thanh ghi nguồn thứ 1 để so sánh
funct3	3	14-12	Chỉ định loại điều kiện rẽ nhánh
imm[4:1]	4	11-8	Các bit 4-1 của immediate
imm[11]	1	7	Bit thứ 11 của immediate
opcode	7	6-0	Mã lệnh xác định loại B

2.1.2.5. Nhóm lệnh truy cập bộ nhớ

Tuân thủ nghiêm ngặt mô hình kiến trúc nạp và lưu, RISC-V quy định rằng mọi thao tác tương tác với bộ nhớ dữ liệu chỉ được thực hiện thông qua nhóm lệnh chuyên biệt này. Các lệnh nạp sử dụng định dạng loại I để di chuyển dữ liệu từ bộ nhớ vào tập thanh ghi, hỗ trợ đa dạng các kích thước từ byte, half-word đến word. Điểm đáng chú ý là RISC-V hỗ trợ cả hai cơ chế nạp có mở rộng dấu (LB, LH) và nạp không mở rộng dấu (LBU, LHU), giúp duy trì tính chính xác của dữ liệu khi thay đổi kích thước biểu diễn bit.

Ngược lại, nhóm lệnh lưu trữ sử dụng định dạng lệnh S để ghi dữ liệu từ tập thanh ghi vào bộ nhớ thông qua các lệnh SB, SH, SW. Trong quá trình thực thi, địa chỉ bộ nhớ được xác định bằng cách cộng giá trị của thanh ghi cơ sở với một độ lệch có dấu 12 bit. Việc thiết kế định dạng lệnh S chia tách trường giá trị tức thời thành hai phần

là một dụng ý kỹ thuật quan trọng, giúp giữ vị trí của các cổng đọc thanh ghi nguồn (rs1, rs2) cố định ở mọi định dạng lệnh. Điều này giúp đơn giản hóa cấu trúc của bộ chọn kênh trong đường dữ liệu và rút ngắn thời gian giải mã lệnh tại tầng giải mã.

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
-----------	-----	-----	--------	----------	--------

Hình 2.4: Cấu trúc lệnh loại S

Bảng 2.4: Chức năng của từng trường trong lệnh S

Trường	Số bit	Vị trí	Ý nghĩa
opcode	7	6-0	Mã lệnh, xác định loại lệnh là S.
imm[4:0]	5	11-7	Phần thấp của immediate 12 bit, được ghép với phần cao để tạo số bù hai offset cho địa chỉ.
funct3	3	14-12	Chỉ định kiểu lưu trong lệnh lưu.
rs1	5	19-15	Thanh ghi chứa địa chỉ cơ sở dùng cho tính địa chỉ lưu dữ liệu.
rs2	5	24-20	Thanh ghi chứa dữ liệu cần lưu.
imm[11:5]	7	31-25	Phần cao của immediate 12 bit.

2.1.3. Phân mở rộng số thực RV32F

2.1.3.1. Biểu diễn số thực theo chuẩn IEEE 754

Chuẩn IEEE 754 là tiêu chuẩn kỹ thuật phổ biến nhất do Viện Kỹ thuật Điện và Điện tử thiết lập để định dạng và biểu diễn số thực dấu phẩy động (số có dấu chấm thập phân) trong máy tính. Nó quy định cách mã hóa nhị phân (độ chính xác đơn 32 bit, kép 64 bit), quy tắc làm tròn và xử lý ngoại lệ (NaN, vô cực) để đảm bảo tính nhất quán trên các phần cứng.

Trong kiến trúc RV32F, dữ liệu số thực được biểu diễn theo chuẩn IEEE 754 với định dạng độ chính xác đơn có độ dài 32 bit. Cấu trúc của số thực này gồm ba thành phần chính: bit dấu, phần mũ và phần định trị. Cụ thể, bit dấu chiếm 1 bit (bit thứ 31), dùng để xác định dấu của số, trong đó giá trị 0 biểu thị số dương và giá trị 1 biểu thị số âm. Phần mũ chiếm 8 bit tiếp theo (từ bit 30 đến bit 23), được biểu diễn dưới

dạng số nguyên không dấu có sử dụng độ lệch, với giá trị lệch trong chuẩn 32 bit là 127. Cuối cùng, phần định trị chiếm 23 bit còn lại (từ bit 22 đến bit 0), dùng để lưu trữ các chữ số có nghĩa của số thực sau dấu phẩy, qua đó quyết định độ chính xác của giá trị được biểu diễn.

Giá trị của một số thực chuẩn hóa theo chuẩn IEEE 754 được xác định bằng công thức sau:

$$(-1)^S \times [1 + f] \times 2^{E-127}$$

Trong đó:

- S là bit dấu, xác định số là dương hay âm.
- f là phần định trị, đại diện cho giá trị thập phân sau dấu phẩy.
- E là số mũ với độ lệch số mũ là 127.

2.1.3.2. Các nhóm lệnh RV32F

Tập lệnh RV32F được thiết kế để cung cấp khả năng tính toán số thực đơn chính xác (32 bit) dựa trên nền tảng của tập lệnh số nguyên RV32I. Điểm cốt lõi của phần mở rộng này là sự xuất hiện của các lệnh tính toán số học như cộng, trừ, nhân và chia. Đặc biệt, RISC-V còn hỗ trợ các lệnh nhân-cộng kết hợp, cho phép thực hiện phép tính $(a \times b) + c$ với chỉ một lần làm tròn duy nhất ở cuối quy trình. Cơ chế này không chỉ giúp tăng tốc độ thực thi mà còn giảm thiểu sai số tích lũy, một yếu tố cực kỳ quan trọng trong các thuật toán xử lý tín hiệu số và đồ họa máy tính [6].

Đề tương tác với hệ thống lưu trữ, RV32F cung cấp nhóm lệnh nạp và lưu trữ chuyên dụng bao gồm FLW(nạp) và FSW(lưu). Mặc dù các lệnh này di chuyển dữ liệu trực tiếp vào tập thanh ghi số thực f0 đến f31, chúng vẫn sử dụng các thanh ghi số nguyên x để tính toán địa chỉ bộ nhớ theo mô hình nạp lưu chuẩn. Sự phân tách này cho phép bộ xử lý thực hiện nạp dữ liệu số thực song song với các tính toán số nguyên, tối ưu hóa hiệu suất của đường ống và giảm thiểu xung đột dữ liệu giữa các đơn vị thực thi khác nhau trong chip.

Một thành phần quan trọng khác là nhóm lệnh so sánh và chuyển đổi, đóng vai trò cầu nối giữa thế giới số thực và số nguyên. Các lệnh so sánh như FEQ (bằng) hoặc FLT (nhỏ hơn) sẽ kiểm tra điều kiện giữa hai toán hạng số thực và trả về kết quả logic 0 hoặc 1 vào tập thanh ghi số nguyên. Kết quả này sau đó được sử dụng bởi các lệnh

nhánh của CPU để điều khiển luồng chương trình. Ngoài ra, các lệnh chuyển đổi kiểu dữ liệu cho phép hệ thống linh hoạt thay đổi định dạng giữa số nguyên và số thực IEEE 754, đảm bảo tính tương thích khi xử lý các thuật toán hỗn hợp.

2.1.3.3. Tập thanh ghi số thực

Phần mở rộng RV32F bổ sung một tập thanh ghi dấu phẩy động riêng biệt gồm 32 thanh ghi 32 bit, ký hiệu từ f0 đến f31, hoàn toàn độc lập với tập thanh ghi số nguyên x0 đến x31. Khác với thanh ghi x0 luôn bằng 0 trong tập số nguyên, tất cả 32 thanh ghi f0–f31 đều là thanh ghi đa năng, không có thanh ghi nào được nối cứng với giá trị cố định. Mỗi thanh ghi lưu trữ một giá trị dấu phẩy động đơn chính xác 32 bit theo chuẩn IEEE 754 như đã trình bày ở mục 2.1.3.1. Tập thanh ghi này hỗ trợ đọc đồng thời hai toán hạng (rs1, rs2) và ghi một kết quả (rd) mỗi chu kỳ, tương tự cơ chế hoạt động của tập thanh ghi số nguyên. Về tương tác với bộ nhớ, dữ liệu số thực chỉ có thể được nạp vào tập thanh ghi FP thông qua lệnh FLW và lưu ra bộ nhớ thông qua lệnh FSW. Trong cả hai trường hợp, địa chỉ bộ nhớ được tính dựa trên thanh ghi số nguyên cộng với độ lệch, điều này có nghĩa là các lệnh FLW và FSW vẫn phụ thuộc vào tập thanh ghi số nguyên để tính toán địa chỉ. Ngoài ra, một số lệnh chuyển đổi kiểu dữ liệu như FCVT.S.W và FMV.W.X đọc từ tập thanh ghi số nguyên và ghi vào tập thanh ghi số thực, tạo nên sự tương tác hai chiều giữa hai tập thanh ghi.

2.2. KIẾN TRÚC ĐƯỜNG ỐNG 5 TẦNG

2.2.1. Tổng quan về kỹ thuật đường ống

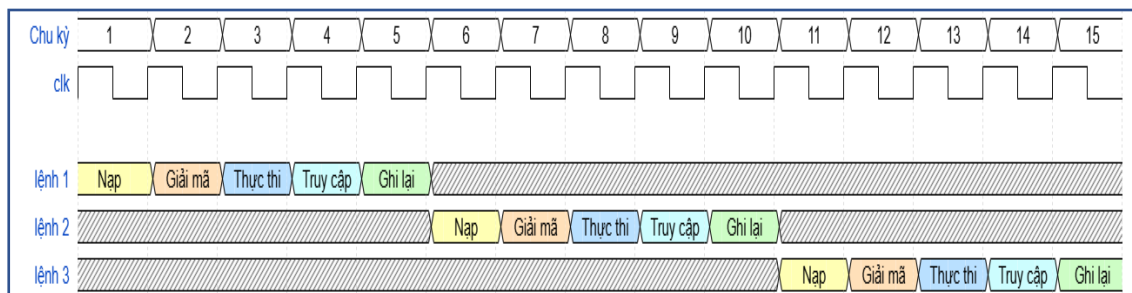
Kỹ thuật đường ống là một phương pháp triển khai kiến trúc phần cứng cho phép thực thi đồng thời nhiều lệnh bằng cách chia quá trình xử lý của một lệnh thành các giai đoạn độc lập. Trong một bộ vi xử lý không sử dụng đường ống, một lệnh mới chỉ được nạp sau khi lệnh hiện tại đã hoàn tất toàn bộ chu kỳ. Ngược lại, với kỹ thuật đường ống, ngay khi giai đoạn đầu tiên của một lệnh kết thúc, tầng phần cứng đó sẽ ngay lập tức nạp lệnh kế tiếp mà không cần chờ đợi các giai đoạn sau của lệnh trước hoàn thành.

Bản chất của kỹ thuật này dựa trên nguyên lý tính song song mức lệnh. Việc chia nhỏ luồng xử lý giúp tận dụng tối đa các tài nguyên phần cứng khác nhau trong chip tại cùng một thời điểm. Mục tiêu chính của đường ống không phải là rút ngắn thời

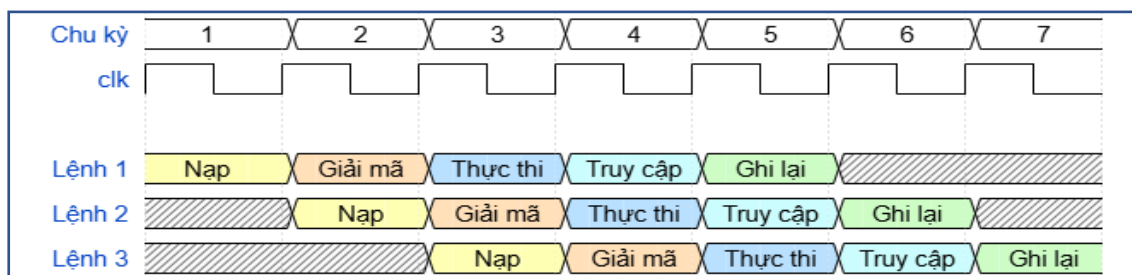
gian thực thi của một lệnh đơn lẻ, mà là nhằm tối ưu hóa lưu lượng lệnh được hoàn tất trong một đơn vị thời gian.

2.2.2. Đường ống 5 tầng

Trong kiến trúc RISC-V, mô hình 5 tầng là kiểu đường ống cổ điển [4], giúp tăng hiệu suất thông qua việc chồng chéo các bước thực thi và cải thiện thông lượng hệ thống. Mỗi giai đoạn xử lý một phần nhỏ của lệnh và mỗi chu kỳ đồng hồ đưa lệnh tiếp theo vào đường ống, giúp xử lý nhiều lệnh song song.



Hình 2.5: Thực thi tuần tự không có đường ống



Hình 2.6: Thực thi song song có đường ống

Ở hình 2.5 và hình 2.6, điều dễ thấy nhất là sự chênh lệch về thời gian hoàn thành: cùng 3 lệnh, thực thi tuần tự cần 15 chu kỳ trong khi thực thi song song có đường ống chỉ cần 7 chu kỳ. Ở hình 2.5, mỗi hàng lệnh hoạt động riêng lẻ với phần lớn thời gian là đường gạch chéo - tức phần cứng đang chờ không làm gì. Ở hình 2.6, các hàng lệnh chồng lấp lên nhau theo đường chéo, cho thấy nhiều tầng hoạt động đồng thời trong cùng một chu kỳ. Việc triển khai đường ống không giúp rút ngắn thời gian xử lý của một lệnh đơn lẻ, mà nó làm tăng đáng kể lưu lượng lệnh được xử lý trên một đơn vị thời gian.

2.2.3. Ưu và nhược điểm đường ống 5 tầng

Việc chuyển đổi từ kiến trúc thực thi đơn chu kỳ sang kiến trúc đường ống 5 tầng mang lại sự cải thiện vượt trội về hiệu suất tổng thể của bộ vi xử lý. Bằng cách cho

phép tối đa năm lệnh được thực thi đồng thời tại các giai đoạn khác nhau, kỹ thuật này tối ưu hóa lưu lượng lệnh hoàn tất trên một đơn vị thời gian, đưa số chu kỳ trên mỗi lệnh tiệm cận mức lý tưởng bằng 1. Sự đột phá này xuất phát từ việc khai thác tối đa tính song song mức lệnh, giúp các đơn vị chức năng như ALU, bộ nhớ và tập thanh ghi hoạt động liên tục thay vì phải rơi vào trạng thái nhàn rỗi để chờ đợi một lệnh duy nhất hoàn tất toàn bộ chu kỳ xử lý. Qua đó, tài nguyên phần cứng được tận dụng triệt để, giúp hệ thống đạt được hiệu năng cao nhất trên cùng một diện tích linh kiện bán dẫn.

Tuy nhiên, việc gia tăng hiệu suất cũng đi kèm với những thách thức lớn về mặt thiết kế và quản lý vi kiến trúc. Khi các lệnh được thực thi xen kẽ và chồng chéo, việc duy trì tính toàn vẹn dữ liệu và điều khiển luồng trở nên phức tạp hơn đáng kể. Để đảm bảo sự đồng bộ giữa các tầng, hệ thống bắt buộc phải tích hợp các thanh ghi đường ống nhằm lưu trữ trạng thái trung gian của lệnh qua từng chu kỳ xung nhịp. Sự phức tạp này không chỉ nằm ở việc tiêu tốn thêm tài nguyên phần cứng mà còn đòi hỏi các khối logic điều khiển phải có khả năng nhận diện và xử lý các loại xung đột phát sinh.

Sự xuất hiện của các xung đột dữ liệu, xung đột cấu trúc và xung đột điều khiển là hệ quả tất yếu của một hệ thống vận hành song song nhưng phụ thuộc lẫn nhau. Nếu các rủi ro kỹ thuật này không được giải quyết thông qua các cơ chế như chen chu kỳ đợi hoặc chuyển tiếp dữ liệu, chúng sẽ trực tiếp gây ra hiện tượng "nghẽn" đường ống. Khi đó, thời gian thực thi của lệnh sẽ bị kéo dài, làm giảm hiệu suất thực tế và làm mất đi lợi thế vốn có của kỹ thuật đường ống. Do đó, việc thiết kế một bộ xử lý tích hợp đường ống như RISC-V không chỉ đơn thuần là phân chia các giai đoạn xử lý, mà còn là quá trình tối ưu hóa các giải pháp phần cứng để giảm thiểu tác động tiêu cực của các xung đột này lên toàn bộ hệ thống.

2.2.4. Xung đột trong đường ống

2.2.4.1. Xung đột dữ liệu

Xung đột dữ liệu là hiện tượng một lệnh trong đường ống không thể thực thi đúng chu kỳ dự kiến do các toán hạng cần thiết chưa sẵn sàng. Loại xung đột này phát sinh từ sự phụ thuộc dữ liệu giữa các lệnh được thực thi gần nhau, khi lệnh sau yêu cầu

dữ liệu từ kết quả của lệnh trước đó vốn vẫn còn nằm trong các tầng xử lý của đường ống.

Ví dụ về sự phụ thuộc giữa hai lệnh kế tiếp nhau:

- ADD x19, x0, x1 (kết quả được tính toán tại tầng xử lý và ghi lại tại tầng ghi lại)
- SUB x2, x19, x3 (toán hạng x19 cần được sẵn sàng tại đầu vào tầng xử lý)

Trong cấu trúc 5 tầng tiêu chuẩn, lệnh ADD chỉ thực hiện ghi kết quả vào tập thanh ghi tại giai đoạn thứ năm (ghi lại). Tuy nhiên, lệnh SUB ngay sau đó lại yêu cầu dữ liệu từ thanh ghi x19 ngay tại giai đoạn thứ ba (thực thi) để thực hiện phép trừ. Nếu không có cơ chế can thiệp, lệnh SUB sẽ sử dụng giá trị cũ của x19, dẫn đến sai sót nghiêm trọng trong kết quả tính toán của toàn bộ chương trình.

Để khắc phục tình trạng này mà không làm giảm hiệu suất hệ thống bằng cách chèn các chu kỳ đợi, kỹ thuật chuyển tiếp dữ liệu được áp dụng như một giải pháp tối ưu. Cơ chế này cho phép trích xuất kết quả tính toán trực tiếp từ các thanh ghi đường ống ngay khi chúng vừa được tạo ra, sau đó đưa ngược trở lại đầu vào của đơn vị ALU cho lệnh kế tiếp.

Về mặt kiến trúc, một đơn vị điều khiển chuyển tiếp được tích hợp để giám sát sự trùng lặp giữa chỉ số thanh ghi đích của các lệnh đang ở tầng thực thi và tầng truy xuất bộ nhớ hoặc truy xuất bộ nhớ và tầng ghi lại với chỉ số thanh ghi nguồn của lệnh đang ở tầng giải mã lệnh và tầng thực thi. Cụ thể:

- Xung đột dữ liệu tại tầng thực thi: đây là cơ chế cho phép kết quả tính toán được chuyển tiếp trực tiếp từ thanh ghi đường ống nằm giữa tầng thực thi và tầng truy cập bộ nhớ đến đầu vào của đơn vị xử lý số học và logic cho lệnh kế tiếp, trong trường hợp lệnh đứng trước đó là một phép toán số học đã có kết quả sẵn sàng.
- Xung đột dữ liệu tại tầng bộ nhớ: đây là cơ chế cho phép kết quả được chuyển tiếp trực tiếp từ thanh ghi đường ống nằm giữa tầng truy cập bộ nhớ và tầng ghi lại đến đầu vào của đơn vị xử lý số học và logic cho lệnh kế tiếp, áp dụng khi lệnh đứng trước là một lệnh nạp dữ liệu từ bộ nhớ hoặc một phép toán số học đã hoàn tất giai đoạn truy xuất dữ liệu.

Việc triển khai đơn vị chuyển tiếp dữ liệu giúp loại bỏ hầu hết các yêu cầu tạm dừng đường ống do xung đột dữ liệu phát sinh, từ đó đảm bảo luồng thực thi các lệnh luôn được duy trì liên tục và đạt hiệu suất tối ưu. Tuy nhiên, hệ thống vẫn tồn tại một số trường hợp đặc biệt không thể xử lý hoàn toàn bằng kỹ thuật chuyển tiếp, điển hình là khi một lệnh nạp dữ liệu từ bộ nhớ được theo sau trực tiếp bởi một lệnh khác sử dụng chính kết quả của phép nạp đó làm toán hạng đầu vào. Trong tình huống này, do dữ liệu từ bộ nhớ chỉ thực sự sẵn sàng sau khi kết thúc giai đoạn truy cập bộ nhớ, đơn vị điều khiển bắt buộc phải chèn thêm một chu kỳ đợi để trì hoãn lệnh kế tiếp cho đến khi dữ liệu được nạp xong và có thể chuyển tiếp tới đầu vào của ALU.

2.2.4.2. Xung đột điều khiển

Xung đột điều khiển, hay còn gọi là xung đột rẽ nhánh, phát sinh khi bộ vi xử lý không thể xác định chính xác lệnh kế tiếp cần nạp vào đường ống do kết quả của một lệnh thay đổi luồng chương trình (như lệnh rẽ nhánh hoặc lệnh nhảy) chưa được tính toán xong. Trong kiến trúc đường ống 5 tầng, việc nạp lệnh diễn ra liên tục tại mỗi chu kỳ để duy trì hiệu suất. Tuy nhiên, đối với các lệnh rẽ nhánh có điều kiện, quyết định nhảy đến địa chỉ mới hay tiếp tục luồng tuần tự thường chỉ được xác định muộn tại tầng thực thi hoặc tầng truy cập bộ nhớ. Hệ quả là các lệnh nằm ngay sau lệnh rẽ nhánh có thể đã được nạp và bắt đầu xử lý sai trước khi đích đến thực sự của chương trình được xác nhận.

Để giảm thiểu tác động tiêu cực của xung đột điều khiển lên hiệu năng hệ thống, các kỹ thuật xử lý sau thường được áp dụng trong thiết kế vi kiến trúc:

- Tạm dừng đường ống: đây là phương án đơn giản nhất bằng cách đình chỉ việc nạp các lệnh mới cho đến khi kết quả rẽ nhánh được xác định rõ ràng. Mặc dù đảm bảo tính chính xác tuyệt đối, phương pháp này gây lãng phí nhiều chu kỳ xung nhịp và làm giảm đáng kể chỉ số lệnh mỗi chu kỳ xung nhịp của bộ xử lý.
- Dự đoán không rẽ nhánh: hệ thống mặc định giả định rằng điều kiện rẽ nhánh sẽ không xảy ra và tiếp tục nạp các lệnh tuần tự kế tiếp. Nếu dự đoán đúng, đường ống sẽ vận hành với tốc độ tối đa mà không bị gián đoạn. Trong trường hợp dự đoán sai, các lệnh đã nạp sai mục tiêu phải được loại bỏ hoàn toàn khỏi

đường ống bằng cách xóa các thanh ghi đường ống và đặt các tín hiệu điều khiển về trạng thái nghỉ, sau đó nạp lại lệnh từ địa chỉ đích chính xác.

- Tối ưu hóa thời điểm quyết định: một kỹ thuật hiệu quả khác nhằm giảm thiểu tác động của xung đột điều khiển là di chuyển toàn bộ logic tính toán địa chỉ đích và kiểm tra điều kiện rẽ nhánh từ tầng thực thi lên tầng giải mã lệnh. Bằng cách tích hợp bổ sung một bộ cộng địa chỉ chuyên dụng và mạch so sánh dữ liệu ngay tại giai đoạn giải mã, bộ vi xử lý có khả năng xác định sớm luồng thực thi của chương trình. Giải pháp này cho phép hệ thống nhận biết kết quả rẽ nhánh ngay từ chu kỳ thứ hai của đường ống, từ đó giảm số lượng chu kỳ xung nhịp bị lãng phí do phải hủy bỏ lệnh từ mức 2-3 chu kỳ xuống chỉ còn duy nhất 1 chu kỳ.

2.3. PHƯƠNG PHÁP XÁC MINH BẰNG MÔI TRƯỜNG UVM

2.3.1. Các thành phần trong UVM

Môi trường xác minh UVM được xây dựng dựa trên kiến trúc hướng đối tượng bằng ngôn ngữ SystemVerilog, phân tách rõ ràng giữa các thành phần tạo dữ liệu, điều khiển và giám sát. Cấu trúc này không chỉ giúp quản lý độ phức tạp của các thiết kế vi mạch hiện đại mà còn cho phép tái sử dụng các thành phần kiểm thử trên nhiều dự án khác nhau thông qua tính kế thừa và đóng gói [7].

Đơn vị cơ bản nhất trong luồng dữ liệu UVM là UVM Sequence Item, một lớp đối tượng định nghĩa cấu trúc của gói dữ liệu giao dịch. Thành phần này chứa các trường dữ liệu và phương thức cần thiết để mô phỏng các gói tin hoặc lệnh mà thiết kế cần xử lý. Đi kèm với đó là UVM Sequence, đóng vai trò là kịch bản kiểm thử động, chịu trách nhiệm tạo ra chuỗi các Sequence Item dựa trên các thuật toán ngẫu nhiên hoặc có định hướng. Việc tách biệt lớp dữ liệu này khỏi cấu trúc phần cứng của môi trường giúp người thiết kế linh hoạt thay đổi các kịch bản kiểm thử mà không cần tái cấu trúc lại toàn bộ hệ thống.

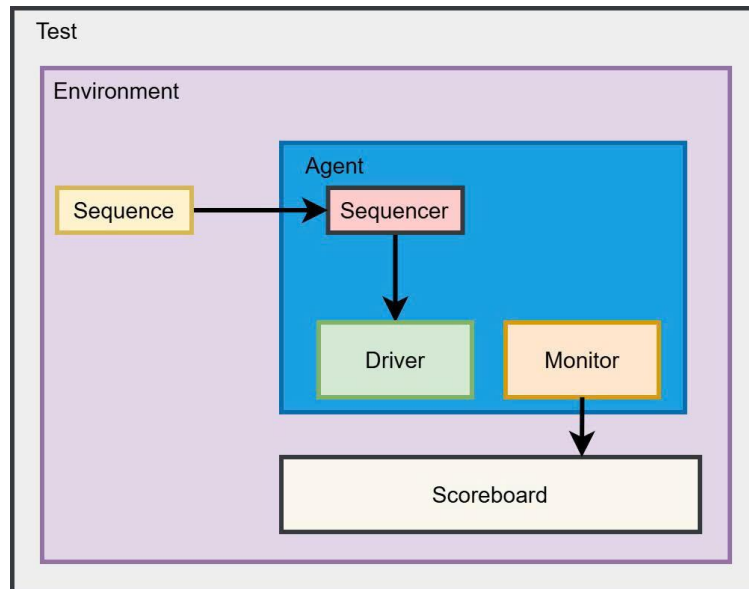
Để điều phối luồng dữ liệu từ các kịch bản kiểm thử, UVM Sequencer được triển khai như một bộ trung chuyển và quản lý hàng đợi. Nhiệm vụ trọng tâm của sequencer là tiếp nhận các yêu cầu từ sequence và cung cấp các gói dữ liệu cho driver khi có yêu cầu. Sự hiện diện của sequencer tạo ra một lớp đệm logic, giúp cách ly hoàn toàn

giữa tầng tạo kịch bản và tầng thực thi vật lý. Cơ chế này cho phép môi trường kiểm thử duy trì tính ổn định, hỗ trợ việc điều phối nhiều luồng dữ liệu đồng thời trên cùng một giao diện điều khiển.

Thành phần chịu trách nhiệm chuyển đổi các giao dịch từ mức logic sang các tín hiệu vật lý là UVM Driver. Driver nhận các gói dữ liệu từ sequencer thông qua cổng kết nối TLM và thực hiện quá trình giải mã để tác động trực tiếp lên các chân tín hiệu của thiết kế cần kiểm thử. Driver vận hành dựa trên các giao thức thời gian cụ thể, đóng vai trò như một bộ mô phỏng hành vi của các tác nhân bên ngoài khi tương tác với thiết kế, đảm bảo sự đồng bộ về xung nhịp và pha của tín hiệu.

Khác với driver, UVM Monitor là một thành phần thụ động liên tục quan sát các hoạt động trên giao diện của thiết kế mà không tác động vào luồng tín hiệu. Monitor thu thập các mức logic từ các chân tín hiệu, sau đó đóng gói ngược lại thành các giao dịch để gửi đến các khối phân tích cấp cao hơn. Một môi trường UVM tiêu chuẩn thường bố trí các monitor tại các điểm giao tiếp trọng yếu để ghi nhận chính xác dữ liệu đầu vào và phản hồi đầu ra, phục vụ cho việc kiểm tra tính đúng đắn và thu thập số liệu về độ bao phủ của quá trình xác minh.

Tổng quan về kiến trúc môi trường UVM như hình 2.7 bao gồm các thành phần driver, sequencer và monitor thường được đóng gói một cách có hệ thống bên trong UVM Agent. Agent đại diện cho một giao diện chức năng cụ thể của thiết kế, cho phép người dùng cấu hình linh hoạt giữa chế độ hoạt động đầy đủ hoặc chỉ thực hiện giám sát. Việc sử dụng agent giúp chuẩn hóa kiến trúc môi trường, tạo điều kiện thuận lợi cho việc quản lý các thành phần theo từng khối chức năng riêng biệt và tăng khả năng mở rộng của hệ thống xác minh.

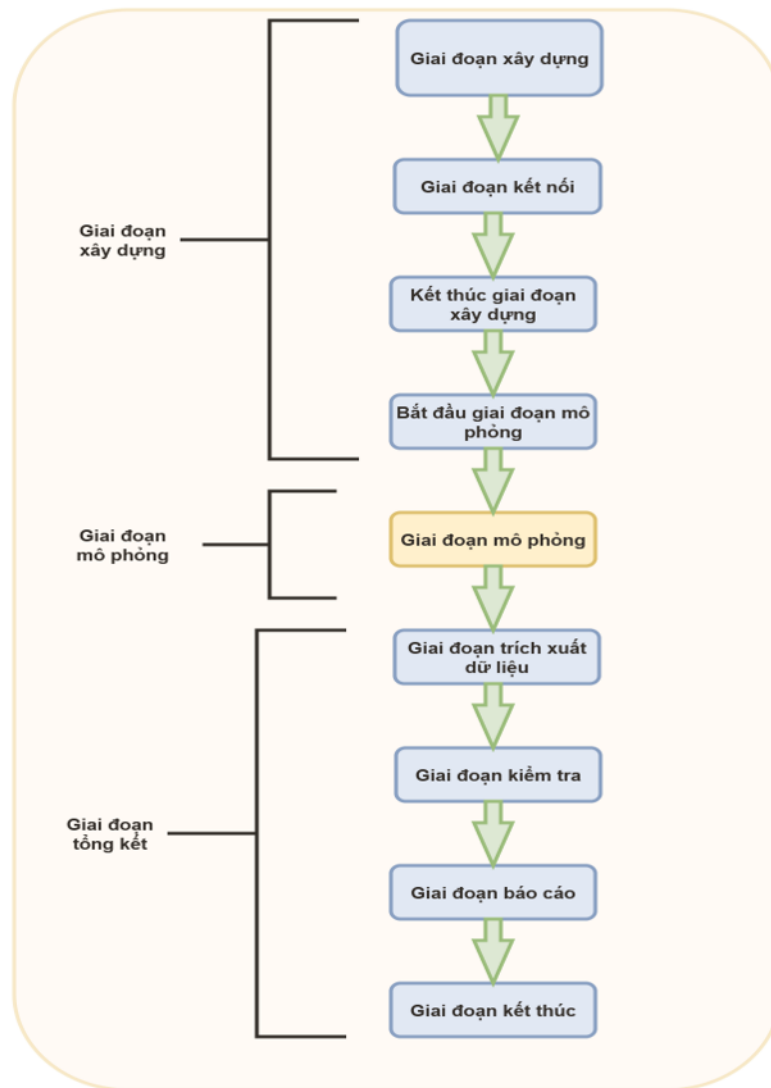


Hình 2.7: Kiến trúc tổng quan của môi trường UVM

Cuối cùng, thành phần quyết định kiểm tra tính đúng đắn chính là UVM Scoreboard. Đây là thành phần thu thập dữ liệu từ các monitor để đối chiếu kết quả thực thi của thiết kế với kết quả mong đợi từ một mô hình tham chiếu. Mọi sai lệch về dữ liệu hoặc lỗi giao thức đều được scoreboard phát hiện và báo cáo. Toàn bộ các thành phần này được bao bọc bởi UVM Environment và được điều phối bởi UVM Test, tạo thành một hệ sinh thái xác minh hoàn chỉnh và khép kín, đảm bảo thiết kế đạt được độ tin cậy cao nhất trước khi tiến hành tổng hợp.

2.3.2. Trình tự vận hành trong UVM

Trình tự vận hành của UVM được chia thành ba nhóm giai đoạn chính như hình 2.8: giai đoạn xây dựng, giai đoạn thực thi và giai đoạn tổng kết. Việc phân chia này giúp kiểm soát vòng đời của các đối tượng từ lúc bắt đầu mô phỏng cho đến khi xuất báo cáo kết quả cuối cùng.



Hình 2.8: Trình tự vận hành trong UVM

Giai đoạn xây dựng và kết nối. Đây là nhóm các giai đoạn chạy bằng logic tổ hợp (không tiêu tốn thời gian mô phỏng). Đầu tiên là giai đoạn xây dựng, nơi các thành phần trong môi trường như agent, driver, monitor được khởi tạo và cấu hình dựa trên cơ chế `uvm_config_db`. Ngay sau đó là giai đoạn kết nối, giai đoạn thiết lập các liên kết vật lý và logic giữa các thành phần thông qua các cổng TLM. Cuối cùng là kết thúc giai đoạn xây dựng, cho phép người thiết kế thực hiện các kiểm tra cuối cùng về cấu trúc và sự kết nối trước khi bắt đầu quá trình thực thi thực sự.

Khác với các giai đoạn khác, giai đoạn chạy là giai đoạn duy nhất tiêu tốn thời gian mô phỏng và chạy dưới dạng một tiến trình song song. Đây là thời điểm các kịch bản kiểm thử bắt đầu hoạt động, tạo ra dữ liệu để driver đưa vào thiết kế, đồng thời các monitor cũng bắt đầu thu thập dữ liệu phản hồi. UVM cung cấp cơ chế phản đối trong

giai đoạn này để kiểm soát thời điểm kết thúc mô phỏng: quá trình chạy sẽ chỉ dừng lại khi tất cả các thành phần đã hoàn thành nhiệm vụ và rút lại các phản đối của mình.

Sau khi giai đoạn thực thi kết thúc, UVM chuyển sang nhóm các giai đoạn hậu xử lý. Giai đoạn trích xuất và giai đoạn kiểm tra được sử dụng để thu thập các dữ liệu còn sót lại trong bộ nhớ đệm và thực hiện đối chiếu cuối cùng tại scoreboard để xác định tính đúng đắn của thiết kế. Tiếp theo, giai đoạn báo cáo sẽ tổng hợp toàn bộ kết quả, từ số lượng giao dịch đã thực hiện đến các lỗi phát sinh, để xuất ra báo cáo định dạng chuẩn. Trình tự này kết thúc bằng giai đoạn cuối, nơi thực hiện các tác vụ dọn dẹp hệ thống và đóng các tệp tin lưu trữ dữ liệu mô phỏng.

Sự phân cấp rõ ràng trong trình tự vận hành giúp người thiết kế dễ dàng quản lý luồng mô phỏng phức tạp, đảm bảo tính nhất quán của dữ liệu và khả năng tái sử dụng môi trường kiểm thử trên nhiều nền tảng khác nhau.

CHƯƠNG 3

THIẾT KẾ

3.1. YÊU CẦU CỦA HỆ THỐNG

Bộ vi xử lý được thiết kế dựa trên kiến trúc RISC-V 32 bit với tập lệnh cơ bản RV32I cho các phép toán số nguyên và RV32F cho các phép toán dấu phẩy động đơn theo chuẩn IEEE 754. Hệ thống thực thi lệnh theo cơ chế đường ống 5 tầng, trong đó mỗi tầng được cách ly bởi các thanh ghi độc lập. Thiết kế sử dụng 32 thanh ghi số nguyên từ x0 đến x31, độ rộng của mỗi thanh ghi là 32 bit, riêng thanh ghi x0 được cố định với giá trị là 32 bit 0. Để xử lý số thực, CPU có thêm một tập thanh ghi 32 bit chuyên biệt gồm 32 thanh ghi từ f0 đến f31. Toàn bộ thiết kế sẽ được mô tả bằng ngôn ngữ Verilog ở mức RTL.

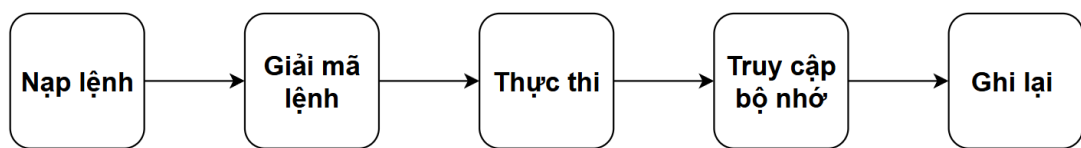
Hệ thống tính toán có bộ ALU mở rộng thêm bộ FPU tại tầng thực thi, bộ điều khiển sẽ giải mã mã lệnh để chọn đường xử lý tương ứng, sau đó kết quả giữa ALU và FPU được tín hiệu từ bộ điều khiển quyết định trước khi ghi vào thanh ghi đích.

Hệ thống được thiết kế để giải quyết hai loại xung đột chính. Đầu tiên là xung đột dữ liệu, cho phép chuyển trực tiếp kết quả từ tầng thực thi hoặc tầng truy xuất bộ nhớ sang lệnh đang cần dữ liệu ở tầng trước đó mà không cần chờ đến tầng ghi kết quả, qua đó giảm thiểu số chu kỳ trì hoãn. Trong trường hợp lệnh đọc bộ nhớ mà lệnh ngay sau đó phụ thuộc vào kết quả đọc, cơ chế chuyển tiếp không thể áp dụng được, khi đó khối phát hiện xung đột sẽ tự động chèn một chu kỳ trì hoãn bằng cách giữ nguyên trạng thái các tầng trước và chèn lệnh rỗng vào tầng thực thi. Thứ hai, đối với xung đột điều khiển khi lệnh rẽ nhánh được thực hiện, hệ thống sẽ xóa các lệnh đã nạp sai trong tầng nạp lệnh và tầng giải mã bằng cách ghi đè giá trị rỗng vào các thanh ghi đường ống tương ứng, đồng thời nạp lại lệnh đúng từ địa chỉ đích của nhánh.

Việc kiểm thử sẽ được thực hiện ở mức độ mô hình hành vi cho từng khối để đảm bảo chức năng và kiểm thử hệ thống với môi trường UVM. Việc thiết kế môi trường UVM phải đảm bảo môi trường kiểm tra được đầy đủ các tính năng của CPU, kết quả báo cáo trình bày từ môi trường phải tường minh, dễ dàng theo dõi.

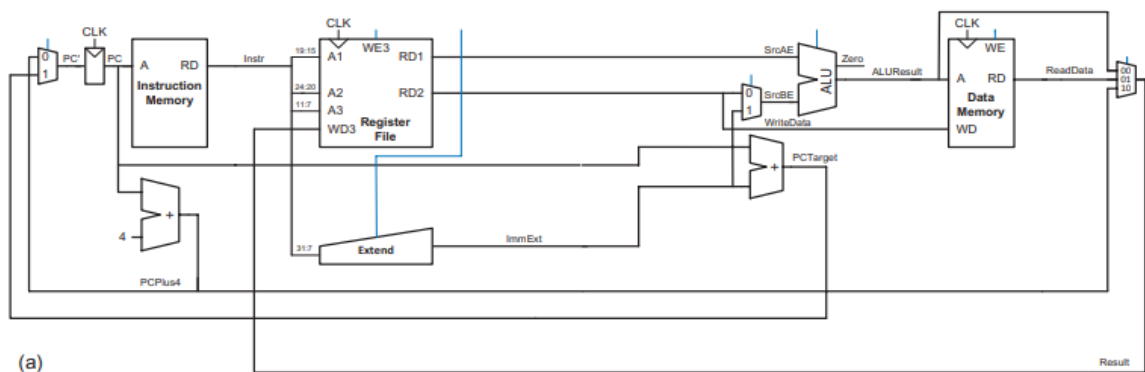
3.2. SƠ ĐỒ KHỐI HỆ THỐNG

Để triển khai bộ vi xử lý RISC-V 32 bit theo kiến trúc đường ống 5 tầng, nhóm thực hiện thiết kế bộ xử lý đơn chu kỳ làm cơ sở ban đầu, với trình tự thực thi các lệnh được trình bày trong hình 3.1. Ở kiến trúc này, toàn bộ quá trình thực thi một lệnh, từ giai đoạn lấy lệnh, giải mã, thực hiện phép toán, truy cập bộ nhớ cho đến ghi kết quả trở lại tập thanh ghi đều được hoàn thành trong một chu kỳ xung nhịp duy nhất. Do đó, các lệnh được xử lý theo cơ chế tuần tự, trong đó một lệnh chỉ được thực hiện sau khi lệnh trước đó đã hoàn tất toàn bộ các giai đoạn xử lý.



Hình 3.1: Trình tự thực thi của bộ vi xử lý đơn chu kỳ

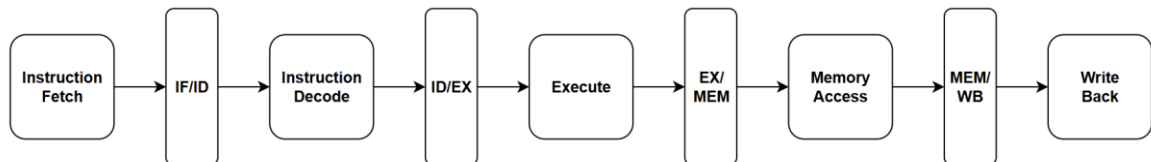
Về mặt cấu trúc, bộ xử lý đơn chu kỳ bao gồm các thành phần chức năng cơ bản như bộ đếm chương trình, khối điều khiển, tập thanh ghi gồm 32 thanh ghi đa dụng 32 bit, bộ mở rộng giá trị tức thời, khối ALU, bộ cộng, bộ nhớ dữ liệu và các bộ chọn kênh như hình 3.2. Sự phối hợp giữa các khối chức năng này cho phép hệ thống thực hiện các thao tác xử lý dữ liệu, tính toán địa chỉ, điều khiển luồng chương trình và truy xuất bộ nhớ theo đúng đặc tả kiến trúc.



Hình 3.2: Sơ đồ khối của bộ xử lý đơn chu kỳ [1]

Từ mô hình đơn chu kỳ ban đầu, các khối chức năng cần thiết cho một CPU đã được xây dựng để có thể thực hiện được các lệnh trong tập lệnh RV32I. Nhóm thực

hiện đề tài tiến hành thiết kế kiến trúc sang dạng đường ống 5 tầng nhằm nâng cao hiệu năng xử lý. Việc chuyển đổi này được thực hiện bằng cách phân tách quá trình thực thi lệnh thành năm tầng độc lập như hình 3.3: tầng nạp lệnh, tầng giải mã lệnh, tầng thực thi, tầng truy cập bộ nhớ và tầng ghi kết quả. Các thanh ghi bao gồm IF/ID, ID/EX, EX/MEM và MEM/WB được bổ sung vào giữa các tầng xử lý. Các thanh ghi này có vai trò lưu giữ lệnh, dữ liệu toán hạng, địa chỉ, giá trị tức thì đã mở rộng dấu và các tín hiệu điều khiển tương ứng trong suốt quá trình dịch chuyển qua từng tầng.



Hình 3.3: Tất cả các giai đoạn trong bộ vi xử lý kiến trúc đường ống

Tầng đầu tiên của đường ống là tầng nạp lệnh, tại tầng này, thanh ghi PC cung cấp địa chỉ lệnh hiện tại đến bộ nhớ lệnh, bộ nhớ lệnh trả về mã lệnh 32 bit tương ứng. Đồng thời, bộ cộng tính giá trị $PC + 4$ để chuẩn bị địa chỉ lệnh kế tiếp.

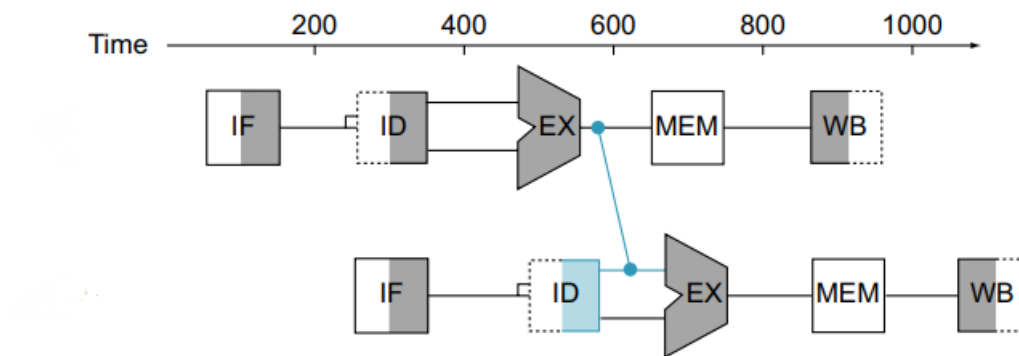
Tiếp theo, tầng giải mã lệnh thực hiện đồng thời hai nhiệm vụ chính là Control Unit nhận mã lệnh, funct3 và funct7 từ mã lệnh để giải mã và tạo ra toàn bộ tín hiệu điều khiển cần thiết cho các tầng phía sau, tập thanh ghi đọc đồng thời hai thanh ghi nguồn Rs1 và Rs2 theo địa chỉ được trích từ các trường bit tương ứng trong mã lệnh.

Sau khi đã giải mã lệnh, tầng thực thi là nơi diễn ra các phép tính toán chính của bộ xử lý, ALU thực hiện phép toán số học hoặc logic theo mã điều khiển và kết quả của ALU, dữ liệu ghi bộ nhớ và các tín hiệu điều khiển còn lại được chốt vào thanh ghi EX/MEM. Đối với các lệnh như nạp hay lưu dữ liệu sẽ được xử lý tại tầng truy cập bộ nhớ, dữ liệu sau đó sẽ được đẩy vào thanh ghi MEM/WB để ghi kết quả vào tập thanh ghi.

So với kiến trúc đơn chu kỳ, mô hình đường ống không làm thay đổi bản chất hoạt động của các khối chức năng chính, mà chủ yếu tái tổ chức lại đường dữ liệu để nhiều lệnh có thể được xử lý đồng thời tại các tầng khác nhau. Nhờ đó, trong cùng một chu kỳ xung nhịp, bộ xử lý có thể vừa lấy lệnh mới, vừa giải mã lệnh trước đó, đồng thời thực hiện tính toán hoặc truy cập bộ nhớ cho các lệnh khác đang tồn tại trong đường

ống. Cách tổ chức này làm tăng thông lượng xử lý lệnh của hệ thống, đồng thời cho phép rút ngắn chu kỳ xung nhịp so với kiến trúc đơn chu kỳ.

Bên cạnh đó, khi áp dụng kiến trúc đường ống, hệ thống bắt đầu xuất hiện các dạng xung đột có thể ảnh hưởng đến tính đúng đắn của quá trình thực thi, đặc biệt là xung đột dữ liệu giữa các lệnh phụ thuộc nhau. Để giải quyết vấn đề này, thiết kế cần tích hợp thêm các cơ chế hỗ trợ như chuyển tiếp nhằm chuyển trực tiếp kết quả từ các tầng sau về tầng thực thi, cũng như khối phát hiện xung đột để chèn trạng thái chờ khi cần thiết. Theo như hình 3.4, tại thời điểm 600, lệnh đầu tiên đang ở tầng thực thi tính toán kết quả, lệnh 2 đang ở tầng giải mã đọc thanh ghi chuẩn bị thực thi, đường nối giữa 2 lệnh thể hiện kết quả sau khi thực thi của lệnh 1 sẽ được chuyển trực tiếp sang đầu vào của lệnh 2, không cần chờ lệnh 1 ghi vào thanh ghi tầng ghi lại.

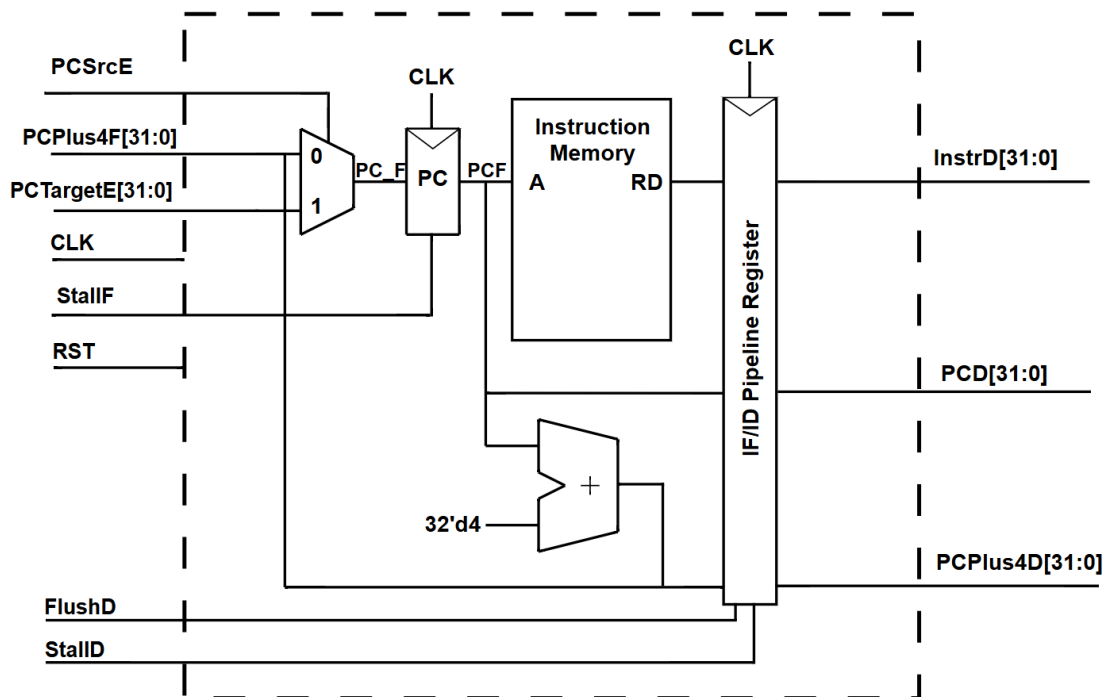


Hình 3.4: Cơ chế hỗ trợ chuyển tiếp tránh xung đột [1]

3.3. THIẾT KẾ CHI TIẾT CỦA HỆ THỐNG

3.3.1. Khối nạp lệnh

Khối nạp lệnh bao gồm bốn khối con: bộ đếm chương trình lưu địa chỉ lệnh hiện tại, bộ nhớ lệnh cung cấp mã lệnh 32 bit theo địa chỉ đó, bộ cộng tính song song địa chỉ lệnh kế tiếp và bộ chọn kênh quyết định giá trị nạp vào bộ đếm chương trình ở chu kỳ tiếp theo. Đầu ra của tầng được chốt qua thanh ghi đường ống IF/ID trước khi chuyển sang tầng giải mã.



Hình 3.5: Sơ đồ khối tầng nạp lệnh

Các tín hiệu vào và ra của tầng nạp lệnh được mô tả trong bảng 3.1. Tín hiệu PCSrcE điều khiển bộ chọn kênh với hai giá trị 0 và 1 được truyền vào, bộ chọn kênh sẽ chọn PCPlus4F và giá trị PC+4 từ bộ cộng làm địa chỉ lệnh tiếp theo nếu giá trị đó là 0. Khi PCSrcE bằng 1, bộ chọn kênh chọn PCTargetE làm địa chỉ đích của lệnh rẽ nhánh do tầng thực thi cung cấp. Tín hiệu StallF giữ nguyên giá trị bộ đếm chương trình khi phát hiện nguy cơ xung đột, trong khi RST đưa bộ đếm chương trình về 0 khi khởi động lại hệ thống.

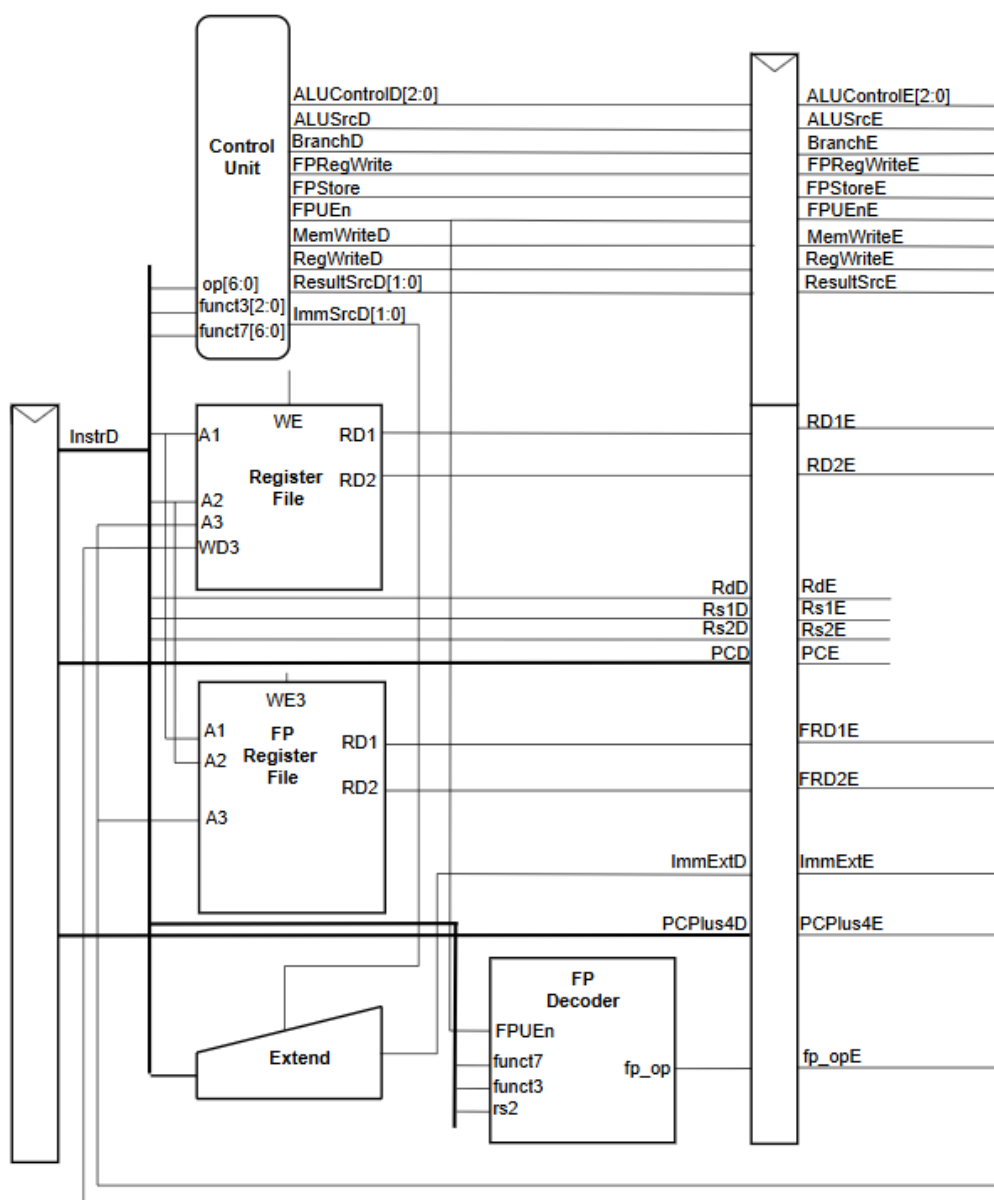
Tại mỗi cạnh lên của CLK, bộ đếm chương trình cập nhật giá trị mới, bộ nhớ lệnh xuất mã lệnh 32 bit tương ứng, đồng thời bộ cộng tính sẵn PC+4. Ba giá trị gồm InstrD, PCD và PCPlus4D được ghi vào thanh ghi IF/ID để chuyển sang tầng giải mã. Khi rẽ nhánh xảy ra, tín hiệu FlushD xóa thanh ghi IF/ID về 0, hủy lệnh đã nạp ở chu kỳ trước vì lệnh đó được nạp theo địa chỉ PC+4 trong khi chờ tầng thực thi xác nhận kết quả rẽ nhánh. Tín hiệu StallID ngăn thanh ghi IF/ID cập nhật khi đường ống bị dừng.

Bảng 3.1: Các tín hiệu của khối nạp lệnh

Tên tín hiệu	Số bit	Ý nghĩa
PCSrcE	1	Tín hiệu chọn nguồn bộ đếm chương trình
PCPlus4F	32	Giá trị PC+4 địa chỉ lệnh kế tiếp
PCTargetE	32	Địa chỉ đích của lệnh rẽ nhánh/nhảy
CLK	1	Xung nhịp hệ thống
StallF	1	Dừng cập nhật PC
RST	1	Tín hiệu đặt lại tích cực mức thấp
FlushD	1	Xóa thanh ghi đường ống IF/ID
StallID	1	Dừng cập nhật thanh ghi đường ống IF/ID
InstrD	32	Mã lệnh 32 bit đã nạp, chuyển sang khối giải mã
PCD	32	Giá trị PC tương ứng với lệnh đang được giải mã
PCPlus4D	32	Giá trị PC+4, dùng cho lệnh nhảy lưu địa chỉ trả về

3.3.2. Khối giải mã

Khối giải mã bao gồm năm khối con: khối điều khiển phân tích mã lệnh để sinh tín hiệu điều khiển, tập thanh ghi số nguyên và tập thanh ghi dấu phẩy động cung cấp giá trị toán hạng, bộ giải mã lệnh dấu phẩy động xử lý các lệnh FPU và khối mở rộng số tức thì chuyển hằng số trong lệnh thành giá trị 32 bit. Các tín hiệu vào và ra của tầng giải mã được mô tả trong bảng 3.2. Tín hiệu đầu vào gồm hai nhóm: nhóm từ thanh ghi IF/ID mang dữ liệu về các lệnh và nhóm từ tầng ghi lại mang dữ liệu ghi ngược vào tập thanh ghi. Tín hiệu đầu ra gồm dữ liệu toán hạng, tín hiệu điều khiển và địa chỉ thanh ghi được chốt qua thanh ghi ID/EX để chuyển sang tầng thực thi.



Hình 3.6: Sơ đồ khối chi tiết của khối giải mã

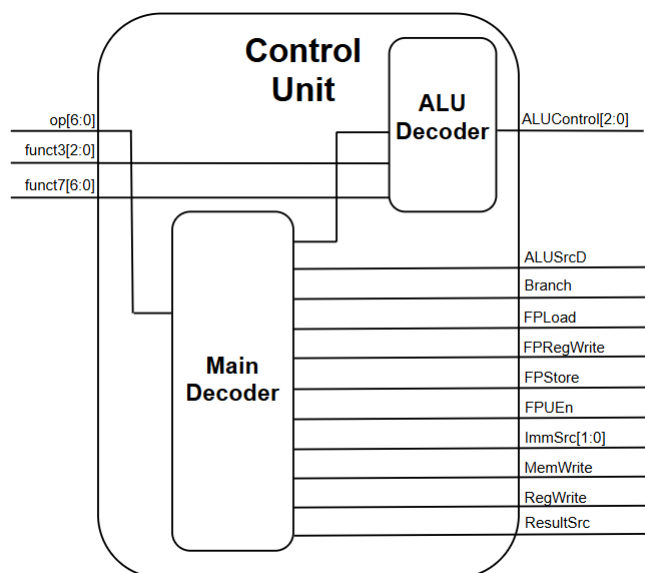
Bảng 3.2: Mô tả các tín hiệu của khối giải mã

Tên tín hiệu	Số bit	Ý nghĩa
CLK	1	Xung nhịp hệ thống
RST	1	Tín hiệu đặt lại tích cực mức thấp
FlushE	1	Xóa thanh ghi ID/EX khi rẽ nhánh hoặc xung đột nạp - sử dụng
InstrD	32	Mã lệnh 32 bit từ thanh ghi IF/ID

PCD	32	Giá trị PC của lệnh đang giải mã
PCPlus4D	32	Giá trị PC+4, dùng cho lệnh nhảy lưu địa chỉ trả về
RegWriteW	1	Tín hiệu ghi thanh ghi số nguyên từ tầng ghi lại
RDW	5	Địa chỉ thanh ghi đích từ tầng ghi lại
ResultW	32	Dữ liệu ghi vào tập thanh ghi số nguyên từ tầng ghi lại
FPRegWriteW	1	Tín hiệu ghi thanh ghi dấu phẩy động từ tầng ghi lại
FP_ResultW	32	Dữ liệu ghi vào tập thanh ghi dấu phẩy động từ tầng ghi lại
RD1_E	32	Toán hạng A từ tập thanh ghi số nguyên
RD2_E	32	Toán hạng B từ tập thanh ghi số nguyên
FRD1_E	32	Toán hạng A từ tập thanh ghi dấu phẩy động
FRD2_E	32	Toán hạng B từ tập thanh ghi dấu phẩy động
Imm_Ext_E	32	Giá trị tức thì mở rộng 32 bit
PCE	32	Giá trị PC chuyển sang tầng thực thi
PCPlus4E	32	Giá trị PC+4 chuyển sang tầng thực thi
RS1_E	5	Địa chỉ thanh ghi nguồn 1, cung cấp cho đơn vị phát hiện xung đột
RS2_E	5	Địa chỉ thanh ghi nguồn 2, cung cấp cho đơn vị phát hiện xung đột
RD_E	5	Địa chỉ thanh ghi đích, truyền qua đường ống đến tầng ghi lại
RegWriteE	1	Cho phép ghi thanh ghi số nguyên
ALUSrcE	1	Chọn toán hạng B ALU, chọn thanh ghi nếu ALUSrcE là 0 và chọn giá trị tức thì nếu ALUSrcE là 1
MemWriteE	1	Cho phép ghi bộ nhớ dữ liệu

ResultSrcE	2	Chọn nguồn kết quả ghi vào thanh ghi: 00 lấy từ ALU, 01 lấy từ bộ nhớ, 10 lấy giá trị PC+4
BranchE	1	Đánh dấu lệnh rẽ nhánh có điều kiện
ALUControlE	3	Mã phép tính cho ALU
JumpE	1	Đánh dấu lệnh nhảy không điều kiện JAL
JalrE	1	Đánh dấu lệnh nhảy gián tiếp JALR
FPUEnE	1	Kích hoạt FPU tại tầng thực thi
FPRegWriteE	1	Cho phép ghi thanh ghi dấu phẩy động
FPStoreE	1	Chọn nguồn dữ liệu ghi vào bộ nhớ: 0 lấy từ thanh ghi số nguyên, 1 lấy từ thanh ghi dấu phẩy động.
fp_opE	18	Vector one-hot chọn lệnh FP cụ thể cho FPU

Khi nhận mã lệnh 32 bit từ thanh ghi IF/ID, khối điều khiển phân tích các trường mã lệnh, funct3 và funct7 qua hai bộ giải mã con. Main Decoder nhận mã lệnh để xác định loại lệnh và xuất các tín hiệu: RegWrite cho phép ghi kết quả vào tập thanh ghi số nguyên, ALUSrc chọn toán hạng B của ALU là giá trị tức thì hay thanh ghi rs2, MemWrite cho phép ghi bộ nhớ khi thực thi lệnh SW/FSW, ResultSrc chọn giữa kết quả ALU và dữ liệu đọc từ bộ nhớ để ghi lại, Branch đánh dấu lệnh rẽ nhánh có điều kiện, ImmSrc chọn định dạng mở rộng số tức thì và ALUOp phân nhóm lệnh cho ALU Decoder. ALU Decoder nhận ALUOp, funct3 và funct7 để tạo tín hiệu ALUControl xác định phép tính cụ thể cho ALU, chi tiết theo bảng 3.3 và bảng 3.4.



Hình 3.7: Sơ đồ khối của khối điều khiển

Bảng 3.3: Bảng giải mã mã lệnh

Opcode	Loại lệnh	Reg-Write	Imm-Src	ALU-Src	Mem-Write	Result-Src	Branch	ALUOp
0110011	R	1	00	0	0	00	0	10
0010011	I (số học)	1	00	1	0	00	0	00
0000011	LW	1	00	1	0	01	0	00
0000111	FLW	0	00	1	0	01	0	00
0100011	SW	0	01	1	1	00	0	00
0100111	FSW	0	01	1	1	00	0	00
1100011	B	0	10	0	0	00	1	01
1101111	JAL	1	11	0	0	10	0	00
1100111	JALR	1	00	1	0	10	0	00
1010011	FP-op	0	00	0	0	00	0	00

Bảng 3.4: Bảng tổng hợp mã lệnh ALU

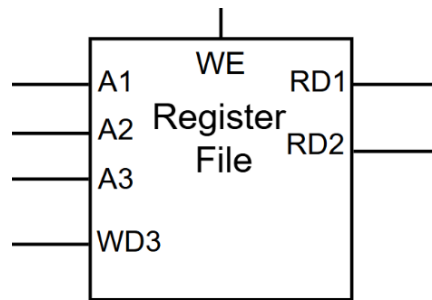
ALUOp	funct3	funct7	ALUControl	Lệnh
00	xxx	xxxxxxx	000	ADD (nạp/lưu/JALR)
01	xxx	xxxxxxx	001	SUB (rẽ nhánh BEQ)
10	000	0000000	000	ADD (R-type)
	000	0100000	001	SUB (R-type, op[5]=1)
	111	0000000	010	AND
	110	0000000	011	OR
	000	0000001	100	MUL
	010	0000000	101	SLT
	100	0000001	110	DIV
	110	0000001	111	REM

Khởi mở rộng số tức thì trích xuất hằng số từ mã lệnh và mở rộng thành 32 bit theo định dạng do ImmSrc chỉ định: lệnh loại I lấy 12 bit [31:20], lệnh loại S ghép [31:25] và [11:7], lệnh loại B ghép các bit rời rạc và dịch trái 1 bit tạo độ lệch rẽ nhánh, chi tiết theo bảng 3.5.

Bảng 3.5: Bảng mở rộng giá trị tức thì

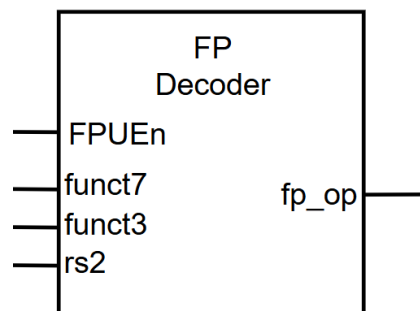
ImmSrc	Định dạng	Trường bit nguồn	Mô tả
00	Lệnh loại I	$\{20\{Instr[31]\}, Instr[31:20]\}$	Giá trị tức thì 12 bit
01	Lệnh loại S	$\{20\{Instr[31]\}, Instr[31:25], Instr[11:7]\}$	Giá trị tức thì 12 bit
10	Lệnh loại B	$\{20\{Instr[31]\}, Instr[7], Instr[30:25], Instr[11:8], 1'b0\}$	Giá trị tức thì 13 bit
11	Lệnh loại J	$\{12\{Instr[31]\}, Instr[19:12], Instr[20], Instr[30:21], 1'b0\}$	Giá trị tức thì 21 bit cho lệnh JAL

Đồng thời với quá trình giải mã, tập thanh ghi số nguyên gồm 32 thanh ghi 32 bit đọc hai toán hạng theo địa chỉ rs1 và rs2 trong mã lệnh, xuất ra RD1D và RD2D để chuyển sang tầng thực thi. Cổng ghi nhận địa chỉ đích rd và dữ liệu WD3 từ tầng ghi lại và được kích hoạt bởi RegWrite tại cạnh lên xung nhịp. Nếu địa chỉ ghi trùng với địa chỉ đọc và RegWrite đang tích cực, cơ chế chuyển tiếp nội bộ trả về giá trị mới ngay trong chu kỳ hiện tại mà không cần đợi chu kỳ kế tiếp.



Hình 3.8: Sơ đồ khối tập thanh ghi số nguyên

Khi mã lệnh thuộc nhóm lệnh dấu phẩy động, bộ giải mã lệnh FPU nhận mã lệnh, funct3 và funct7 để xuất các tín hiệu điều khiển: FPUEn kích hoạt khối tính toán FPU, FPRegWrite cho phép ghi kết quả vào tập thanh ghi dấu phẩy động, FPLoad báo hiệu lệnh FLW nạp dữ liệu từ bộ nhớ vào thanh ghi dấu phẩy động, FPStore báo hiệu lệnh FSW lưu dữ liệu từ thanh ghi dấu phẩy động xuống bộ nhớ. Tập thanh ghi dấu phẩy động gồm 32 thanh ghi 32 bit (f0–f31), hoạt động song song với tập thanh ghi số nguyên, nó đọc hai toán hạng theo rs1 và rs2, xuất FRD1D và FRD2D và có cơ chế chuyển tiếp nội bộ tương tự.



Hình 3.9: Sơ đồ khối bộ giải mã FPU

Bảng 3.6: Bảng giải mã lệnh FPU

Bit fp_op	funct5	funct3	rs2	Lệnh
0	00000	xxx	xxxxxx	FADD.S
1	00001	xxx	xxxxxx	FSUB.S
2	00010	xxx	xxxxxx	FMUL.S
3	00011	xxx	xxxxxx	FDIV.S
6	10100	010	xxxxxx	FEQ.S
7	10100	001	xxxxxx	FLT.S
8	10100	000	xxxxxx	FLE.S
11	11100	000	00000	FMV.X.W
12	11110	000	00000	FMV.W.X
13	11000	xxx	00000	FCVT.W.S
15	11010	xxx	00000	FCVT.S.W

3.3.3. Khối thực thi

Khối thực thi bao gồm bảy khối con: hai bộ chọn kênh chuyển tiếp dữ liệu giải quyết xung đột dữ liệu bằng cách chọn toán hạng mới nhất, bộ chọn kênh ALUSrc chọn toán hạng B cuối cùng cho ALU, khối ALU số nguyên thực hiện phép tính, khối FPU xử lý lệnh dấu phẩy động, bộ chọn kênh kết quả chọn giữa đầu ra ALU hoặc FPU và bộ cộng tính địa chỉ rẽ nhánh đích. Toàn bộ kết quả được chốt qua thanh ghi đường ống EX/MEM để chuyển sang tầng truy cập bộ nhớ.

nhánh. Khi FPUEnE bằng 1, FPU nhận FRD1_E, FRD2_E cùng mã hóa one-hot fp_opE chọn lệnh FP cụ thể, xuất FP_ResultE và cờ ngoại lệ fflags_E. Bộ chọn kênh kết quả dùng FPUEnE để chọn giữa ALUResultE và FPResultE, xuất ResultE và chuyển sang tầng tiếp theo.

Các tín hiệu điều khiển còn lại gồm RegWriteE, MemWriteE, ResultSrcE, FPRegWriteE, FPStoreE không tác động trực tiếp tại tầng thực thi mà được truyền thẳng qua thanh ghi EX/MEM để các tầng sau sử dụng, chi tiết tại bảng 3.7, bảng 3.8 và bảng 3.9.

Bảng 3.7: Nhóm tín hiệu đầu vào dữ liệu

Tên tín hiệu	Số bit	Ý nghĩa
RD1_E	32	Dữ liệu từ thanh ghi số nguyên nguồn 1 (toán hạng A gốc)
RD2_E	32	Dữ liệu từ thanh ghi số nguyên nguồn 2 (toán hạng B gốc)
FRD1_E	32	Dữ liệu từ thanh ghi dấu phẩy động nguồn 1 (toán hạng A FPU)
FRD2_E	32	Dữ liệu từ thanh ghi dấu phẩy động nguồn 2 (toán hạng B FPU)
Imm_Ext_E	32	Giá trị tức thời đã mở rộng dấu từ tầng giải mã
PCE	32	Địa chỉ lệnh hiện tại, dùng tính địa chỉ đích rẽ nhánh
PCPlus4E	32	Địa chỉ lệnh tiếp theo (PC + 4)
ResultW	32	Kết quả từ tầng ghi lại, dùng cho chuyển tiếp dữ liệu
ALU_ResultM	32	Kết quả từ tầng truy cập bộ nhớ, dùng cho chuyển tiếp

Bảng 3.8: Nhóm tín hiệu đầu vào điều khiển

Tên tín hiệu	Số bit	Ý nghĩa
ALUControlE	3	Chọn phép toán cho bộ ALU (000 đến 111)
ALUSrcE	1	Chọn toán hạng B: 0 = thanh ghi, 1 = giá trị tức thời

BranchE	1	Đánh dấu lệnh rẽ nhánh, kết hợp cờ Zero tạo tín hiệu PCSrcE
RegWriteE	1	Cho phép ghi thanh ghi số nguyên, truyền thẳng qua thanh ghi EX/MEM
MemWriteE	1	Cho phép ghi bộ nhớ dữ liệu, truyền thẳng qua thanh ghi EX/MEM
ResultSrcE	2	Chọn nguồn kết quả ở tầng ghi lại: 00 = ALU, 01 = bộ nhớ, 10 = PC+4
FPUEnE	1	Kích hoạt FPU: bằng 1 chọn kết quả FPU thay ALU
FPRegWriteE	1	Cho phép ghi thanh ghi dấu phẩy động, truyền thẳng qua thanh ghi EX/MEM
FPStoreE	1	Chọn dữ liệu ghi bộ nhớ: 0 = số nguyên, 1 = dấu phẩy động
fp_opE	18	Vector one-hot chọn lệnh FP cụ thể cho FPU
ForwardAE	2	Chọn nguồn toán hạng A: 00 = RD1_E, 01 = ResultW, 10 = ALU_ResultM
ForwardBE	2	Chọn nguồn toán hạng B: 00 = RD2_E, 01 = ResultW, 10 = ALU_ResultM

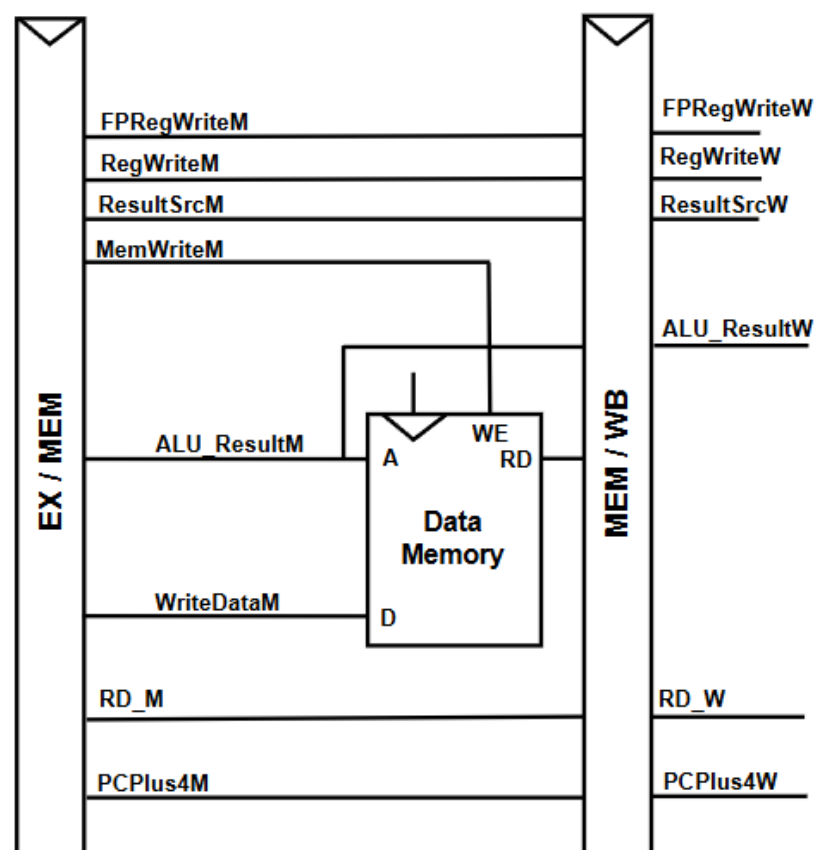
Bảng 3.9: Các tín hiệu nội bộ

Tên tín hiệu	Số bit	Ý nghĩa
Src_A	32	Toán hạng A sau chuyển tiếp, đưa vào ALU và FPU
Src_B_interim	32	Toán hạng B sau chuyển tiếp, chưa qua bộ chọn kênh ALUSrc
Src_B	32	Toán hạng B cuối cùng của ALU (Src_B_interim hoặc Imm_Ext_E)
ALU_ResultE	32	Kết quả từ ALU số nguyên

FP_ResultE	32	Kết quả từ FPU dấu phẩy động
ResultE	32	Kết quả cuối cùng: FPUEnE = 0 chọn ALU, FPUEnE = 1 chọn FPU
ZeroE	1	Cờ bằng không từ ALU, dùng cho rẽ nhánh
fflags_E	5	Cờ ngoại lệ FPU: NV, DZ, OF, UF, NX

3.3.4. Khối truy cập bộ nhớ

Khối truy cập bộ nhớ bao gồm hai khối chính: bộ nhớ dữ liệu thực hiện đọc hoặc ghi dữ liệu theo địa chỉ do tầng thực thi cung cấp và bộ chọn kênh dữ liệu ghi chọn giữa dữ liệu số nguyên và dấu phẩy động khi thực thi lệnh lưu. Các tín hiệu điều khiển không tác động tại tầng này được truyền thẳng qua thanh ghi đường ống MEM/WB để tầng ghi lại sử dụng.



Hình 3.11: Sơ đồ khối của khối truy cập bộ nhớ

Khi nhận dữ liệu từ thanh ghi EX/MEM, ALU_ResultM đóng vai trò địa chỉ truy cập bộ nhớ dữ liệu. Khi tín hiệu MemWriteM bằng 1, bộ nhớ thực hiện lệnh ghi. Nếu

FPStoreM bằng 0, dữ liệu ghi là WriteDataM từ thanh ghi số nguyên rs2, nếu FPStoreM có giá trị bằng 1, dữ liệu ghi là giá trị từ thanh ghi dấu phẩy động rs2. Khi MemWriteM bằng 0 và ResultSrcM bằng 1, bộ nhớ thực hiện lệnh đọc tại địa chỉ ALU_ResultM và xuất ReadDataM, đây là dữ liệu sẽ được ghi vào thanh ghi đích ở tầng ghi lại. Với lệnh FLW, ReadDataM được ghi vào tập thanh ghi dấu phẩy động thay vì tập thanh ghi số nguyên, điều này do FPRegWriteM điều khiển ở tầng ghi lại. Đồng thời, ALU_ResultM cũng được chuyển tiếp ngược về tầng thực thi thông qua đường chuyển tiếp để có thể giải quyết xung đột.

Các tín hiệu RegWriteM, ResultSrcM, FPRegWriteM và địa chỉ thanh ghi đích RdM không tác động tại tầng này mà được truyền thẳng qua thanh ghi MEM/WB sang tầng ghi lại, chi tiết tại bảng 3.10.

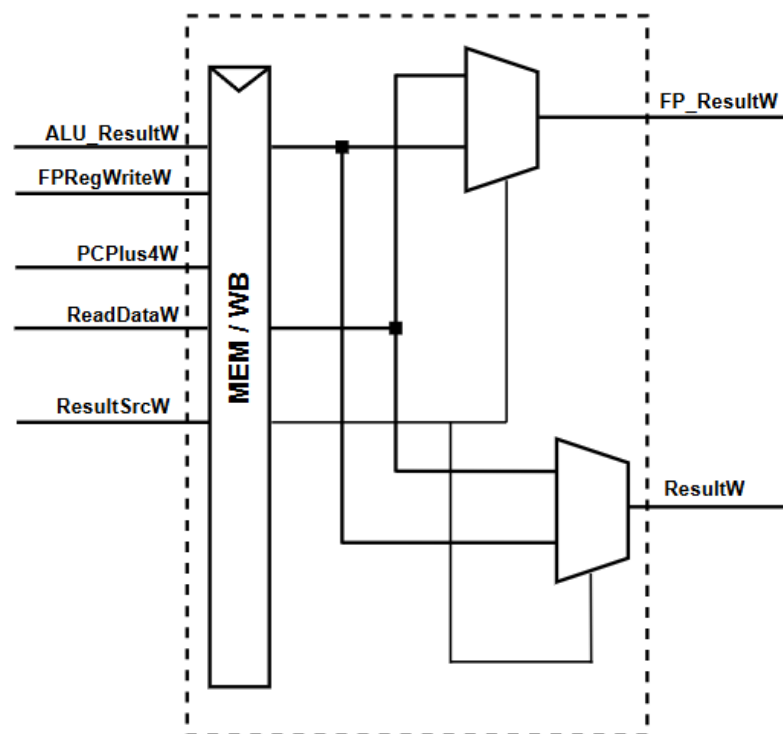
Bảng 3.10: Tín hiệu của khối truy cập bộ nhớ

Tên tín hiệu	Số bit	Ý nghĩa
CLK	1	Xung nhịp hệ thống
RST	1	Reset tích cực mức thấp
ALU_ResultM	32	Địa chỉ truy cập bộ nhớ, đồng thời chuyển tiếp về tầng thực thi
WriteDataM	32	Dữ liệu ghi vào bộ nhớ (số nguyên hoặc dấu phẩy động)
MemWriteM	1	Cho phép ghi bộ nhớ dữ liệu
RegWriteM	1	Truyền thẳng sang tầng ghi lại
ResultSrcM	2	Truyền thẳng sang tầng ghi lại
FPRegWriteM	1	Truyền thẳng sang tầng ghi lại
RD_M	5	Địa chỉ thanh ghi đích, truyền thẳng sang tầng ghi lại
PCPlus4M	32	Giá trị PC+4, truyền thẳng sang tầng ghi lại
ReadDataW	32	Dữ liệu đọc từ bộ nhớ, chuyển sang tầng ghi lại
ALU_ResultW	32	Kết quả ALU chót qua thanh ghi MEM/WB

PCPlus4W	32	Giá trị PC+4 chốt qua thanh ghi MEM/WB
RegWriteW	1	Tín hiệu ghi thanh ghi số nguyên chuyển sang tầng ghi lại
ResultSrcW	2	Tín hiệu chọn nguồn kết quả chuyển sang tầng ghi lại
FPRegWriteW	1	Tín hiệu ghi thanh ghi dấu phẩy động chuyển sang tầng ghi lại
RD_W	5	Địa chỉ thanh ghi đích chuyển sang tầng ghi lại

3.3.5. Khối ghi lại

Tầng ghi lại là tầng cuối cùng trong đường ống, bao gồm một bộ chọn kênh kết quả chọn dữ liệu cần ghi lại vào tập thanh ghi. Khác với các tầng trước, tầng này không có thanh ghi đường ống đầu ra mà ghi trực tiếp vào tập thanh ghi ở tầng giải mã, đồng thời chuyển tiếp kết quả ngược về tầng thực thi để giải quyết xung đột dữ liệu.



Hình 3.12: Sơ đồ khối của khối ghi lại

Khi nhận dữ liệu từ thanh ghi MEM/WB, bộ chọn kênh dùng ResultSrcW để quyết định nguồn dữ liệu ghi vào tập thanh ghi số nguyên: khi bằng 00, chọn ALU_ResultW

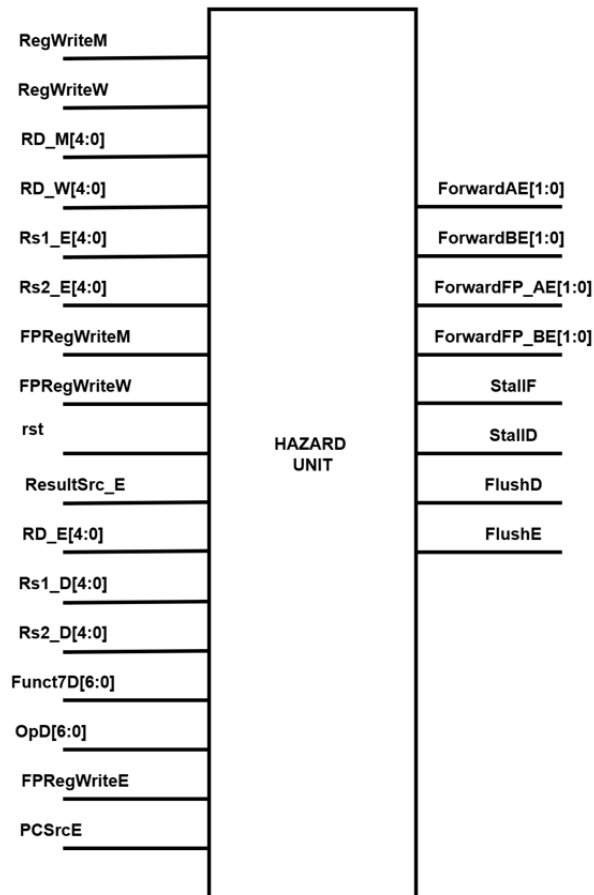
là kết quả tính toán từ tầng thực thi; khi bằng 01, chọn ReadDataW là dữ liệu vừa đọc từ bộ nhớ dành cho lệnh LW và khi bằng 10, chọn PCPlus4W là địa chỉ lệnh kế tiếp dành cho lệnh JAL và JALR lưu địa chỉ trả về. Giá trị được chọn xuất thành ResultW và ghi vào tập thanh ghi số nguyên tại địa chỉ RD_W khi RegWriteW bằng 1. Song song đó, tập thanh ghi dấu phẩy động nhận FP_ResultW riêng biệt: khi lệnh FLW đọc dữ liệu từ bộ nhớ thì FP_ResultW lấy ReadDataW, còn với các lệnh FPU thì FP_ResultW lấy trực tiếp kết quả từ tầng thực thi và được ghi tại địa chỉ RD_W khi FPRegWriteW bằng 1. Đồng thời, ResultW được chuyển tiếp ngược về tầng thực thi qua đường chuyển tiếp dữ liệu, phục vụ cho bộ chọn kênh chuyển tiếp A và B khi ForwardAE hoặc ForwardBE bằng 01.

Bảng 3.11: Tín hiệu của khối ghi lại

Tên tín hiệu	Số bit	Ý nghĩa
ALU_ResultW	32	Kết quả ALU từ tầng thực thi chuyển qua
ReadDataW	32	Dữ liệu đọc từ bộ nhớ chuyển qua
PCPlus4W	32	Giá trị PC+4 dùng cho lệnh JAL/JALR lưu địa chỉ trả về
ResultSrcW	2	Chọn nguồn ghi thanh ghi số nguyên: 00 = ALU, 01 = bộ nhớ, 10 = PC+4
RegWriteW	1	Cho phép ghi vào tập thanh ghi số nguyên
FPRegWriteW	1	Cho phép ghi vào tập thanh ghi dấu phẩy động
RD_W	5	Địa chỉ thanh ghi đích
ResultW	32	Kết quả ghi vào tập thanh ghi số nguyên, đồng thời chuyển tiếp về tầng thực thi
FP_ResultW	32	Kết quả ghi vào tập thanh ghi dấu phẩy động: bằng ReadDataW khi lệnh FLW, bằng ALU_ResultW với các lệnh FPU còn lại

3.3.6. Khối xử lý xung đột

Khối xử lý xung đột bao gồm hai đơn vị là đơn vị chuyển tiếp dữ liệu giải quyết xung đột dữ liệu số nguyên bằng cách đưa kết quả mới nhất trực tiếp về tầng thực thi và đơn vị phát hiện xung đột xử lý các trường hợp còn lại bằng cách dừng đường ống hoặc xóa thanh ghi đường ống. Ngoài ra, thanh ghi dấu phẩy động xử lý mọi xung đột dữ liệu bằng cách dừng đường ống và chuyển tiếp.



Hình 3.13: Sơ đồ khối của khối xử lý xung đột

Đơn vị chuyển tiếp dữ liệu so sánh địa chỉ thanh ghi nguồn tại tầng thực thi với địa chỉ thanh ghi đích tại tầng truy cập bộ nhớ và tầng ghi lại. Nếu RegWriteM bằng 1, RD_M khác 0 và RD_M bằng Rs1_E, ForwardAE được đặt bằng 10 chuyển tiếp ALU_ResultM về toán hạng A. Nếu điều kiện trên không thỏa nhưng RegWriteW bằng 1 và RD_W khác 0 và RD_W bằng Rs1_E, ForwardAE được đặt bằng 01 chuyển tiếp ResultW từ tầng ghi lại.

Đơn vị phát hiện xung đột xử lý ba trường hợp. Thứ nhất là xung đột nạp – lưu: khi lệnh ở tầng thực thi là lệnh nạp bộ nhớ và thanh ghi đích trùng với thanh ghi

nguồn của lệnh đang giải mã, tín hiệu lwStall được kích hoạt. Thứ hai là xung đột dữ liệu FP: do không có đường chuyển tiếp cho thanh ghi dấu phẩy động, đơn vị kiểm tra opcode và funct7 của lệnh đang ở tầng giải mã để xác định lệnh đó có đọc thanh ghi FP không, sau đó so sánh với thanh ghi đích đang được ghi tại tầng thực thi hoặc tầng truy cập bộ nhớ nếu trùng thì kích hoạt FPStall. Thứ ba là xung đột điều khiển do rẽ nhánh: khi PCSrcE bằng 1, FlushD và FlushE xóa hai lệnh nạp sai, chi tiết theo bảng 3.12.

Bảng 3.12: Tín hiệu của khối xử lý xung đột

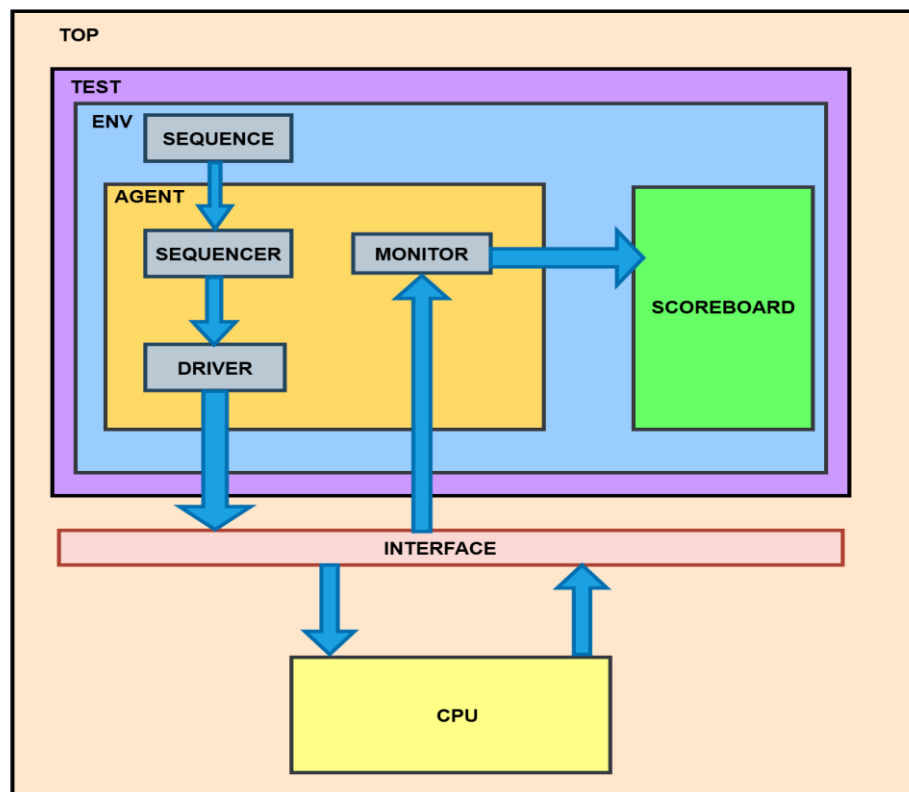
Tên tín hiệu	Số bit	Ý nghĩa
RST	1	Reset tích cực mức thấp
Rs1_E, Rs2_E	5	Địa chỉ thanh ghi nguồn tại tầng thực thi
RD_M, RD_W	5	Địa chỉ thanh ghi đích tại tầng MEM và WB
RegWriteM, RegWriteW	1	Tín hiệu ghi thanh ghi số nguyên tại MEM và WB
FPRegWriteE, FPRegWriteM	1	Tín hiệu ghi thanh ghi FP tại EX và MEM
Rs1_D, Rs2_D	5	Địa chỉ thanh ghi nguồn tại tầng giải mã
RD_E	5	Địa chỉ thanh ghi đích tại tầng thực thi
ResultSrcE[0]	1	Bit thấp của ResultSrcE, dùng phát hiện lệnh nạp bộ nhớ tại tầng thực thi
PCSrcE	1	Tín hiệu rẽ nhánh từ tầng thực thi
OpD	7	Opcode của lệnh đang giải mã, dùng phát hiện xung đột FP
Funct7D	7	Funct7 của lệnh đang giải mã, xác định lệnh FP đọc toán hạng nào
ForwardAE, ForwardBE	2	Chọn nguồn toán hạng A/B: 00 = thanh ghi, 01 = WB, 10 = MEM

StallF, StallD	1	Dừng cập nhật PC và thanh ghi IF/ID
FlushD	1	Xóa thanh ghi IF/ID khi rẽ nhánh
FlushE	1	Xóa thanh ghi ID/EX khi rẽ nhánh, xung đột nạp - sử dụng hoặc xung đột FP

3.4. THIẾT KẾ MÔI TRƯỜNG KIỂM THỬ

3.4.1. Tổng quan kiến trúc môi trường UVM

Môi trường UVM được xây dựng với một agent duy nhất phụ trách toàn bộ đường ống từ khối nạp lệnh đến khối ghi lại. Hình 3.14 trình bày kiến trúc tổng quan môi trường UVM được xây dựng cho hệ thống.



Hình 3.14: Kiến trúc môi trường UVM

Môi trường kiểm chứng được xây dựng dựa trên kiến trúc với cấu trúc phân cấp chặt chẽ, giúp tối ưu hóa khả năng tái sử dụng và quản lý kịch bản kiểm tra. Dựa vào hình 3.14, các thành phần chính bao gồm:

- Top: là lớp cao nhất chứa CPU và môi trường kiểm thử. Cả 2 được kết nối với nhau thông qua interface.

- Test: đóng vai trò cấu hình môi trường, lựa chọn các chuỗi lệnh cần thực thi và bao bọc lớp env.
- Env: môi trường kiểm chứng tổng thể, là nơi kết nối và điều phối hoạt động giữa các thành phần chính như agent, scoreboard và các mô hình tham chiếu.
- Agent: thành phần quản lý giao tiếp tại các cổng của bộ xử lý, bao gồm ba thành phần consequencer, driver và monitor.
- Interface: tập hợp các đường tín hiệu dùng để kết nối giữa CPU và các thành phần trong môi trường.
- CPU: thiết kế RTL của bộ xử lý cần được kiểm tra tính đúng đắn.
- Scoreboard: thành phần tự động thu thập kết quả thực tế từ monitor và kết quả kỳ vọng từ mô hình tham chiếu để thực hiện so sánh.

3.4.2. Thiết kế khối sequence và khối sequencer

Khối sequencer đóng vai trò điều phối luồng giao dịch từ các sequence xuống khối driver theo cơ chế bắt tay TLM. Các sequence được thiết kế theo nguyên tắc phân cấp, trong đó các sequence cơ bản kiểm tra từng nhóm lệnh riêng lẻ và sequence hỗn hợp kết hợp nhiều nhóm lệnh để tạo ra các chuỗi lệnh phức tạp có xung đột dữ liệu thực tế. Bảng 3.13 trình bày các sequence được thiết kế cho môi trường UVM.

Bảng 3.13: Các sequence được thiết kế cho việc kiểm thử

Số thứ tự	Tên sequence	Ý nghĩa
1	riscv_rtype_directed_seq	Kiểm tra trực tiếp lệnh loại R: xác nhận tính đúng đắn của khối ALU số nguyên với các phép toán cơ bản như ADD, SUB, SLT, OR, AND thông qua các giá trị toán hạng cụ thể
2	riscv_itype_directed_seq	Kiểm tra lệnh loại I: xác minh các lệnh tính toán với số tức thời và khả năng truy xuất bộ nhớ
3	riscv_fext_directed_seq	Kiểm tra tập mở rộng F: xác minh quy trình xử lý số thực của FPU

4	riscv_hazard_test_seq	Kiểm tra xung đột đường ống: đánh giá khả năng xử lý của khối xử lý xung đột
5	riscv_mext_test	Kiểm tra các phép toán nhân, chia, chia lấy phần dư

3.4.3. Thiết kế khối driver và khối monitor

Khối driver có nhiệm vụ nhận các giao dịch từ khối sequencer và chuyển đổi thành các tín hiệu và truyền đến CPU. Cụ thể, driver nạp mã lệnh vào bộ nhớ lệnh trước khi bắt đầu mô phỏng, sau đó điều khiển tín hiệu reset và xung nhịp để đường ống bắt đầu thực thi. Mỗi transaction chứa đầy đủ thông tin về lệnh cần thực thi bao gồm opcode, các thanh ghi nguồn, thanh ghi đích và giá trị tức thì.

Monitor quan sát các tín hiệu đầu ra tại khối ghi lại của đường ống bao gồm RegWriteW, RD_W, ResultW cho đường dữ liệu số nguyên và FPRegWriteW, FP_RD_W, FP_ResultW cho đường dữ liệu số thực. Khi phát hiện tín hiệu ghi lại được kích hoạt, monitor đóng gói thông tin quan sát được và gửi đến scoreboard.

3.4.4. Thiết kế khối interface

Khối interface đóng vai trò là kênh kết nối giữa CPU và môi trường kiểm thử UVM, tạo điều kiện cho việc truyền tín hiệu và trao đổi dữ liệu giữa các thành phần như driver, monitor và CPU. Các tín hiệu được sử dụng trong thiết kế này được liệt kê và mô tả trong bảng 3.14.

Bảng 3.14: Các tín hiệu trong khối interface

Tên tín hiệu	Số bit	Ý nghĩa
clk	1	Xung hệ thống
rst	1	Tín hiệu khởi động lại tích cực mức thấp
PC	32	Giá trị bộ đếm chương trình
InstrD	32	Lệnh tại khối giải mã
ALUControlE	3	Mã điều khiển ALU tại khối xử lý

ALU_ResultM	32	Kết quả từ ALU
PCSrcE	1	Tín hiệu rẽ nhánh/nhảy từ tầng thực thi
RegWriteW	1	Tín hiệu cho phép ghi
RD_W	5	Địa chỉ thanh ghi đích
ResultW	32	Kết quả ghi vào thanh ghi số nguyên
FPRegWriteW	1	Tín hiệu cho phép ghi
FP_RD_W	5	Địa chỉ thanh ghi đích
FP_ResultW	32	Kết quả ghi vào thanh ghi số thực
ForwardAE	2	Chuyển tiếp dữ liệu vào khối thực thi cho ngõ vào SrcA(rs1)
ForwardBE	2	Chuyển tiếp dữ liệu vào khối thực thi cho ngõ vào SrcB(rs2)
StallF	1	Tín hiệu dừng cập nhật PC tại khối nạp lệnh
StallID	1	Tín hiệu dừng cập nhật thanh ghi IF/ID
FlushD	1	Xóa dữ liệu trong thanh ghi tại khối giải mã
FlushE	1	Xóa dữ liệu trong thanh ghi tại khối thực thi

3.4.5. Thiết kế khối scoreboard

Scoreboard thực hiện đối chiếu tự động giữa kết quả thực tế từ DUT và kết quả kỳ vọng từ mô hình tham chiếu. Mô hình tham chiếu được xây dựng bằng SystemVerilog, thực hiện tính toán lại kết quả của từng lệnh dựa trên thông tin transaction nhận được từ monitor. Với lệnh số nguyên, mô hình tham chiếu tính toán kết quả dựa trên opcode và các toán hạng. Với lệnh số thực, mô hình tham chiếu sử dụng phép tính IEEE 754 độ chính xác đơn.

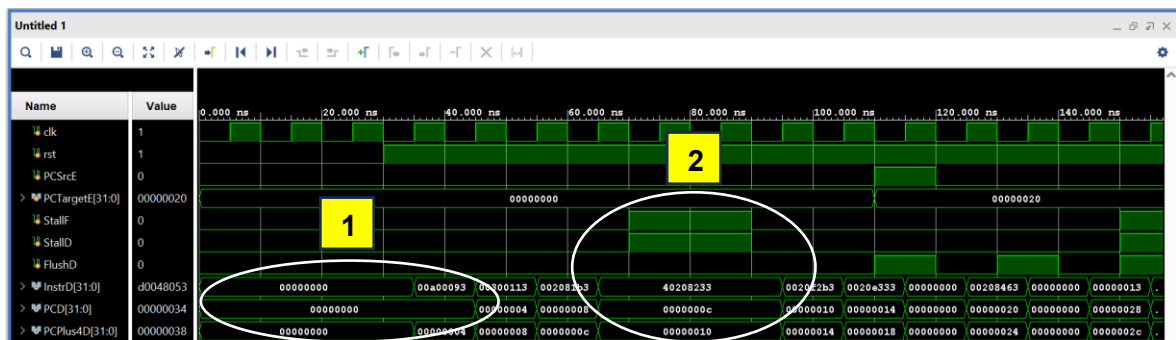
CHƯƠNG 4

KẾT QUẢ

4.1. KẾT QUẢ MÔ PHỎNG TỪNG KHỐI

4.1.1. Khối nạp lệnh

Trong bài kiểm tra của tầng nạp lệnh, lệnh sẽ được khởi tạo bên trong bộ nhớ lệnh, bài mô phỏng sẽ nạp những lệnh đó vào tầng nạp lệnh qua từng chu kỳ và kết quả được thể hiện trên dạng sóng như hình 4.1.



Hình 4.1: Dạng sóng của khối nạp lệnh

Từ 0ns đến 30ns (1), đây là trạng thái khởi tạo hệ thống, tín hiệu reset tích cực mức thấp, toàn bộ thanh ghi đường ống IF/ID bị xóa về 0, InstrD, PCD và PCPlus4D đều giữ giá trị 0x00000000.

Từ 30ns, tín hiệu reset được đưa lên mức 1, bộ đếm chương trình bắt đầu hoạt động, hệ thống chuẩn bị nạp lệnh đầu tiên.

Từ 35ns đến 45ns, chu kỳ thứ nhất, tại cạnh lên đầu tiên của xung clk (35ns), thanh ghi IF/ID chốt lệnh tại PC bằng 0x00000000, InstrD chuyển sang 0x00A00093 tương ứng lệnh addi x1, x0, 10 được lưu tại vị trí mem[0], các tín hiệu PCD và PCPlus4D có giá trị lần lượt là 0x00000000 và 0x00000004.

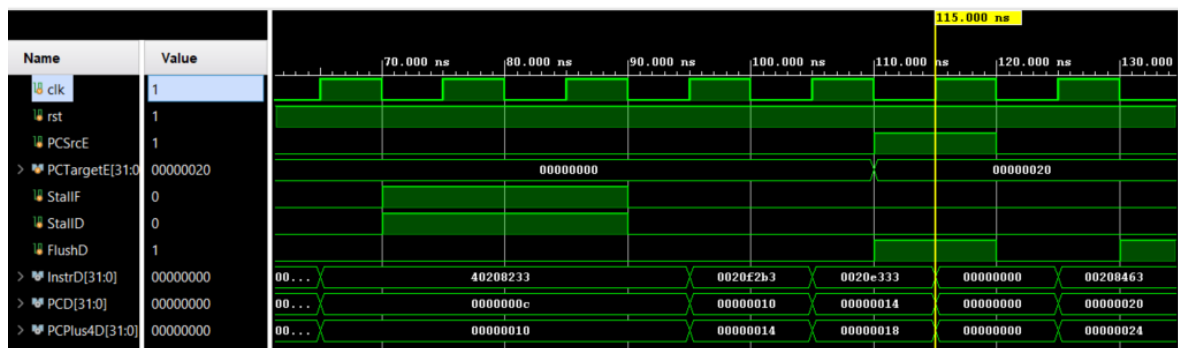
Từ 45ns đến 55ns, chu kỳ thứ 2, tại cạnh lên của xung clk (45ns), thanh ghi IF/ID chốt lệnh tại PC bằng 0x00000004, InstrD chuyển sang lệnh 0x00300113 tương ứng lệnh addi x2, x0, 3 được lưu tại vị trí mem[1], các tín hiệu PCD và PCPlus4D có giá trị lần lượt là 0x00000004 và 0x00000008.

Tại vị trí 70ns (2): khối phát hiện xung đột kích hoạt tín hiệu dừng đường ống, tín hiệu StallF lên mức 1 ngăn PC cập nhật, tín hiệu StallID lên mức 1 ngăn thanh ghi

IF/ID ghi giá trị mới, các giá trị của InstrD, PCD và PCPlus4D có giá trị không đổi trong khoảng thời gian này.

Từ 90ns đến 110ns, StallF và StallID về 0 cho phép PC tiếp tục từ địa chỉ 0x10, chu kỳ đầu tiên sau khi nhả stall InstrD ra 0x0020f2b3 là lệnh and x5, x1, x2 tại PCD 0x00000010, tiếp theo InstrD ra lệnh or x6, x1, x2 tại PCD 0x00000014, xác nhận đường ống tiếp tục đúng vị trí mà không mất hay lộn lệnh nào.

Từ 110ns đến 120ns, PCSrcE và FlushD cùng lên 1 tại cùng một thời điểm, PC nhảy đến địa chỉ đích 0x20 trong khi FlushD xóa trắng thanh ghi IF/ID ngay lập tức, kết quả InstrD ra giá trị 0x00000000 tương đương với 1 NOP và PCD ra giá trị 0x00000000, chi tiết xem tại hình 4.2.



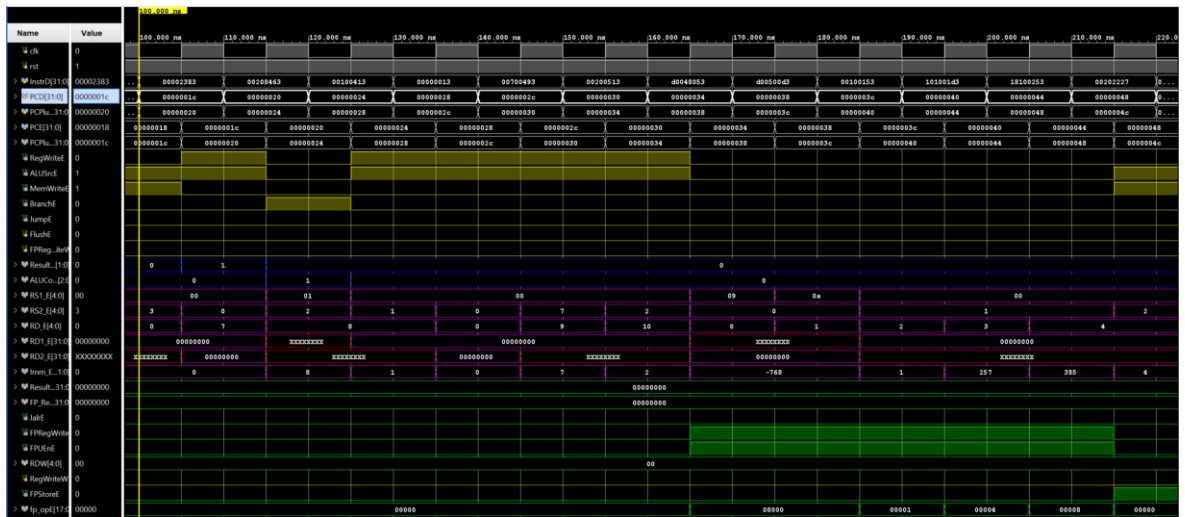
Hình 4.2: PCSrcE được kích hoạt và giá trị các thanh ghi

4.1.2. Khối giải mã

Hình 4.3 và hình 4.4 trình bày dạng sóng mô phỏng kết quả kiểm tra của tầng giải mã.



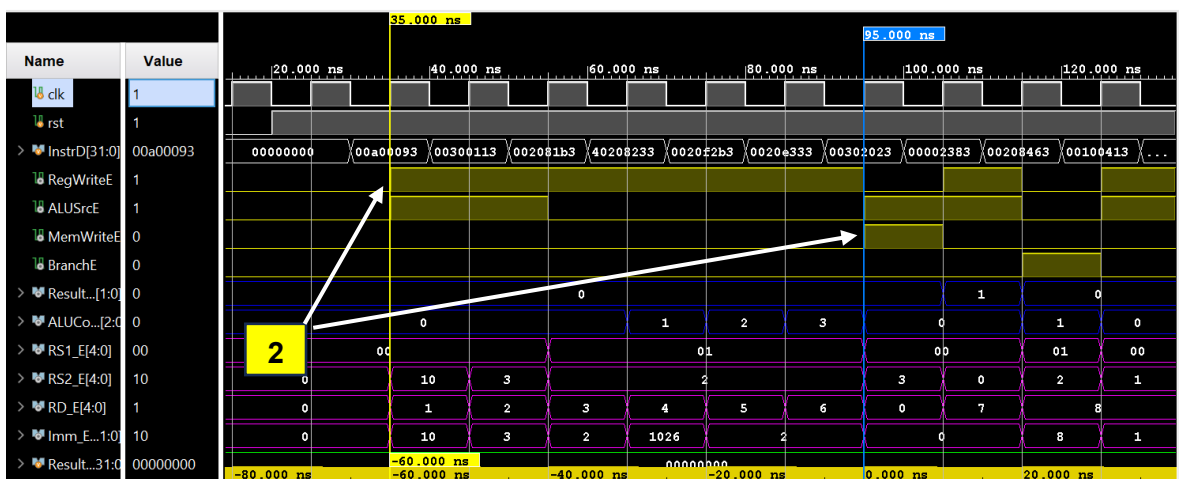
Hình 4.3: Dạng sóng mô phỏng chức năng của khối giải mã từ 0ns đến 100ns



Hình 4.4: Dạng sóng mô phỏng chức năng của khối giải mã sau 100ns

Từ 0ns đến 35ns (1), trong hai chu kỳ đầu rst duy trì mức thấp, toàn bộ thanh ghi ID/EX được xóa về 0 và tất cả tín hiệu điều khiển đầu ra không mang ý nghĩa. Tại 20 ns rst chuyển lên cao, tuy nhiên InstrD vẫn là 0x00000000 nên chu kỳ tại 25 ns tiếp tục ghi nhận NOP. Hệ thống sẵn sàng nhận lệnh thực từ 35 ns.

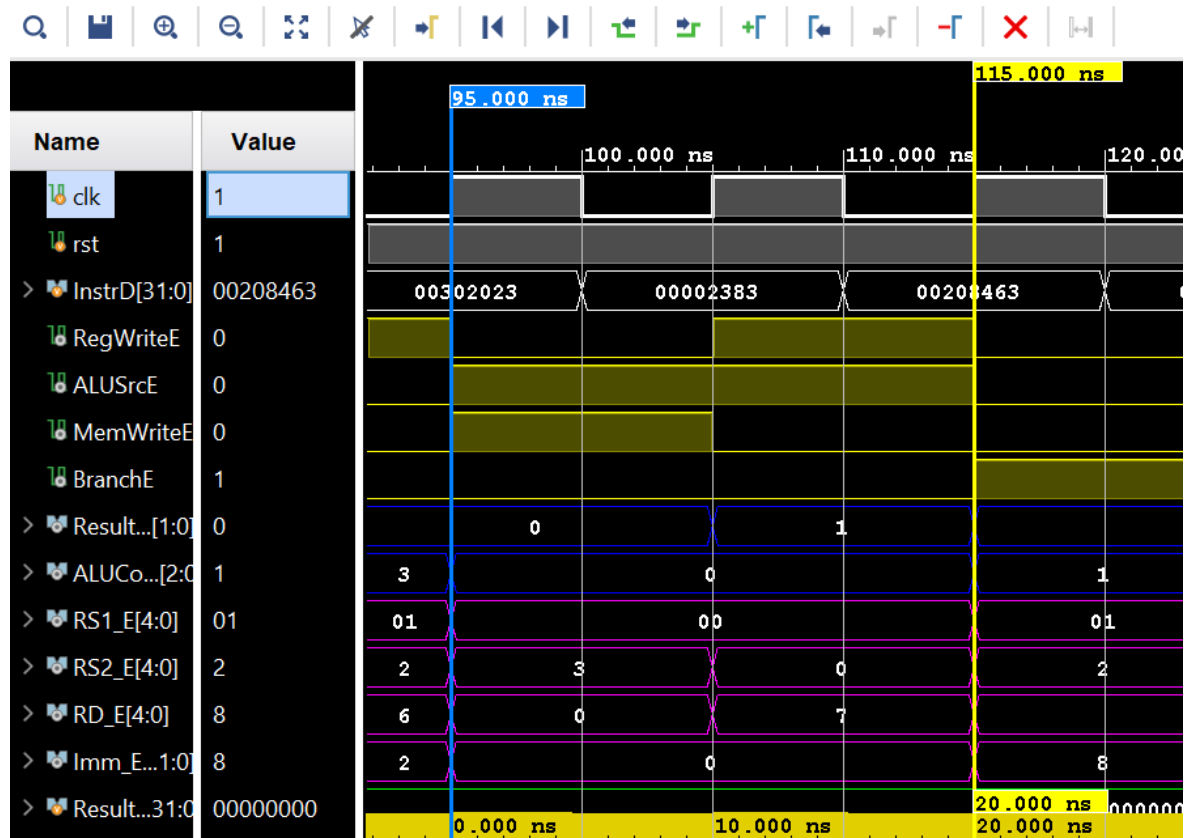
Từ 35ns đến 95ns được thể hiện qua hình 4.5, các lệnh từ tầng nạp lệnh bắt đầu được nạp vào tầng giải mã để tiến hành giải mã ra loại lệnh và xuất ra các tín hiệu điều khiển. Trong khoảng thời gian này, các lệnh addi, add, sub, and và or được giải mã, tín hiệu RegWriteE luôn bằng 1 (2) để có thể cho phép ghi thanh ghi số nguyên, tín hiệu ALUSrcE bằng 1 đối với 2 lệnh addi vì toán hạng B của ALU trong 2 lệnh này đang sử dụng giá trị tức thời.



Hình 4.5: Kết quả mô phỏng của tầng giải mã từ 35ns đến 95ns

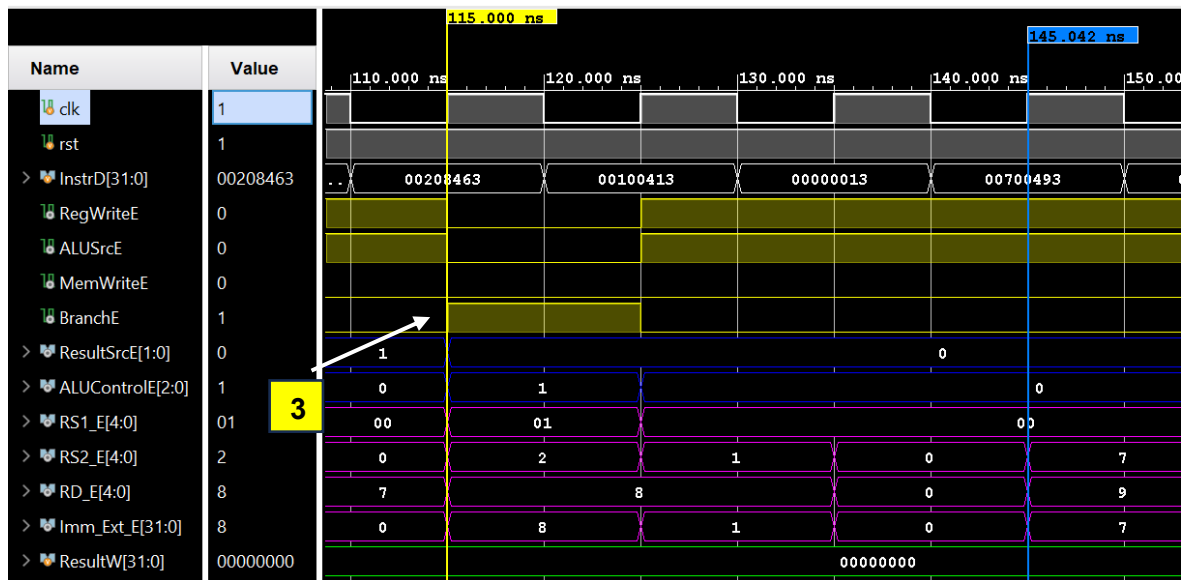
Từ 95ns đến 115ns, hai lệnh truy cập bộ nhớ sw x3, 0(x0) và lw x7, 0(x0) được giải mã. Đối với lệnh sw, tín hiệu MemWriteE được kích hoạt lên 1 (2) để cho phép

ghi vào bộ nhớ dữ liệu, đồng thời RegWriteE bằng 0 vì lệnh store không ghi vào thanh ghi. ALUSrcE bằng 1 để ALU dùng giá trị tức thì tính địa chỉ ghi, thanh ghi rs2 = x3 chứa dữ liệu cần ghi vào bộ nhớ. Sang lệnh lw, RegWriteE trở lại 1 và ResultSrcE chuyển sang 01, báo hiệu tầng ghi lại sẽ lấy dữ liệu từ bộ nhớ thay vì từ kết quả ALU để ghi vào thanh ghi đích rd = x7, chi tiết xem tại hình 4.6.



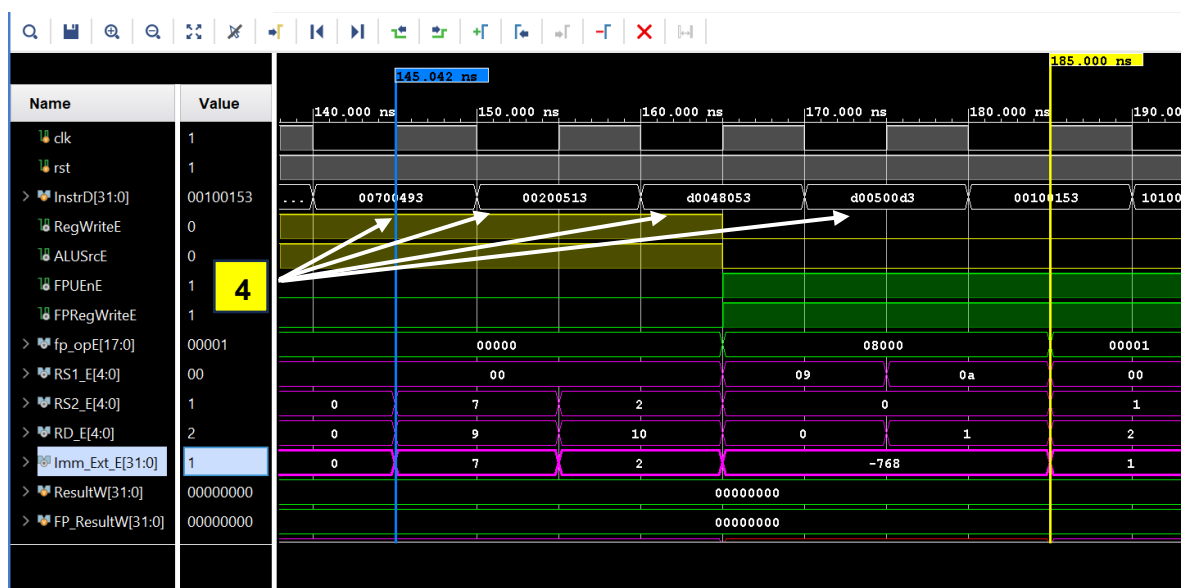
Hình 4.6: Kết quả mô phỏng của tầng giải mã từ 95ns đến 115ns

Hình 4.7 cho thấy thời điểm từ 115ns đến 145ns, ba lệnh tiếp theo được giải mã gồm beq x1, x2, +8, addi x8, x0, 1 và nop. Đối với lệnh beq, tín hiệu BranchE được kích hoạt lên 1 (3), RegWriteE và MemWriteE đều bằng 0 vì lệnh rẽ nhánh không ghi vào thanh ghi hay bộ nhớ. ALUControlE = 001 để khởi thực thi thực hiện phép trừ rs1 - rs2 nhằm kiểm tra điều kiện bằng nhau, ALUSrcE = 0 để ALU lấy giá trị từ hai thanh ghi nguồn rs1 = x1 và rs2 = x2. Giá trị tức thời Imm_Ext_E = 0x00000008 là offset nhảy được giải mã từ định dạng B-type. Hai lệnh addi x8 và nop tiếp theo trở lại hoạt động bình thường với RegWriteE = 1, ALUSrcE = 1 và ALUControlE = 000.



Hình 4.7: Kết quả mô phỏng của tầng giải mã từ 115ns đến 145ns

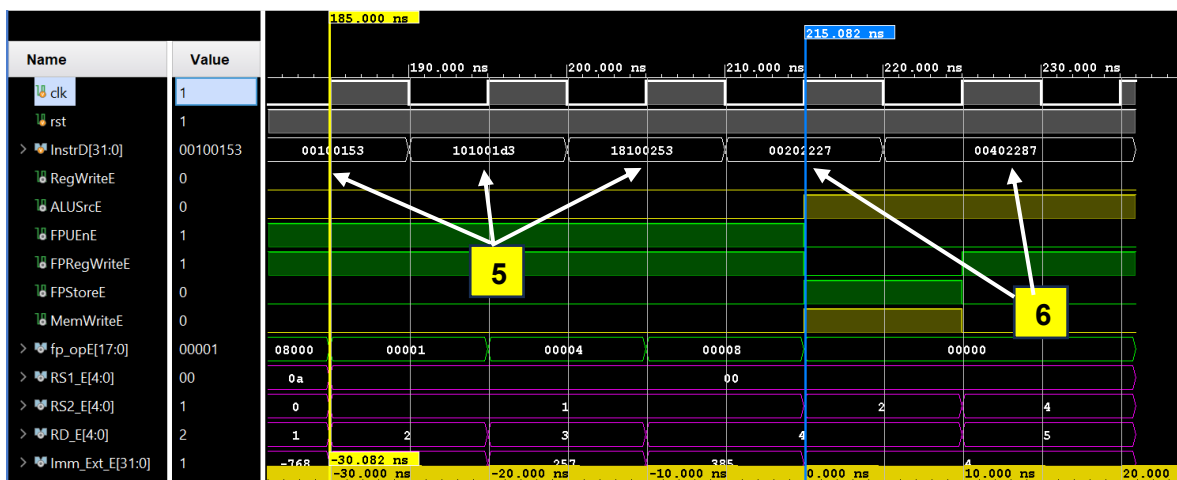
Từ 145ns đến 185ns, khoảng thời điểm này được thể hiện như hình 4.8, bốn lệnh chuẩn bị dữ liệu dấu phẩy động được giải mã (4). Hai lệnh `addi x9, x0, 7` và `addi x10, x0, 2` là lệnh số nguyên thông thường với `RegWriteE = 1`, `ALUSrcE = 1`, ghi lần lượt giá trị 7 và 2 vào thanh ghi số nguyên x9 và x10. Tiếp theo, hai lệnh `fcvt.s.w f0, x9` và `fcvt.s.w f1, x10` chuyển đổi giá trị nguyên sang dấu phẩy động. Với hai lệnh này, `FPUEnE = 1` kích hoạt khối FPU, `FPURegWriteE = 1` cho phép ghi kết quả vào tập thanh ghi dấu phẩy động, `RegWriteE = 0` vì kết quả không ghi vào thanh ghi nguyên. Tín hiệu `fp_opE` ở bit 15 bằng 1 tương ứng với phép chuyển đổi `fcvt.s.w`, thanh ghi nguồn `rs1` trở vào tập thanh ghi nguyên lần lượt là x9 và x10.



Hình 4.8: Kết quả mô phỏng của tầng giải mã từ 145ns đến 185ns

Từ 185ns đến 215ns, ba lệnh số học dấu phẩy động `fadd.s f2, f0, f1`, `fmul.s f3, f0, f1` và `fdiv.s f4, f0, f1` được giải mã (5). Cả ba lệnh duy trì `FPUEnE` = 1 và `FPRegWriteE` = 1 xuyên suốt, `RegWriteE` = 0 và `ALUSrcE` = 0 vì ALU không tham gia tính toán. Tín hiệu `fp_opE` thay đổi theo cơ chế one-hot qua từng chu kỳ: bit 0 bằng 1 cho `fadd.s`, bit 2 bằng 1 cho `fmul.s` và bit 3 bằng 1 cho `fdiv.s`, tương ứng với trường `funct5` được giải mã từ `funct7` của mã lệnh. Địa chỉ thanh ghi nguồn `rs1` = `f0`, `rs2` = `f1` và thanh ghi đích `rd` lần lượt là `f2`, `f3`, `f4` được giải mã từ mã lệnh.

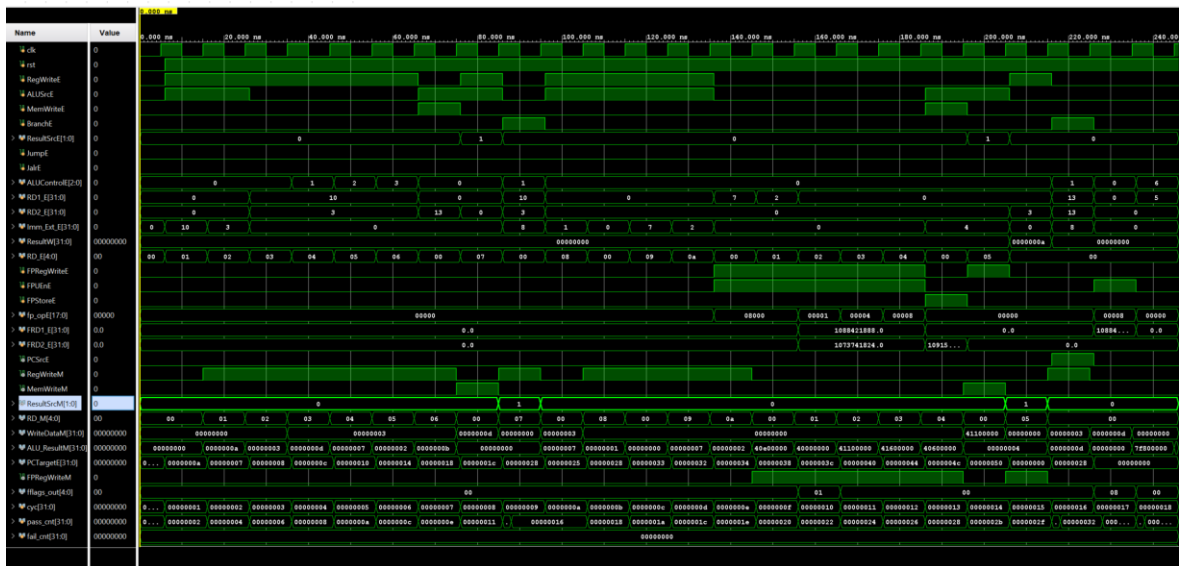
Từ 215ns đến 235ns, hai lệnh truy cập bộ nhớ dấu phẩy động `fsw f2, 4(x0)` và `flw f5, 4(x0)` được giải mã (6). Đối với lệnh `fsw`, tín hiệu `FPStoreE` = 1 và `MemWriteE` = 1 được kích hoạt, báo hiệu dữ liệu ghi vào bộ nhớ lấy từ thanh ghi dấu phẩy động `rs2` = `f2` thay vì thanh ghi số nguyên, `RegWriteE` = 0 và `FPRegWriteE` = 0 vì không ghi vào bất kỳ tập thanh ghi nào. `ALUSrcE` = 1 để ALU tính địa chỉ ghi từ `rs1` = `x0` cộng với offset `Imm_Ext_E` = `0x00000004`. Sang lệnh `flw`, tín hiệu `FPRegWriteE` = 1 và `ResultSrcE` = 01 chỉ thị tầng WB lấy dữ liệu từ bộ nhớ để ghi vào thanh ghi dấu phẩy động đích `rd` = `f5`, chi tiết xem tại hình 4.9.



Hình 4.9: Kết quả mô phỏng của tầng giải mã từ 185ns đến sau 215ns

4.1.3. Khối thực thi

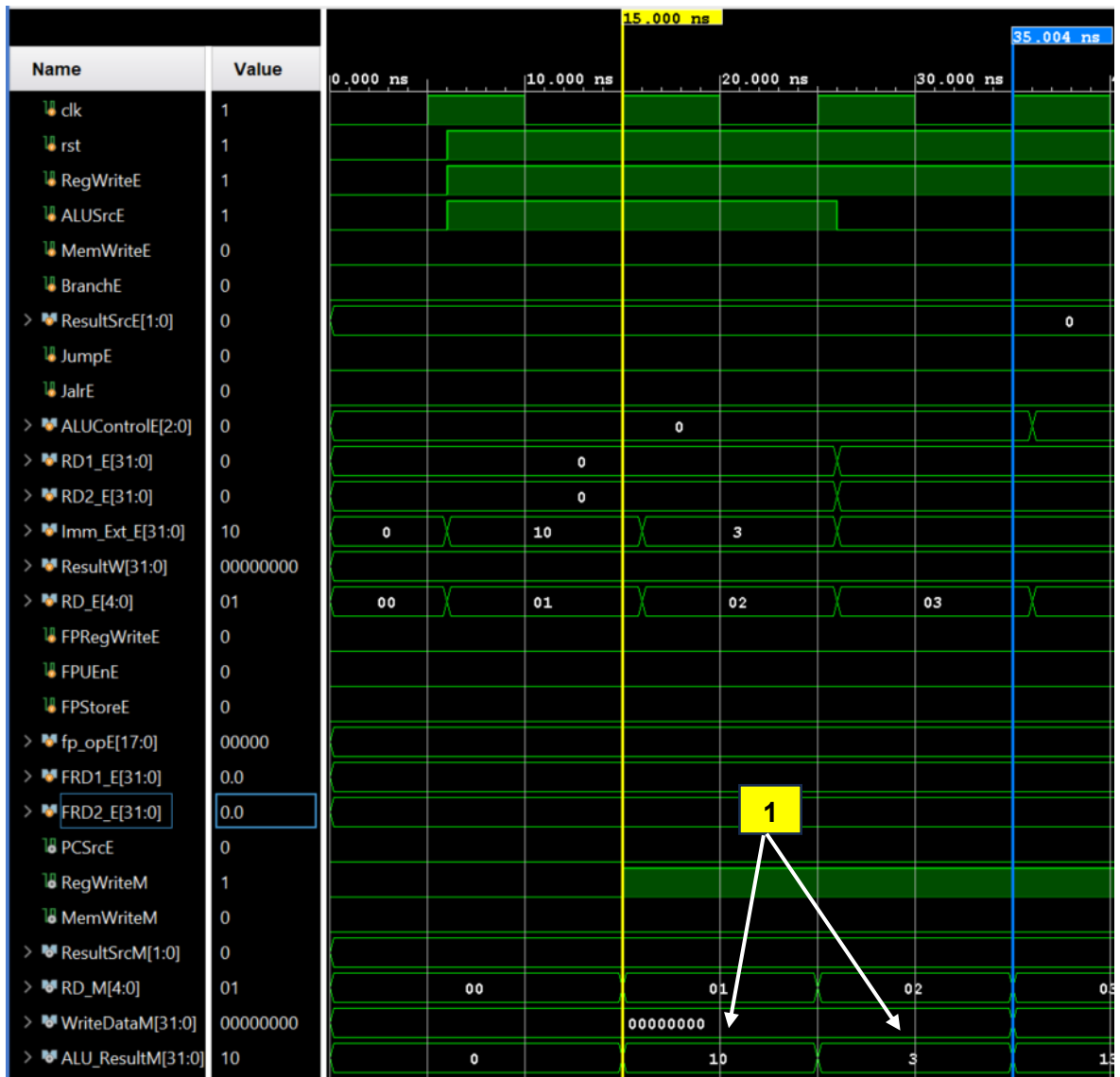
Bài kiểm tra của tầng thực thi bao gồm các lệnh tương tự như tầng giải mã, chia thành nhiều nhóm kiểm tra. Hình 4.10 thể hiện toàn bộ kết quả mô phỏng của tầng thực thi, chi tiết từng khoảng thời điểm quan sát các hình từ hình 4.11 đến hình 4.14.



Hình 4.10: Kết quả dạng sóng mô phỏng của tầng thực thi

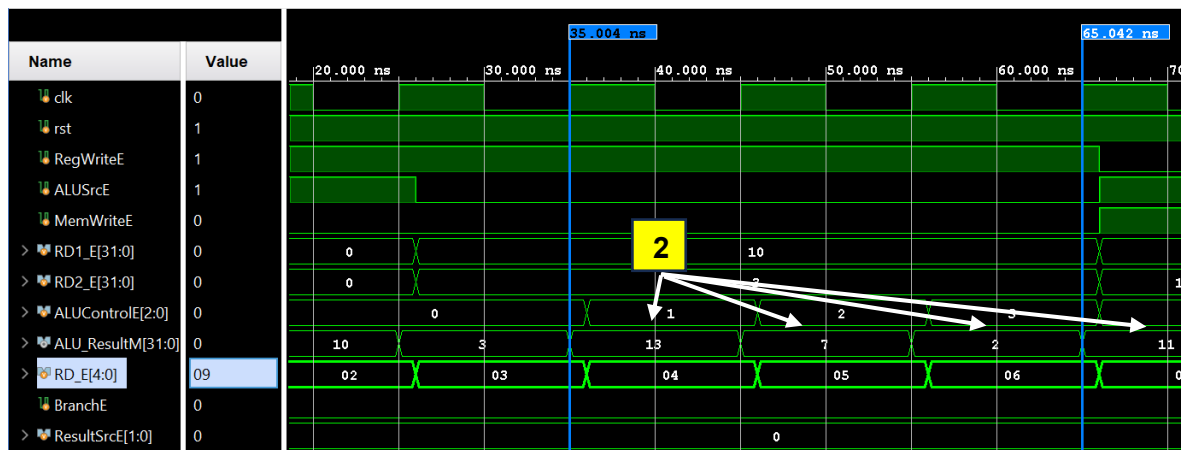
Quan sát hình 4.11, từ 0ns đến 15ns, tín hiệu reset giữ mức thấp khiến toàn bộ thanh ghi đường ống EX/MEM được xóa về 0. Các tín hiệu ALU_ResultM, WriteDataM, RegWriteM, MemWriteM, ResultSrcM và FPRegWriteM đều bằng 0, xác nhận logic reset hoạt động đúng.

Hình 4.11 còn biểu diễn kết quả từ 16ns đến 26ns, hai lệnh `addi x1, x0, 10` (0x00A00093) và `addi x2, x0, 3` (0x00300113) được thực thi liên tiếp tại thời điểm 16 ns và thời điểm 26 ns. Cả hai lệnh có tín hiệu $ALUSrcE = 1$, khối ALU cộng giá trị tức thì với $RD1_E = 0$. Kết quả ALU_ResultM lần lượt là 0x0000000A (10 thập phân) và 0x00000003 (3 thập phân) (1). $RegWriteM = 1$ và RD_M đúng với thanh ghi đích x1 và x2, xác nhận kết quả sẽ được ghi về tập thanh ghi ở tầng ghi lại.



Hình 4.11: Kết quả dạng sóng mô phỏng của tầng thực thi từ 15ns đến 35ns

Theo hình 4.12, từ 36ns đến 66ns, các lệnh ADD, SUB, AND và OR được thực thi liên tiếp, tín hiệu ALUSrcE = 0, lấy trực tiếp giá trị từ RD1_E = 10 (x1) và RD2_E = 3 (x2) làm toán hạng. Tại thời điểm 36ns, lệnh add x3, x1, x2 với ALUControlE = 000 cho ALU_ResultM = 0x0000000D (13 thập phân), đúng với $10 + 3 = 13$. Tại thời điểm 46ns, lệnh sub x4, x1, x2 với ALUControlE = 001 sử dụng bộ cộng 33 bit chế độ bù 2, cho ALU_ResultM = 0x00000007 (7 thập phân), đúng với $10 - 3 = 7$. Tại thời điểm 56ns, lệnh and x5, x1, x2 với ALUControlE = 010 cho ALU_ResultM = 0x00000002 (2 thập phân), đúng với $0b1010 \& 0b0011 = 0b0010 = 2$. Tại thời điểm 66ns, lệnh or x6, x1, x2 với ALUControlE = 011 cho ALU_ResultM = 0x0000000B (11 thập phân), đúng với $0b1010 | 0b0011 = 0b1011 = 11$ (2).

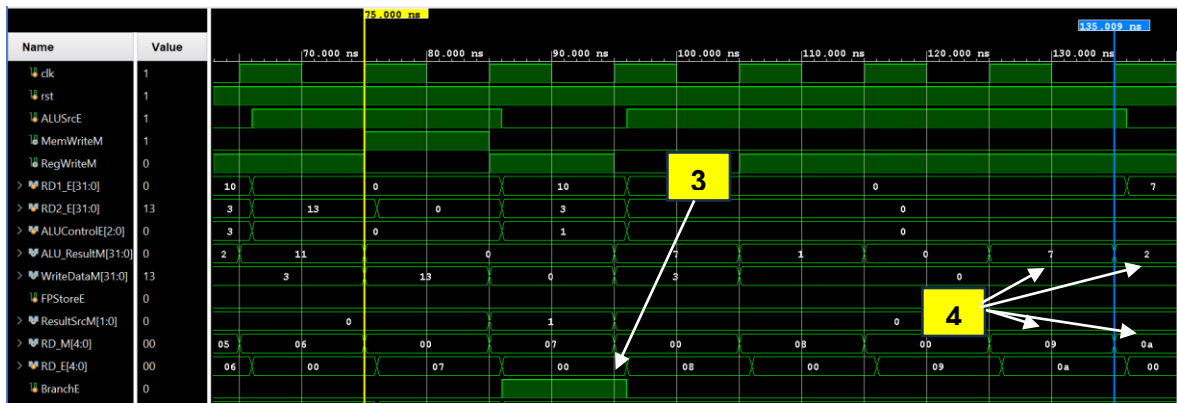


Hình 4.12: Kết quả dạng sóng mô phỏng của tầng thực thi từ 36ns đến 66ns

Quan sát thời điểm từ 76ns đến 86ns trên hình 4.13, lệnh sw x3, 0 (x0) kích hoạt tín hiệu MemWriteM lên mức 1 và RegWriteM có giá trị là 0. ALU thực hiện phép cộng $0 + 0 = 0$, đưa ra địa chỉ ghi ALU_ResultM = 0x00000000. WriteDataM = 0x0000000D (13 thập phân) là giá trị của x3 được chọn qua bộ chọn kênh, FPStoreE có giá trị là 0 xác nhận dữ liệu ghi lấy từ thanh ghi số nguyên, không phải thanh ghi dấu phẩy động. Lệnh lw x7, 0 (x0) cho MemWriteM = 0, RegWriteM = 1, ResultSrcM = 01, chỉ thị tầng ghi lại lấy dữ liệu từ bộ nhớ. ALU tính địa chỉ đọc $0 + 0 = 0$ đúng với offset = 0 và RD_M = 7 xác nhận kết quả sẽ ghi vào x7.

Tại 86ns, lệnh rẽ nhánh beq x1, x2, +8 được thực thi (3), các dữ liệu từ thanh ghi số nguyên nguồn 1 và nguồn 2 lần lượt là RD1_E = 10 và RD2_E = 3, ALU thực hiện phép trừ theo ALUControlE có giá trị là 001, cờ ZeroE = 0 do 10 khác 3, kết hợp với tín hiệu BranchE = 1 nên tín hiệu PCSrcE = 0 và rẽ nhánh không được thực hiện.

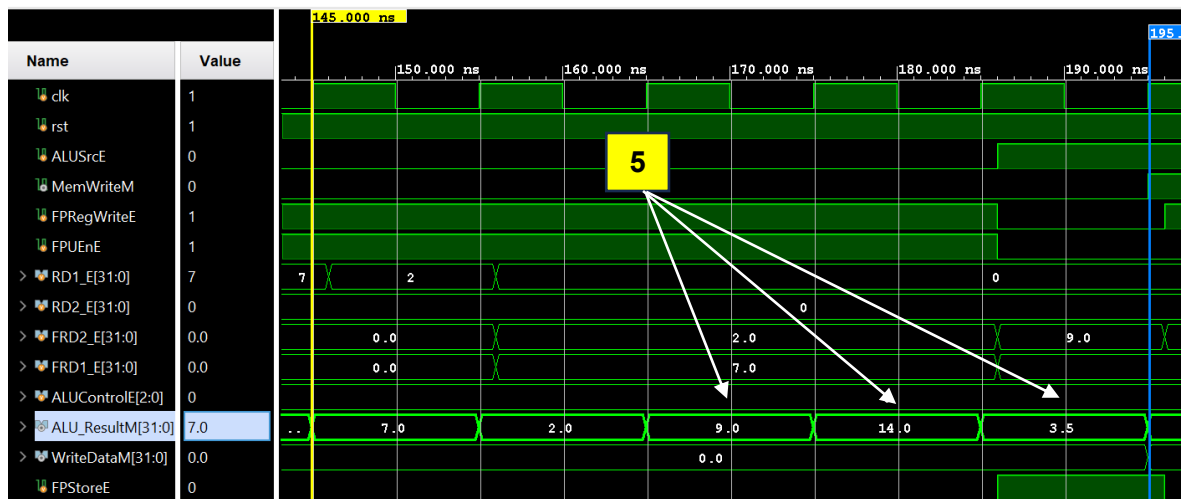
Từ 126ns đến 136ns, kết quả của thời điểm này thể hiện qua hình 4.13, hai lệnh addi lần lượt là addi x9, x0, 7 và addi x10, x0, 2 chuẩn bị giá trị nguyên cho các lệnh chuyển đổi FP tiếp theo. Giá trị của ALU_ResultM lần lượt là 0x00000007 và 0x00000002, và RD_M là 9 và 10, xác nhận giá trị sẽ được lưu vào x9 và x10 (4).



Hình 4.13: Kết quả dạng sóng mô phỏng của tầng thực thi từ 85ns đến 135ns

Theo hình 4.14, từ 146ns đến 156ns, các lệnh dấu phẩy động được thực thi lần lượt là lệnh fcvt.s.w f0, x9 chuyển đổi số nguyên 7 và fcvt.s.w f1, x10 chuyển đổi số nguyên 2 thành số có dấu phẩy động. Tín hiệu FPUEnE = 1 kích hoạt khối FPU, fp_opE[15] = 1 chọn phép chuyển đổi FCVT.S.W. RD1_E = 7 đi qua bộ chọn kênh chuyển tiếp, đưa giá trị 7 vào ngõ toán hạng nguyên của khối FPU. Kết quả giá trị ALU_ResultM = 0x40E00000 tương ứng 7.0 và fcvt.s.w f1, x10 chuyển đổi số nguyên 2 thành 0x40000000, hoàn tất việc nạp giá trị cho f0 và f1 chuẩn bị cho các phép tính dấu phẩy động.

Từ 166ns đến 186ns, các lệnh fadd, fmul, fdiv lần lượt được thực thi, cả 3 lệnh duy trì tín hiệu FPUEnE = 1, FPRegWriteM = 1 và RegWriteM = 0 trong khoảng thời gian này. Tại thời điểm 166 ns, lệnh fadd.s f2, f0, f1 với FRD1_E = 0x40E00000 (7.0) và FRD2_E = 0x40000000 (2.0) cho kết quả ALU_ResultM = 0x41100000 (9.0) (5), đúng với $7.0 + 2.0$. Tại thời điểm 176 ns, fmul.s f3, f0, f1 cho kết quả ALU_ResultM = 0x41600000 (14.0) (5), đúng với 7.0×2.0 , fflags = 00000. Tại thời điểm 186 ns, lệnh fdiv.s f4, f0, f1 cho ALU_ResultM = 0x40600000 (3.5) (5), đúng với $7.0 \div 2.0$, fflags = 00000. Ba chu kỳ này xác nhận khối FPU xử lý chính xác cả ba phép tính cơ bản với tín hiệu fp_opE one-hot phân biệt đúng từng phép theo trường funct5 của mã lệnh.



Hình 4.14: Kết quả mô phỏng của khối thực thi từ 145ns đến 185ns

4.1.4. Khối truy cập bộ nhớ

Bài kiểm tra chức năng của tầng truy cập bộ nhớ được xây dựng với năm nhóm kiểm tra nhằm xác nhận toàn bộ chức năng của tầng truy cập bộ nhớ trong bộ xử lý RISC-V 32 bit.

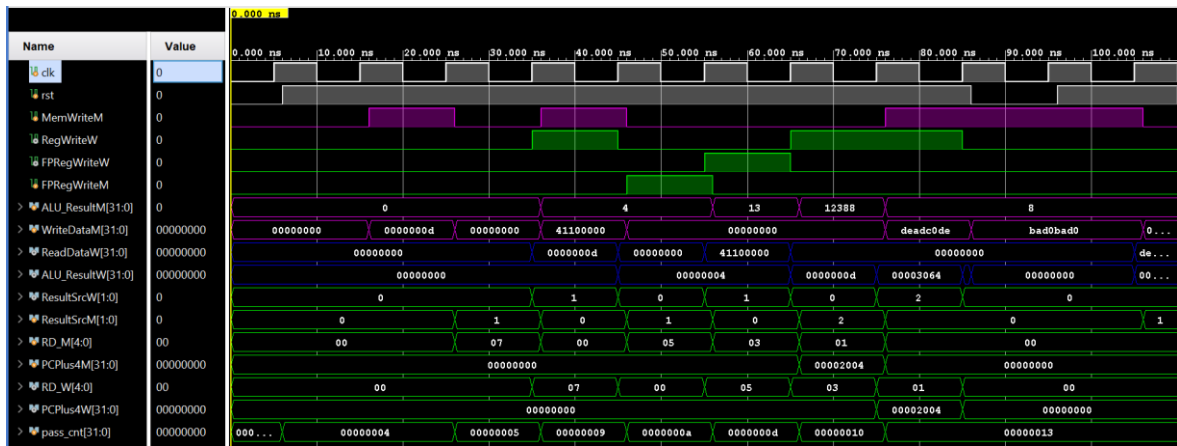
Nhóm đầu tiên kiểm tra tín hiệu reset, xác nhận rằng toàn bộ thanh ghi đường ống MEM/WB được xóa về 0 khi tín hiệu reset tích cực mức thấp.

Nhóm thứ hai kiểm tra cặp lệnh lưu trữ và nạp số nguyên SW/LW, xác nhận rằng dữ liệu được ghi đồng bộ vào bộ nhớ và đọc ra chính xác sau một chu kỳ.

Nhóm thứ ba kiểm tra tương tự với cặp lệnh dấu phẩy động FSW/FLW, đồng thời kiểm tra tín hiệu FPURegWriteW phân biệt đích ghi giữa tập thanh ghi nguyên và tập thanh ghi FP.

Nhóm thứ tư kiểm tra đường truyền thẳng kết quả ALU với ResultSrc = 00, xác nhận ALU_ResultW truyền thẳng không qua bộ nhớ.

Nhóm cuối cùng kiểm tra cơ chế bảo vệ ghi bộ nhớ, xác nhận rằng lệnh ghi bị chặn hoàn toàn khi tín hiệu reset tích cực và giá trị cũ được bảo toàn. Toàn bộ kết quả xem tại hình 4.15.



Hình 4.15: Kết quả mô phỏng chức năng của tầng truy cập bộ nhớ

Từ 0ns đến 15ns, tín hiệu rst duy trì mức thấp, toàn bộ thanh ghi đường ống MEM/WB được xóa về 0, các tín hiệu ALU_ResultW, ReadDataW, RegWriteW và FRegWriteW đều bằng 0, xác nhận tín hiệu reset hoạt động đúng.

Từ 25ns đến 45ns, lệnh sw cùng với MemWriteM = 1, ALU_ResultM = 0x00000000 (địa chỉ của byte 0) và giá trị của WriteDataM=0x0000000D (13 thập phân). Bộ nhớ dữ liệu được ghi đồng bộ tại sườn lên, địa chỉ mem[0] = 13 được lưu, thanh ghi đường ống cho ra tín hiệu RegWriteW=0, xác nhận lệnh sw không ghi về tập thanh ghi. Ngoài ra lệnh lw được thực hiện tại cùng địa chỉ với ResultSrcM = 01 và RD_M = 7. Bộ nhớ dữ liệu đọc tổ hợp trả về RD=13, chốt giá trị vào ReadDataW = 0x0000000D, đồng thời RegWriteW = 1, ResultSrcW=01 và RD_W=7 xác nhận kết quả sẽ được tăng ghi lại lấy từ bộ nhớ và ghi vào thanh ghi x7.

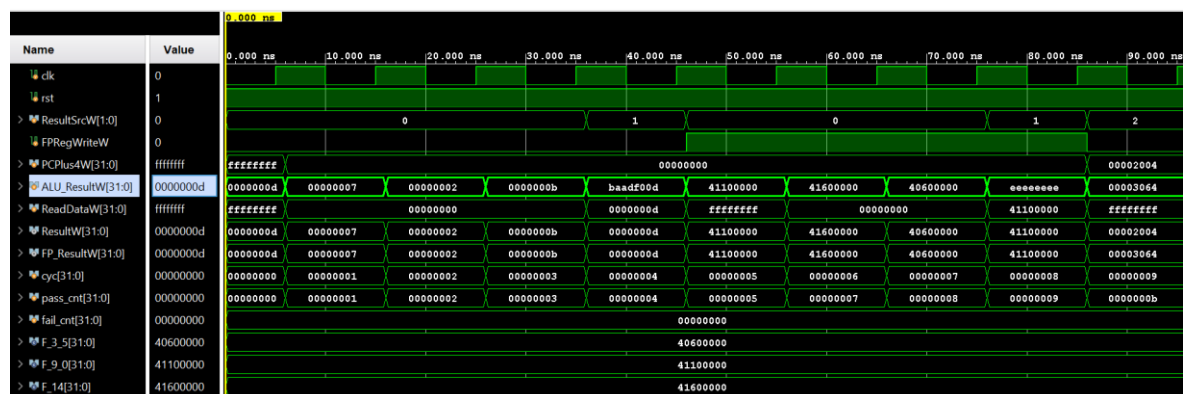
Từ 45ns đến 65ns, thực hiện fsw với ALU_ResultM = 0x00000004 (địa chỉ byte 4) và WriteDataM = 0x41100000 (9.0). Địa chỉ mem[1] = 9.0 được ghi, RegWriteW = 0 xác nhận lệnh fsw không ghi vào thanh ghi. Tiếp theo, thực hiện flw tại địa chỉ 4 với FRegWriteM = 1, ResultSrcM = 01, RD_M = 5. Giá trị của ReadDataW = 0x41100000 khớp chính xác với giá trị đã lưu, ngoài ra tín hiệu FRegWriteW = 1 trong khi RegWriteW = 0, phân biệt đúng đích ghi là tập thanh ghi dấu phẩy động chứ không phải thanh ghi nguyên.

Từ 65 ns đến 85ns, đường truyền thẳng kết quả của ALU với ResultSrcM = 00, ALU_ResultM = 0x0000000D và RD_M = 3. Không truy cập bộ nhớ, thanh ghi đường ống chốt thẳng giá trị ALU_ResultW=0x0000000D, ResultSrcW=00, RD_W=3 và tăng ghi lại sẽ lấy kết quả từ ALU, đúng với nhóm lệnh số học nguyên.

Từ 85ns đến 115ns, ghi bình thường WriteDataM = 0xDEADC0DE vào địa chỉ 8 của bộ nhớ, địa chỉ mem[2] nhận giá trị này. Sau đó, tín hiệu reset bị kéo xuống mức thấp trong khi MemWriteM = 1 và WriteDataM = 0xBAD0BAD0, điều kiện ghi đồng bộ trong bộ nhớ dữ liệu không được thỏa mãn do reset = 0, việc ghi bị chặn hoàn toàn. Sau một chu kỳ để tín hiệu rst_r được chốt lại, hệ thống đọc lại địa chỉ 8 và nhận ReadDataW = 0xDEADC0DE là giá trị hợp lệ đã ghi trước đó được bảo toàn, xác nhận cơ chế bảo vệ bộ nhớ khỏi ghi nhầm trong quá trình reset hoạt động chính xác.

4.1.5. Khối ghi lại

Bài kiểm tra tầng ghi lại được xây dựng với năm nhóm kiểm tra nhằm xác nhận chức năng của khối chọn kết quả cuối cùng trước khi ghi vào tập thanh ghi. Khác với các tầng trước, tầng ghi lại là mạch tổ hợp thuần túy, không có thanh ghi đường ống, không có flip-flop, đầu ra thay đổi ngay lập tức khi đầu vào thay đổi. Toàn bộ logic của tầng này gói gọn trong một bộ chọn kênh 3 ngõ vào điều khiển bởi tín hiệu ResultSrcW với giá trị 00 chọn kết quả từ khối ALU, giá trị 01 chọn dữ liệu đọc từ bộ nhớ, giá trị 10 chọn địa chỉ quay về PC+4 phục vụ lệnh nhảy JAL/JALR. Riêng với kết quả dấu phẩy động, FP_ResultW lấy từ bộ nhớ khi ResultSrcW = 01 (lệnh FLW) và lấy từ kết quả ALU/FPU trong mọi trường hợp còn lại, chi tiết xem tại hình 4.16.



Hình 4.16: Kết quả mô phỏng của tầng ghi lại

Từ 5ns đến 45ns, bốn lệnh ADD, SUB, AND, OR được kiểm tra liên tiếp qua bốn chu kỳ, tất cả đều có tín hiệu ResultSrcW = 00, bộ chọn kênh chọn ALU_ResultW làm đầu ra. Để tăng độ tin cậy, bộ kiểm tra còn nạp các giá trị rác vào ReadDataW và PCPlus4W là 0xFFFFFFFF nhằm xác nhận bộ chọn kênh bỏ qua hoàn toàn hai đầu vào còn lại. Kết quả ResultW lần lượt ra đúng 13, 7, 2, 11 và khớp với giá trị nhất

quán từ các tầng nạp lệnh cho tới tầng truy cập bộ nhớ trong cùng chuỗi lệnh kiểm tra.

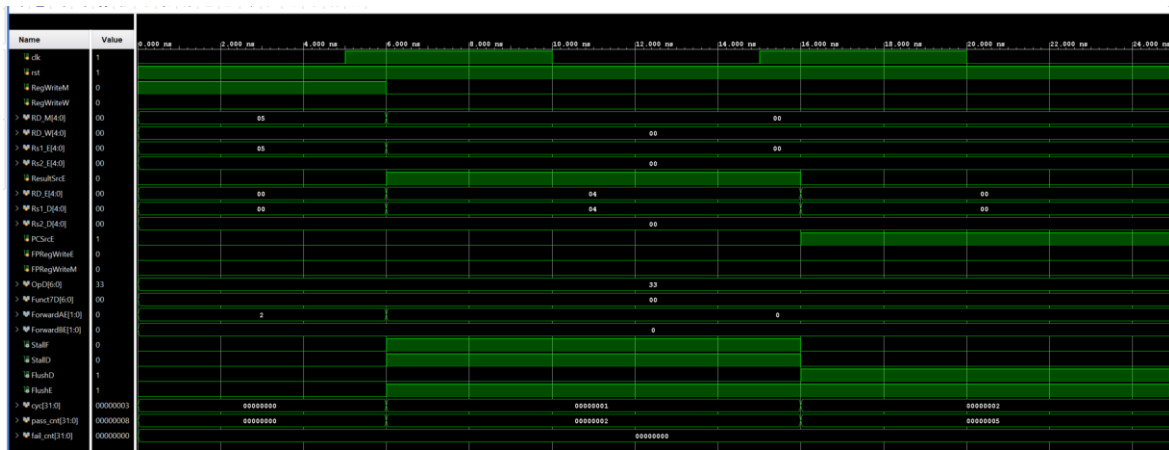
Từ 45ns đến 55ns, kiểm tra lệnh lw với ResultSrcW = 01 và ReadDataW = 13. Bộ kiểm tra nạp ALU_ResultW = 0xBAADF00D để có thể quan sát, bộ chọn kênh phải bỏ qua giá trị này và chọn ReadDataW. Đầu ra ResultW = 13 xác nhận bộ chọn kênh hoạt động đúng, giá trị 13 nhất quán với kết quả sw và lw đã kiểm tra tại tầng truy cập bộ nhớ, xác nhận tính liên thông dữ liệu xuyên suốt các tầng.

Từ 55ns đến 85ns, Ba lệnh FP số học được kiểm tra với tín hiệu FPRegWriteW = 1 và ResultSrcW = 00. Kết quả tính toán từ FPU nằm trong ALU_ResultW, tầng ghi lại gán thẳng FP_ResultW = ALU_ResultW. Bộ kiểm tra nạp dữ liệu ReadDataW = 0xFFFFFFFF làm nhiễu để xác nhận FP_ResultW không lấy nhầm từ bộ nhớ. Kết quả lần lượt: lệnh fadd cho kết quả FP_ResultW = 0x41100000 (9.0), lệnh fmul cho kết quả 0x41600000 (14.0), lệnh fdiv cho kết quả 0x40600000 (3.5), đúng với các giá trị từ chuỗi lệnh kiểm thử.

Từ 85ns đến 95ns, kiểm tra lệnh FLW trong trường hợp duy nhất FP_ResultW lấy từ bộ nhớ thay vì FPU. Tín hiệu ResultSrcW = 01, ReadDataW = 0x41100000 (9.0), ALU_ResultW = 0xFFFFFFFF. Kết quả của cả ResultW lẫn FP_ResultW đều ra giá trị 0x41100000, xác nhận nhánh ResultSrcW trong mạch tổ hợp hoạt động chính xác và hai đầu ra dùng chung cùng một bộ chọn kênh nguồn.

4.1.6. Khối xử lý xung đột

Bài kiểm tra khối xử lý xung đột được xây dựng với ba bài kiểm tra, tương ứng ba loại xung đột cơ bản phát sinh trong kiến trúc đường ống năm tầng của bộ xử lý RISC-V 32 bit. Khối này là mạch tổ hợp thuần túy, không chứa thanh ghi đường ống, nên đầu ra thay đổi tức thời theo đầu vào. Phần thứ nhất kiểm tra cơ chế chuyển tiếp dữ liệu, giải quyết tình huống một lệnh cần kết quả của lệnh phía trước khi kết quả đó chưa kịp ghi vào tập thanh ghi. Phần thứ hai kiểm tra cơ chế dừng đường ống, xử lý phụ thuộc với lệnh nạp khi dữ liệu từ bộ nhớ về quá trễ để chuyển tiếp kịp. Phần thứ ba kiểm tra cơ chế xóa lệnh khi rẽ nhánh được chọn, loại bỏ các lệnh đã nạp nhầm theo luồng tuần tự.



Hình 4.17: Kết quả mô phỏng khối xử lý xung đột

Từ 6ns đến 16ns, kiểm tra cơ chế chuyển tiếp dữ liệu, mô phỏng tình huống lệnh add x5, x1, x2 đang ở tầng truy cập bộ nhớ và đã tính xong giá trị x5, trong khi lệnh sub x9, x5, x3 ngay sau đang ở tầng thực thi cần dùng x5 làm toán hạng. Do x5 chưa kịp ghi vào tập thanh ghi, khối kích hoạt ForwardAE = 10 để chuyển kết quả thẳng từ tầng truy cập bộ nhớ về tầng Thực thi, tránh phải đọc giá trị cũ sai trong tập thanh ghi. Đồng thời tín hiệu StallF = 0 xác nhận việc chuyển tiếp được thực hiện ngay mà không cần dừng đường ống, giữ nguyên hiệu năng.

Từ 16ns đến 26ns, kiểm tra cơ chế dừng đường ống bằng mô phỏng tình huống lệnh lw x4, 0(x2) đang ở tầng Thực thi với ResultSrcE = 1, trong khi lệnh ngay sau ở tầng Giải mã cần dùng x4. Vì dữ liệu nạp từ bộ nhớ về trễ hơn một nhịp, cơ chế chuyển tiếp không kịp đáp ứng, khối buộc phải dừng đường ống một chu kỳ. Tín hiệu StallF = 1 và StallD = 1 đóng băng tầng Nạp lệnh và tầng Giải mã, đồng thời FlushE = 1 chen một lệnh rỗng (bong bóng) vào tầng Thực thi để lấp khoảng trống.

Từ 26ns đến 36ns, kiểm tra cơ chế xóa lệnh khi rẽ nhánh bằng cách mô phỏng lệnh beq x1, x2, +8 được xác định rẽ nhánh, biểu diễn qua tín hiệu PCSrcE = 1. Khi đó hai lệnh đã nạp sẵn theo luồng tuần tự phía sau lệnh rẽ nhánh trở thành sai và phải bị loại bỏ. Khối xử lý xung đột kích hoạt FlushD = 1 và FlushE = 1 để xóa hai lệnh đang nằm trong tầng Giải mã và tầng Thực thi, trong khi StallF = 0 xác nhận đây là thao tác xóa lệnh chứ không phải dừng, đường ống tiếp tục nạp lệnh từ địa chỉ đích mới.

4.2. KẾT QUẢ MÔ PHỎNG TOÀN HỆ THỐNG

4.2.1. Bài kiểm tra số 1

Ở bài kiểm tra chương trình sẽ thực hiện việc tính toán chuyển đổi nhiệt độ theo công thức:

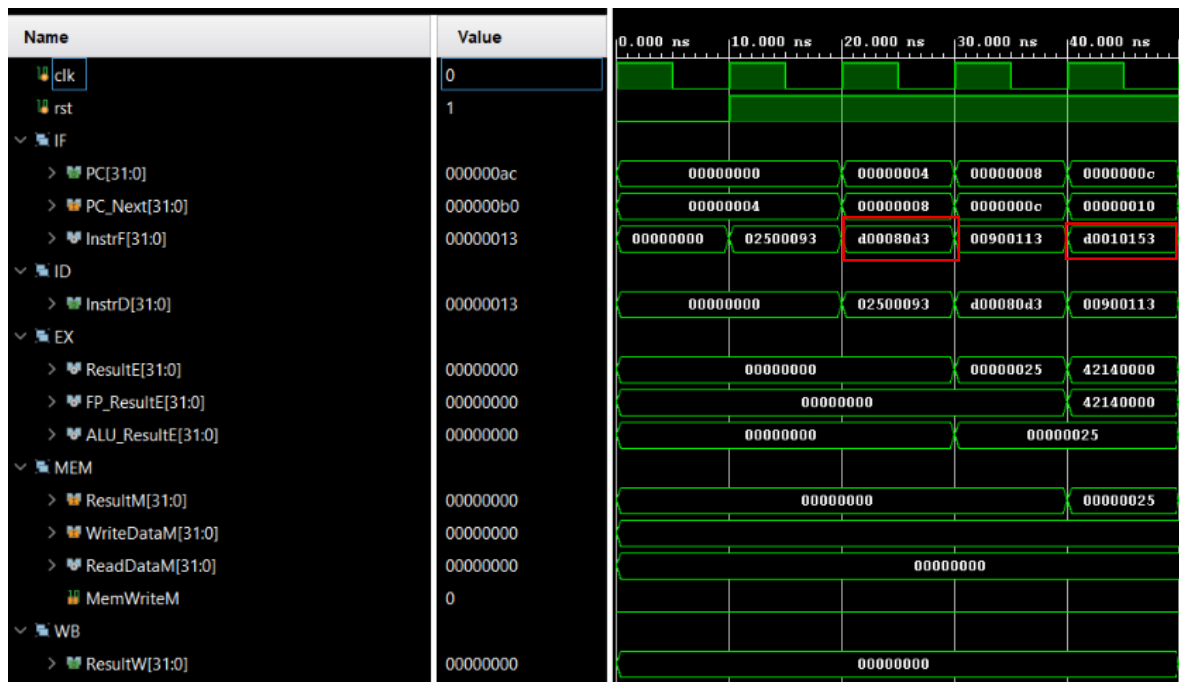
$$F = \frac{9}{5} \times C + 32$$

Với giá trị đầu vào $C = 37^{\circ}\text{C}$, kết quả kỳ vọng là $F = 98.6^{\circ}\text{F}$.

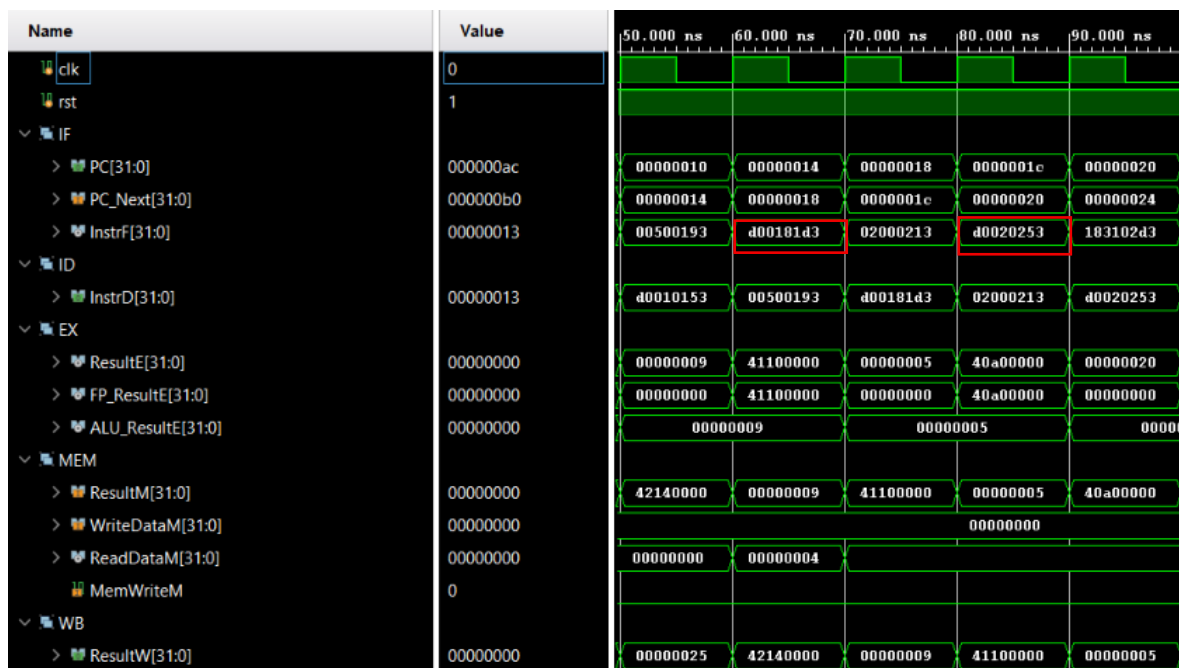
Bảng 4.1: Bảng mã bài kiểm tra số 1

STT	Mã hex	Mô tả lệnh	Kết quả dự kiến
1	02500093	addi x1,x0,37	x1 = 37 (0x00000025)
2	D00D80D3	fcvt.s.w f1,x1	f1 = 37.0 (0x42140000)
3	00900113	addi x2,x0,9	x2 = 9 (0x00000009)
4	D0010153	fcvt.s.w f2,x2	f2 = 9.0 (0x41100000)
5	00500193	addi x3,x0,5	x3 = 5 (0x00000005)
6	D00181D3	fcvt.s.w f3,x3	f3 = 5.0 (0x40A00000)
7	02000213	addi x4,x0,32	x4 = 32 (0x00000020)
8	D0020253	fcvt.s.w f4,x4	f4 = 32.0 (0x42000000)
9	183102D3	fdiv.s f5,f2,f3	f5 = 1.8 (0x3FE66666)
10	10128353	fmul.s f6,f5,f1	f6 = 66.6 (0x42853332)
11	004303D3	fadd.s f7,f6,f4	f7 = 98.6 (0x42C53332)

Bảng 4.1 trình bày trường hợp kiểm thử thứ nhất được sử dụng để đánh giá hoạt động của CPU. Trường hợp kiểm thử này được xây dựng nhằm kiểm tra khả năng thực thi các lệnh số thực của phần mở rộng RV32F, đồng thời xác minh sự phối hợp giữa khối xử lý số nguyên, khối xử lý số thực và kiến trúc đường ống 5 tầng trong quá trình thực thi chương trình.



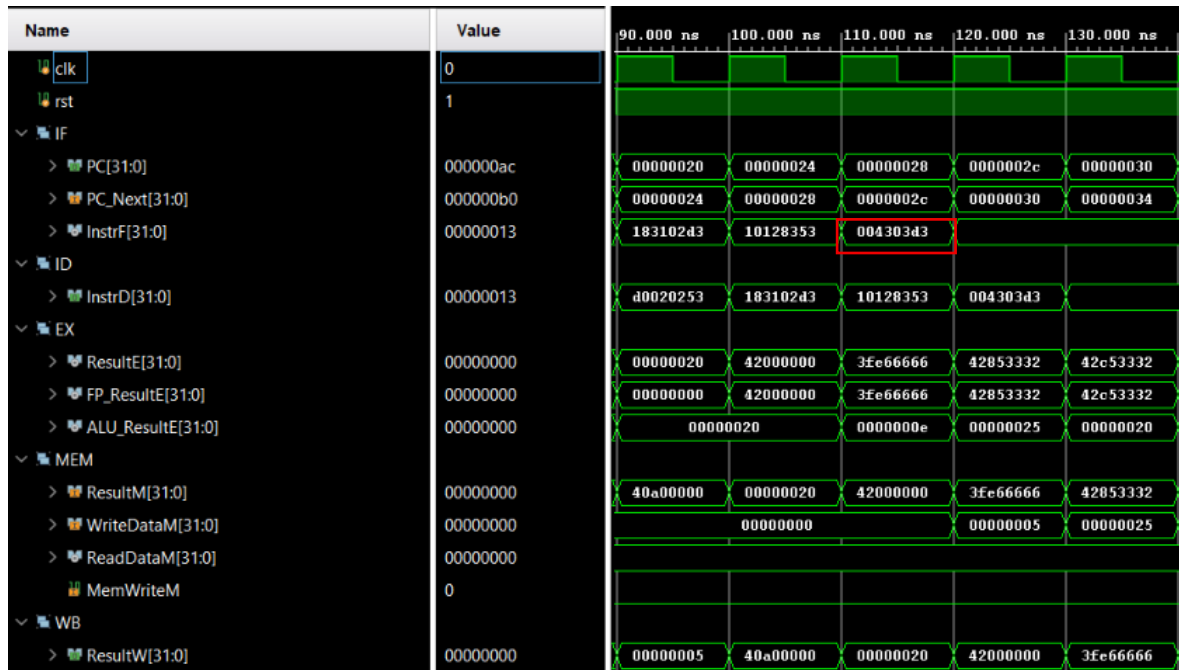
Hình 4.18: Dạng sóng thực thi các lệnh của bài kiểm tra số 1 từ 0ns đến 40ns



Hình 4.19: Dạng sóng thực thi các lệnh của bài kiểm tra số 1 từ 50ns đến 90ns

Phân tích bảng 4.1 ta thấy các lệnh fcvt.s.w đều bị phụ thuộc vào kết quả của lệnh trước đó, đề tài đã áp dụng cơ chế chuyển tiếp để giải quyết vấn đề xung đột này nên các lệnh fcvt.s.w sẽ thực thi luôn mà không cần chờ lệnh trước đó hoàn thành việc ghi kết quả vào thanh ghi. Hình 4.18 và hình 4.19 cho thấy các lệnh fcvt.s.w tại khối thực thi được thực thi ngay sau các lệnh mà nó phụ thuộc. Hình 4.18 cho thấy

tại thời điểm 50ns, kết quả của lệnh đầu tiên đã được tính toán xong. Điều này cho thấy CPU đang hoạt động theo cơ chế đường ống 5 tầng.



Hình 4.20: Dạng sóng thực thi các lệnh của bài kiểm tra số 1 từ 90ns đến 130ns

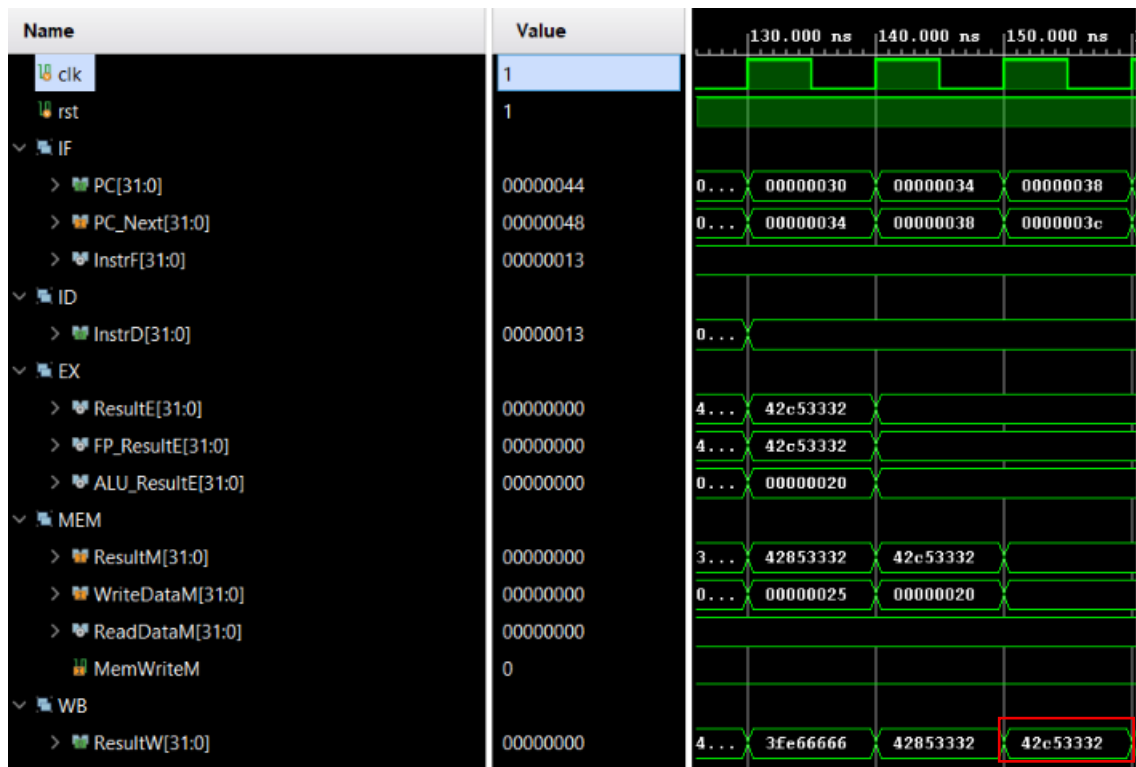
Theo hình 4.20, tại thời điểm 110ns tín hiệu InstrF bằng 004300d3 tức lệnh cuối cùng đã được nạp vào CPU. Hình 4.21 cho thấy tại thời điểm 150ns tín hiệu ResultW cho ra kết quả của lệnh cuối cùng là 42c53332 tương đương với 98.59999 đúng với kết quả mong đợi.

Bài kiểm tra số 1 đã thực hiện phép biến đổi từ độ C sang độ F chương trình gồm 11 lệnh. Kiến trúc CPU trong đề tài là đường ống 5 tầng và thực thi theo lệnh theo thứ tự. Trong điều kiện khi không có trì hoãn hoặc xóa, số chu kỳ cần để hoàn tất một chương trình được tính theo công thức sau:

$$T_{pipeline} = N + (S - 1)$$

Trong đó, N là số lệnh thực hiện, S là số tầng. Áp dụng vào bài kiểm tra số 1, với CPU kiến trúc 5 tầng và thực thi 11 lệnh, số chu kỳ cần để thực thi là:

$$T_{pipeline} = 11 + (5 - 1) = 15 \text{ chu kỳ}$$



Hình 4.21: Dạng sóng thực thi các lệnh của bài kiểm tra số 1 từ 130ns đến 150ns

Đối với kiến trúc không đường ống, mỗi lệnh phải lần lượt đi qua đầy đủ 5 tầng xử lý trước khi lệnh tiếp theo được bắt đầu. Do đó, số chu kỳ thực thi được tính theo công thức sau:

$$T_{không\ pipeline} = N \times S$$

Áp dụng đối với bài kiểm tra số 1 thì tổng số chu kỳ cần thực hiện đối với kiến trúc không đường ống là:

$$T_{không\ pipeline} = 11 \times 5 = 55\ chu\ kỳ$$

Như vậy, đối với bài kiểm tra số 1, kiến trúc CPU đường ống 5 tầng cho thấy hiệu quả xử lý tốt hơn so với kiến trúc không đường ống. Trong điều kiện lý tưởng, không xét trì hoãn do xung đột hoặc xóa, CPU đường ống chỉ cần 15 chu kỳ clock để hoàn thành 11 lệnh, trong khi kiến trúc không đường ống cần 55 chu kỳ clock. Do đó, kiến trúc đường ống giúp cải thiện hiệu năng khoảng 3.67 lần.

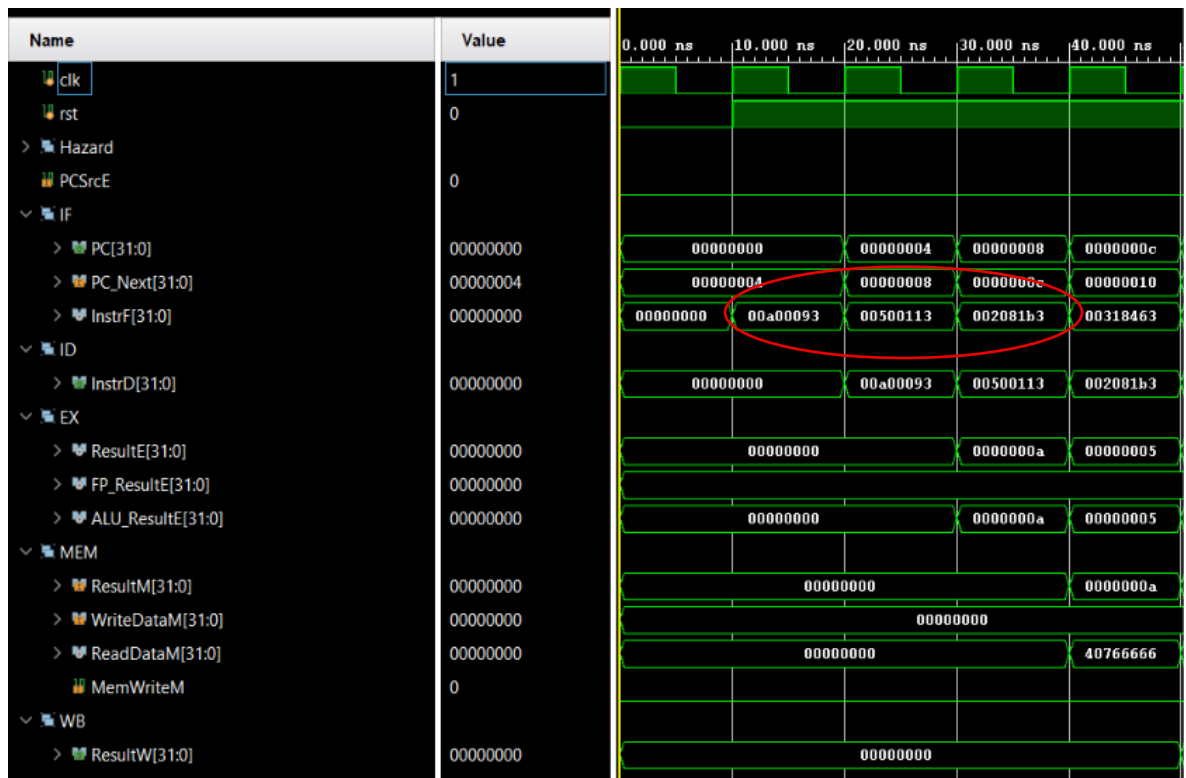
4.2.2. Bài kiểm tra số 2

Ở bài kiểm tra số 2, chương trình sẽ áp dụng các lệnh loại B nhằm kiểm tra khả năng xử lý lệnh rẽ nhánh của CPU. Cụ thể, bài kiểm tra tập trung đánh giá việc so

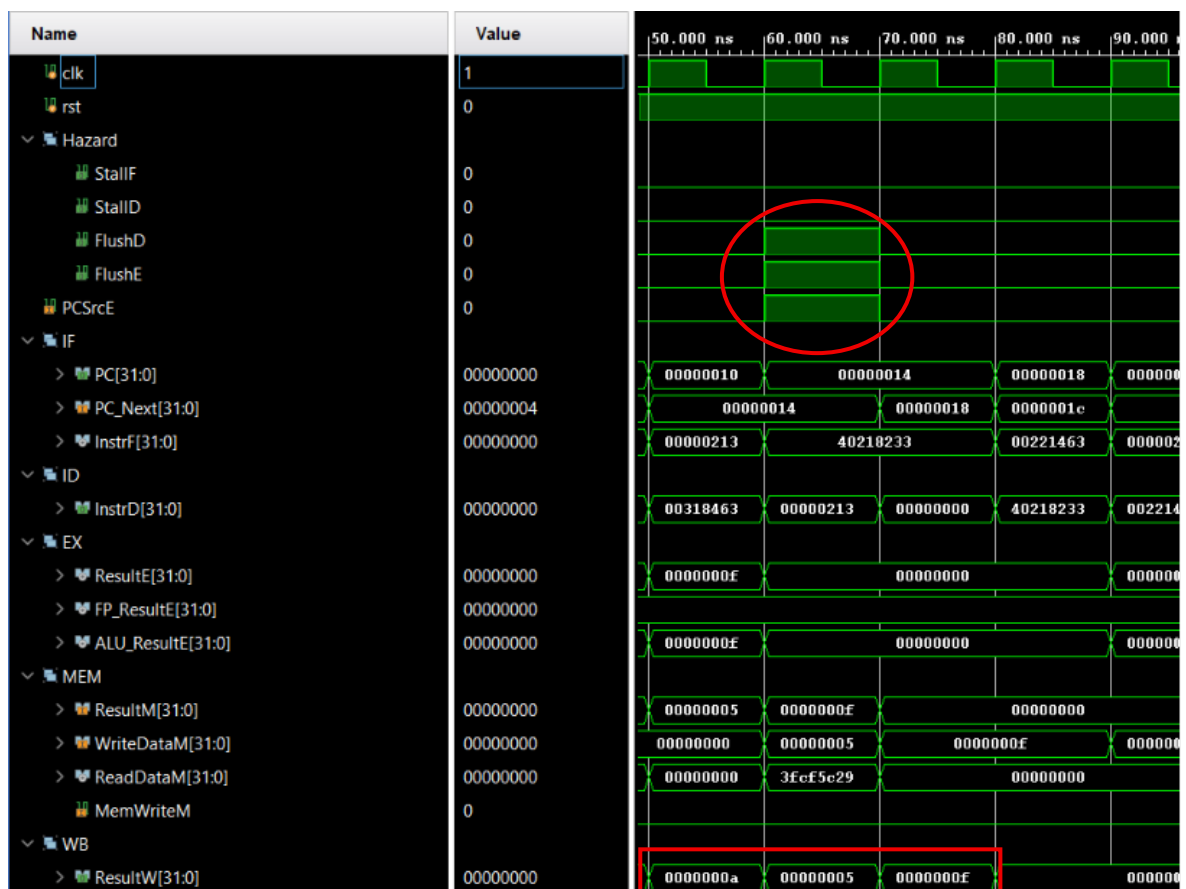
sánh dữ liệu giữa hai thanh ghi, xác định điều kiện nhánh và cập nhật giá trị bộ đếm chương trình khi điều kiện rẽ nhánh được thỏa mãn hoặc không thỏa mãn.

Bảng 4.2: Bảng mã bài kiểm tra số 2

STT	Mã hex	Mô tả lệnh	Kết quả dự kiến
1	00A00093	addi x1,x0,10	$x1 = 10$ (0x0000000A)
2	00500113	addi x2,x0,5	$x2 = 5$ (0x00000005)
3	002081B3	add x3,x1,x2	$x3 = 10 + 5 = 15$ (0x0000000F)
4	00318463	beq x3,x3,+8	$x3 = x3$, điều kiện đúng, bỏ qua lệnh 5
5	00000213	addi x4,x0,0	Không được thực thi
6	40218233	sub x4,x3,x2	$x4 = 15 - 5 = 10$ (0x0000000A)
7	00221463	bne x4,x2,+8	$x4 \neq x2$, điều kiện đúng, bỏ qua lệnh 8
8	00000293	addi x5,x0,0	Không được thực thi
9	022102B3	mul x5,x2,x2	$x5 = 5 \times 5 = 25$ (0x00000019)
10	0222C333	div x6,x5,x2	$x6 = 25 / 5 = 5$ (0x00000005)
11	00134463	blt x6,x1,+8	$5 < 10$, nhánh được lấy, bỏ qua lệnh 12
12	00000393	addi x7,x0,0	Không được thực thi
13	002303B3	add x7,x6,x2	$x7 = 5 + 5 = 10$ (0x0000000A)
14	00139463	bne x7,x1,+8	$x7 = x1 = 10$ sai, điều kiện sai
15	00100413	addi x8,x0,1	$x8 = 1$ (0x00000001)



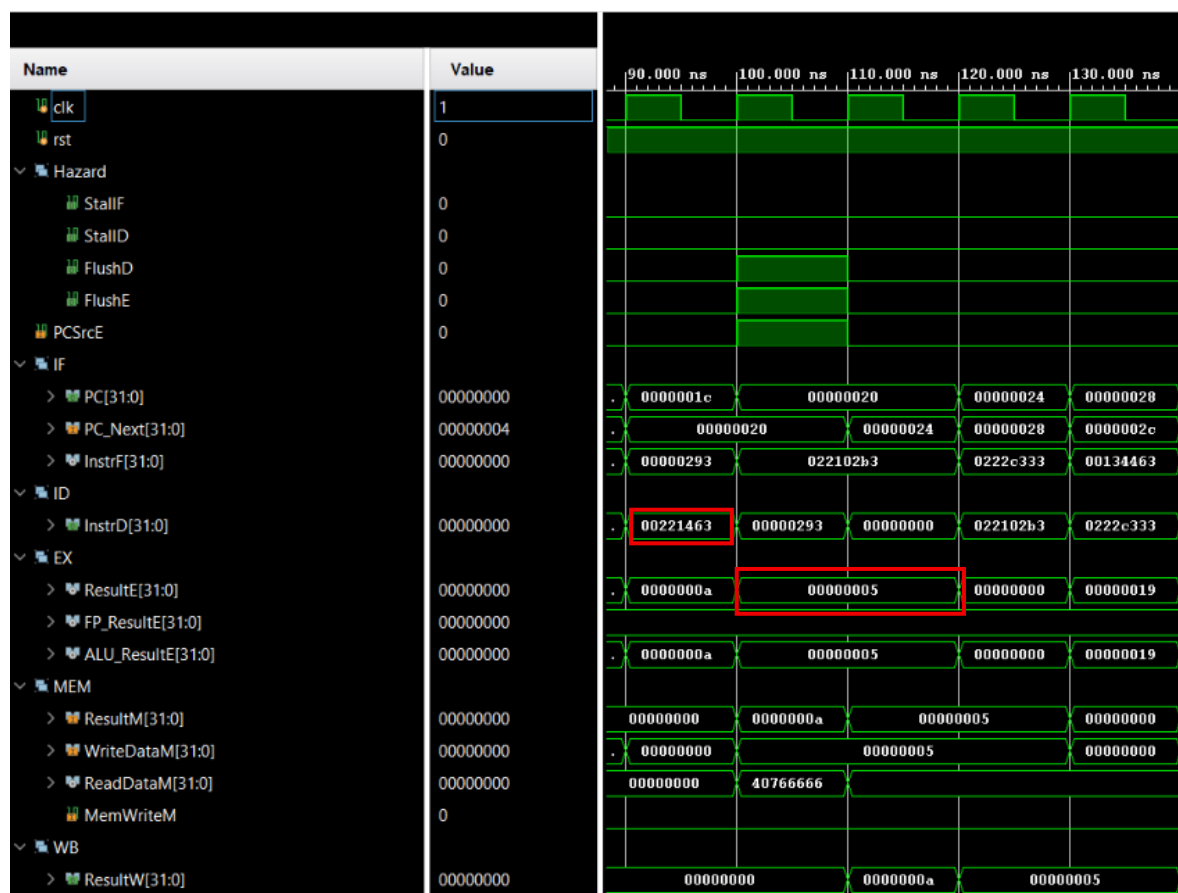
Hình 4.22: Dạng sóng thực thi các lệnh của bài kiểm tra số 2 từ 0ns đến 40ns



Hình 4.23: Dạng sóng thực thi các lệnh của bài kiểm tra số 2 từ 50ns đến 90ns

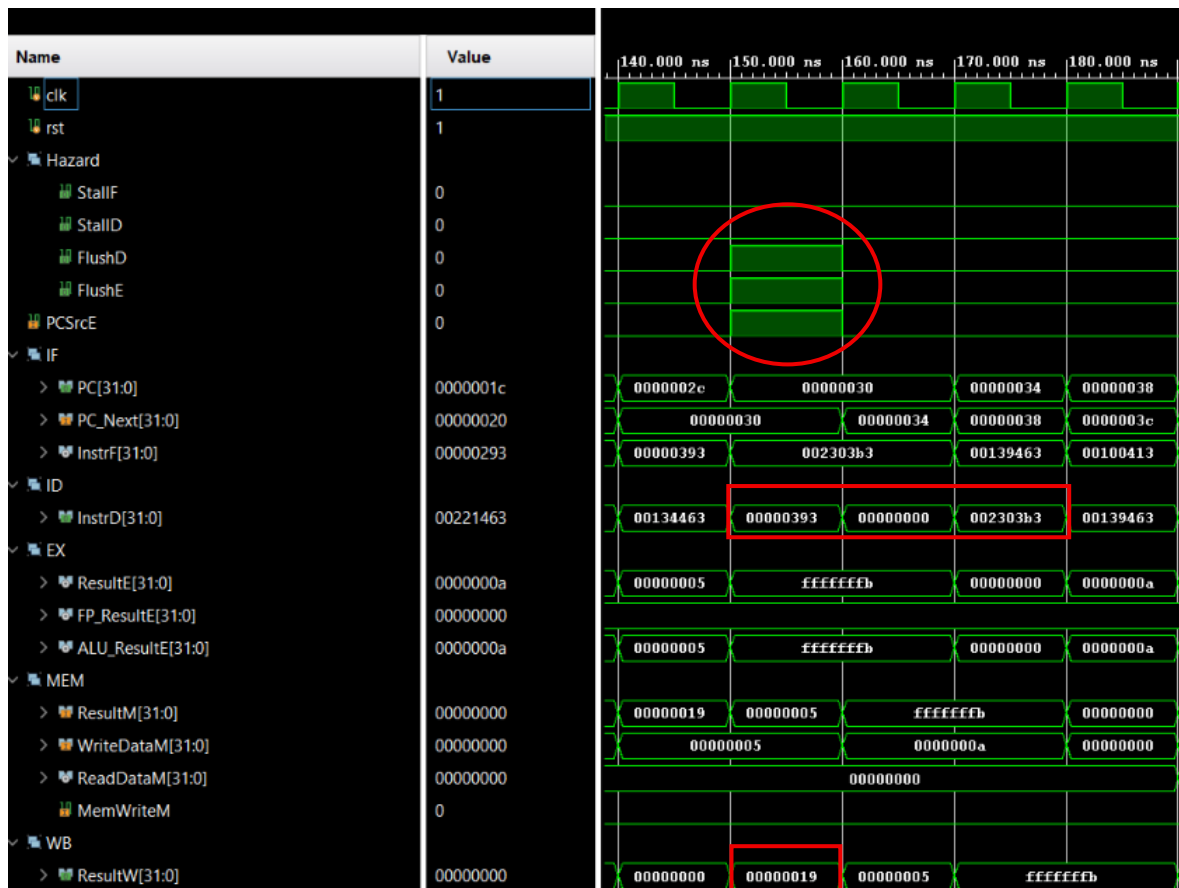
Hình 4.22 và hình 4.23 minh họa hoạt động của CPU trong khoảng thời gian từ 0 ns đến 90ns. Kết quả mô phỏng cho thấy CPU hoạt động đúng trong giai đoạn đầu của chương trình kiểm tra. Các lệnh addi và add lần lượt tạo dữ liệu cho các thanh ghi x1, x2 và x3. Từ thời điểm 50ns, tín hiệu ResultW bắt đầu xuất ra các giá trị lần lượt là 0x0000000a, 0x00000005 và 0x0000000f, tương ứng với kết quả ghi vào các thanh ghi x1, x2 và x3.

Khi lệnh beq x3, x3, +8 được xử lý tại giai đoạn thực thi, do hai toán hạng so sánh bằng nhau nên điều kiện rẽ nhánh được thỏa mãn. Vì vậy, CPU phát tín hiệu chọn địa chỉ nhánh cho bộ đếm chương trình và đồng thời xóa lệnh kế tiếp trong đường ống tại thời điểm 60ns. Sau quá trình xóa, CPU tiếp tục thực thi đúng lệnh tại địa chỉ nhánh, cụ thể là lệnh sub x4, x3, x2, nhằm tạo ra kết quả $x4 = 10$.



Hình 4.24: Dạng sóng thực thi các lệnh của bài kiểm tra số 2 từ 90ns đến 130ns

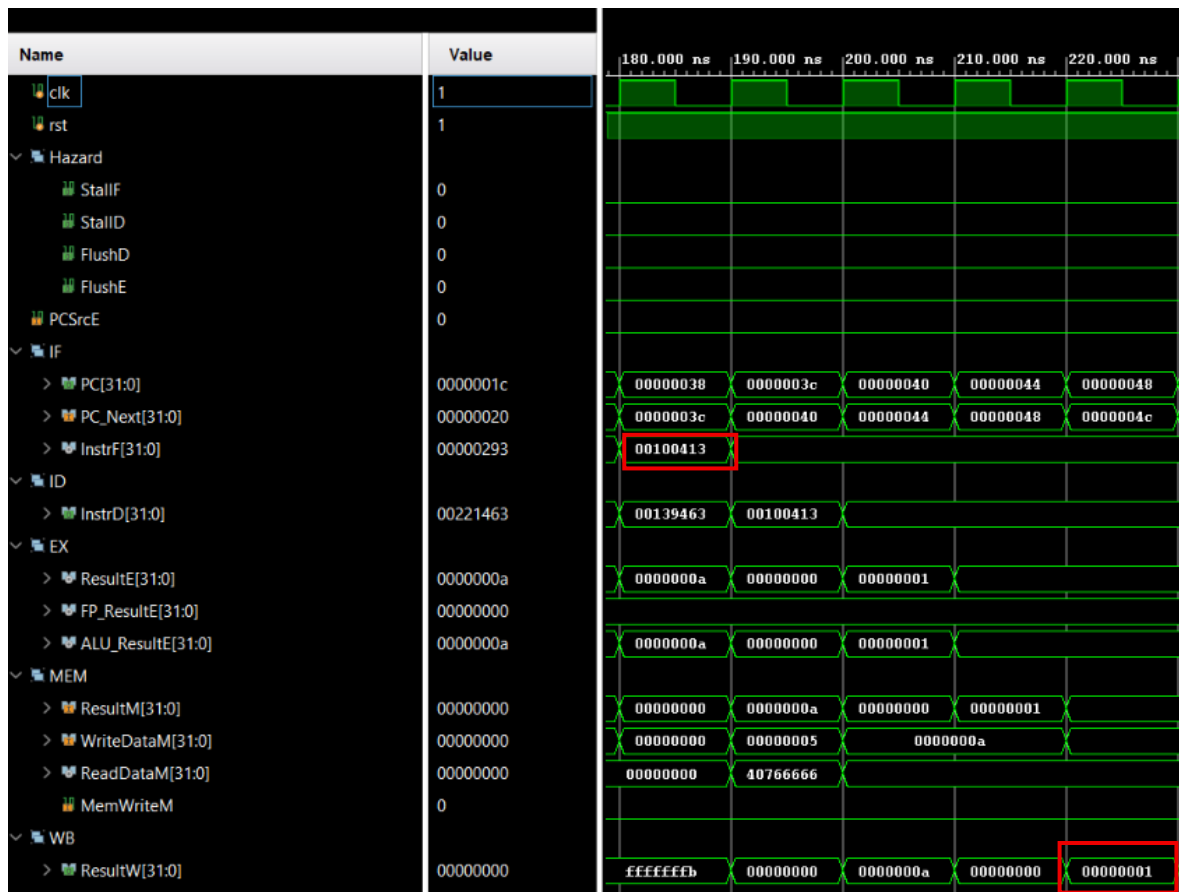
Như hình 4.24, tại thời điểm 100ns lệnh bne x4,x2,+8 được thực thi. Kết quả từ tín hiệu ResultE bằng 0x00000005 khác 0 nên lệnh nhảy sẽ được thực thi. Lệnh nhân sẽ không được thực thi tại khối xử lý mà thay vào đó là lệnh chia.



Hình 4.25: Dạng sóng thực thi các lệnh của bài kiểm tra số 2 từ 140ns đến 180ns

Hình 4.25 cho thấy ở giai đoạn 140ns đến 180ns, CPU thực hiện phép chia và so sánh bé hơn. Tín hiệu ResultW tại 150ns cho thấy CPU tính đúng $x6 = 5$, sau đó so sánh $x6 < x1$. Vì điều kiện đúng, chương trình bỏ qua lệnh `addi x7,x0,0` và tiếp tục với lệnh `add x7,x6,x2`.

Hình 4.26 cho thấy giai đoạn từ 180ns đến 220ns, CPU xử lý lệnh nhánh cuối `bne`. Do giá trị $x7$ bằng $x1$, điều kiện nhánh không thỏa mãn nên CPU tiếp tục thực thi tuần tự. Sau đó, lệnh `addi x8, x0, 1` được thực hiện.



Hình 4.26: Dạng sóng thực thi các lệnh của bài kiểm tra số 2 từ 180ns đến 220ns

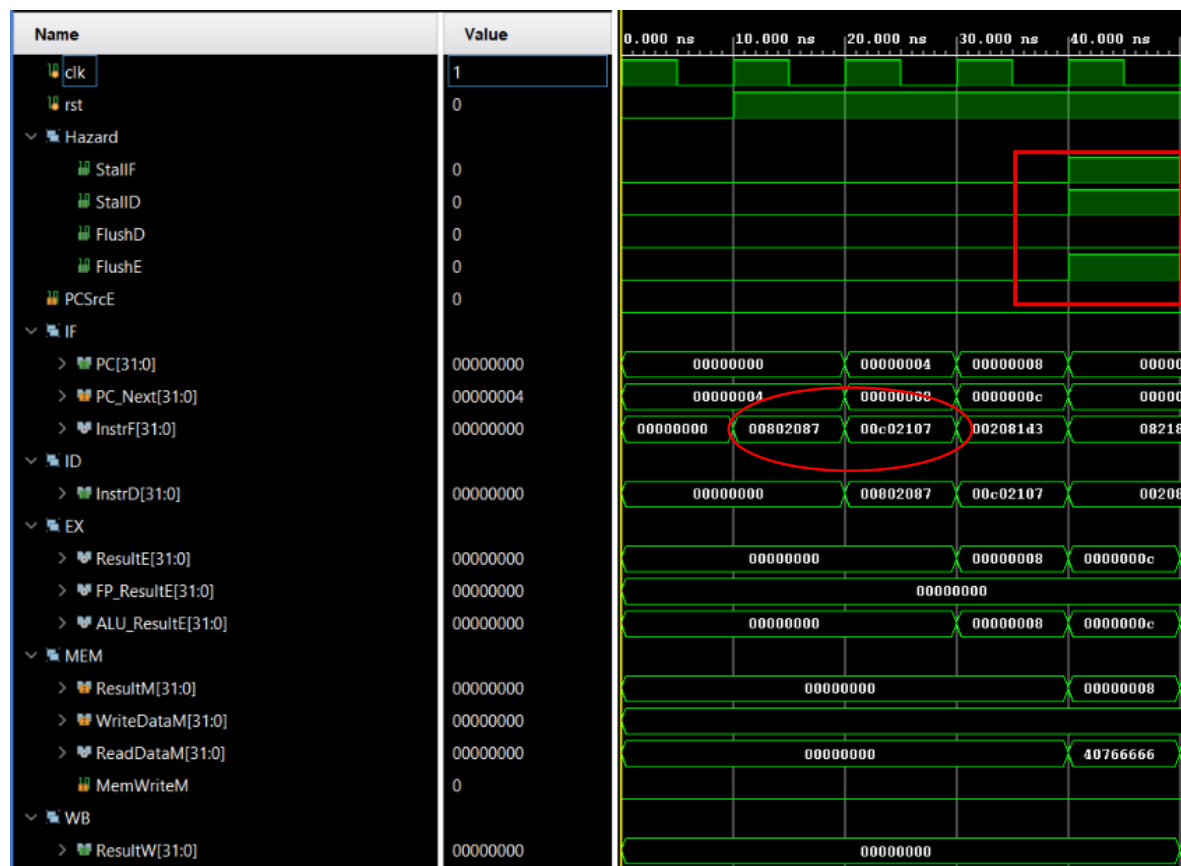
4.2.3. Bài kiểm tra số 3

Ở bài kiểm tra số 3, chương trình sẽ áp dụng các lệnh dấu phẩy động nhằm kiểm tra khả năng tính toán của khối FPU và cơ chế chuyển tiếp của CPU.

Bảng 4.3: Bảng mã các lệnh của bài kiểm tra số 3

STT	Mã hex	Mô tả lệnh	Kết quả dự kiến
1	00802087	flw f1,0(x0)	f1 = 3.85 (0x40766666)
2	00C02107	flw f2,4(x0)	f2 = 1.62 (0x3FCF5C29)
3	002081D3	fadd.s f3,f1,f2	f3 = f1 + f2 = 5.47 (0x40AF0A3D)
4	08218253	fsub.s f4,f3,f2	f4 = f3 – f2 = 3.85 (0x40766666)
5	102202D3	fmul.s f5,f4,f2	f5 = f4 × f2 = 6.237 (0x40C79580)
6	18128353	fdiv.s f6,f5,f1	f6 = f5 / f1 = 1.62 (0x3FCF5C28)

Ở bài kiểm tra này, chương trình thực hiện nạp hai số thực đã được khởi tạo sẵn trong khối Data_mem. Hai giá trị lần lượt là 3.85 và 1.62, tương ứng với mã biểu diễn theo chuẩn IEEE 754 dạng hex là 0x40766666 và 0x3FCF5C29.

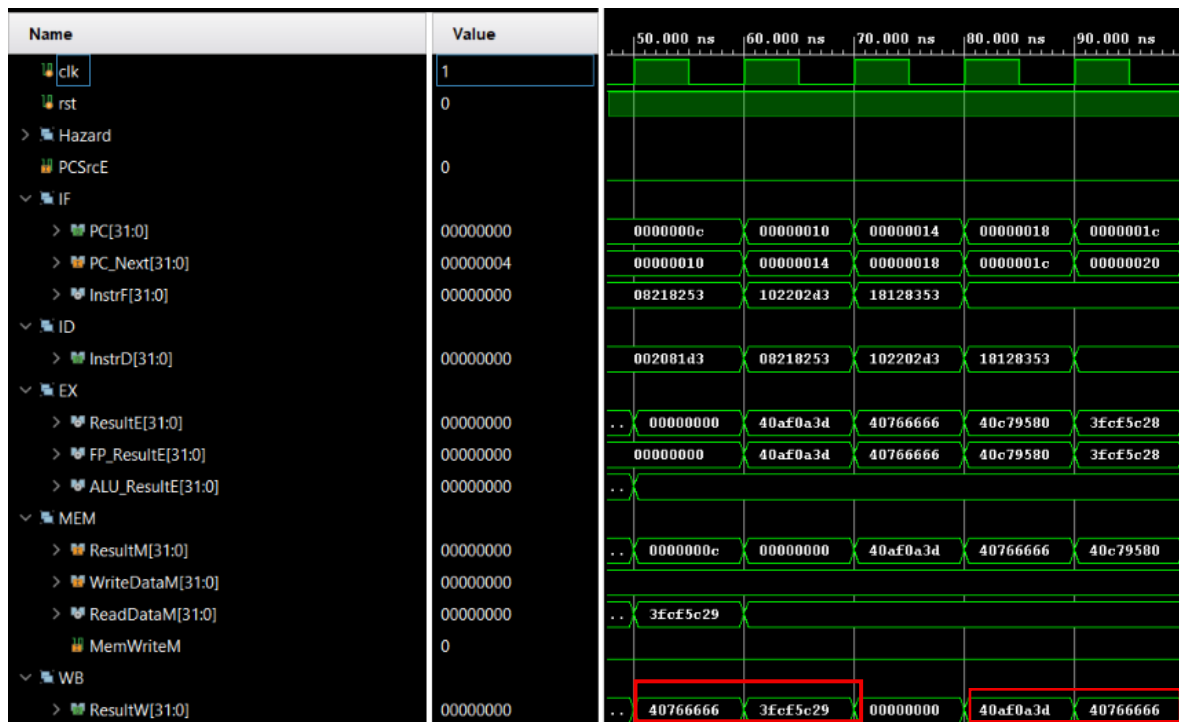


Hình 4.27: Dạng sóng thực thi các lệnh của bài kiểm tra số 3 từ 0ns đến 40ns

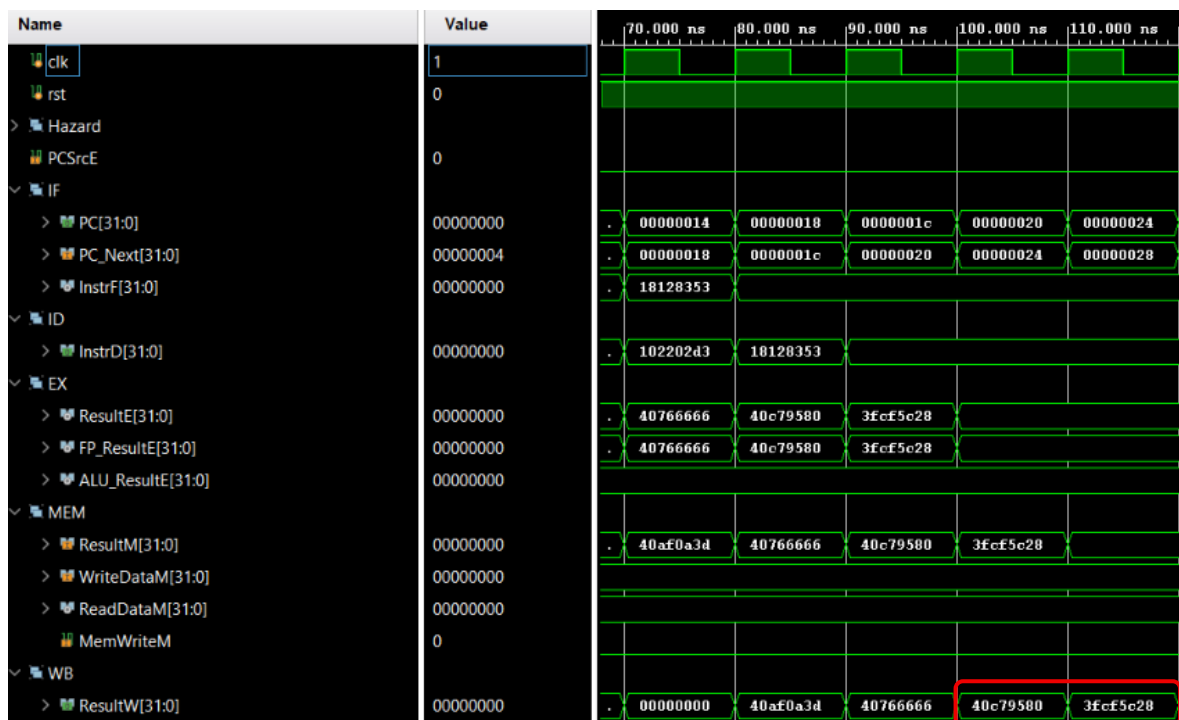
Hình 4.27 cho thấy các lệnh được thực thi tuần tự, CPU bắt đầu nạp hai toán hạng dấu phẩy động từ bộ nhớ: $f1 = 3.85$ tại địa chỉ 0x08 và $f2 = 1.62$ tại địa chỉ 0x0C. Đến 40ns, lệnh fadd.s đã vào khối giải mã và chuẩn bị sử dụng kết quả của flw f2, nên sẽ phát hiện xung đột ở chu kỳ kế tiếp và cần chèn trì hoãn để đảm bảo dữ liệu f2 hợp lệ.

Theo hình 4.28, tín hiệu ResultW tại các thời điểm 50ns và 60ns, các giá trị đã được nạp vào thanh ghi sẵn sàng cho việc thực hiện các phép tính.

Hình 4.28 và hình 4.29, tín hiệu ResultW từ thời điểm 80ns cho ra các giá trị lần lượt là 0x40AF0A3D, 0x40766666, 0x40C79580 và 0x3FCF5C28. Đúng với kết quả kỳ vọng.



Hình 4.28: Dạng sóng thực thi các lệnh của bài kiểm tra số 3 từ 50ns đến 90ns



Hình 4.29: Dạng sóng thực thi các lệnh của bài kiểm tra số 3 từ 70ns đến 90ns

Do các số thực được biểu diễn theo chuẩn IEEE 754 32 bit nên một số giá trị thập phân không thể được biểu diễn chính xác tuyệt đối trong hệ nhị phân. Vì vậy, khi thực hiện các phép toán dấu phẩy động, kết quả có thể xuất hiện sai số do quá trình biểu diễn và làm tròn. Đối với bài kiểm tra số 3, sai số tương đối của các phép tính được xác định theo công thức:

$$\text{Sai số tương đối} = \frac{| \text{giá trị kỳ vọng} - \text{giá trị thực tế} |}{\text{giá trị kỳ vọng}} \times 100\%$$

Bảng 4.4: Bảng sai số giữa kết quả kỳ vọng và kết quả thực tế

Lệnh	Kết quả kỳ vọng	Mã hex	Kết quả thực tế	Sai số tương đối
fadd.s f3, f1, f2	5.47	0x40AF0A3D	5.4699997901	0.0000038373%
fsub.s f4, f3, f2	3.85	0x40766666	3.8499999046	0.0000024779%
fmul.s f5, f4, f2	6.237	0x40C79580	6.2369995117	0.0000078283%
fdiv.s f6, f5, f1	1.62	0x3FCF5C28	1.6199998855	0.0000070679%

Từ bảng 4.4 có thể thấy sai số tương đối của các phép toán dấu phẩy động đều rất nhỏ. Điều này cho thấy kết quả tính toán của FPU có độ chính xác cao, sai lệch nhỏ chủ yếu do quá trình biểu diễn và làm tròn số.

4.3 Kết quả kiểm tra bằng môi trường UVM

Bên cạnh việc kiểm tra chức năng thông qua mô phỏng dạng sóng, đề tài còn xây dựng môi trường kiểm tra bằng UVM nhằm tự động hóa quá trình xác minh hoạt động của CPU. Môi trường UVM được sử dụng để nạp chương trình kiểm tra vào bộ nhớ lệnh, điều khiển quá trình khởi động lại, quan sát các lệnh được thực thi và so sánh kết quả thực tế với kết quả kỳ vọng thông qua scoreboard. Các bài kiểm tra được thực hiện trong môi trường UVM được thể hiện ở hình 4.30.

Dựa trên kết quả mô phỏng, các bài kiểm tra được thực hiện bao gồm kiểm tra các lệnh loại R, kiểm tra các lệnh loại I, kiểm tra phần mở rộng M, kiểm tra phần mở rộng F và các xung đột trong đường ống.

```
# =====
# STEP 5: Per-test coverage reports
# =====
# >>> Per-test report: report_riscv_rtype_test.txt
# >>> Per-test report: report_riscv_itype_test.txt
# >>> Per-test report: report_riscv_mext_test.txt
# >>> Per-test report: report_riscv_fext_test.txt
# >>> Per-test report: report_riscv_hazard_test.txt
#
# =====
# REGRESSION SUMMARY
# =====
# Tests run: 5
# Test list:
#   riscv_rtype_test
#   riscv_itype_test
#   riscv_mext_test
#   riscv_fext_test
#   riscv_hazard_test
#
# Merged UCDB: merged_coverage.ucdb
# HTML report: cov_html_merged/
# Log files: logs/*_20260610_233713.log
#
# HOÀN TẤT! Mở cov_html_merged/index.html để xem coverage tổng hợp.
# =====
```

Hình 4.30: Các bài kiểm tra được thực hiện trong môi trường UVM

```
# =====
# STEP 4: Generating merged coverage reports
# =====
# QuestaSim-64 vcover 10.7c Coverage Utility 2018.08 Aug 18 2018
# Start time: 23:38:27 on Jun 10,2026
# vcover report -detail merged_coverage.ucdb -output report_merged_coverage.txt
# End time: 23:38:27 on Jun 10,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
# >>> Text report: report_merged_coverage.txt
# QuestaSim-64 vcover 10.7c Coverage Utility 2018.08 Aug 18 2018
# Start time: 23:38:27 on Jun 10,2026
# vcover report -html merged_coverage.ucdb -htmldir cov_html_merged
# Report created in cov_html_merged/index.html
# End time: 23:38:28 on Jun 10,2026, Elapsed time: 0:00:01
# Errors: 0, Warnings: 0
# >>> HTML report: cov_html_merged/
# cov_html_merged/testlist.html
# cov_html_merged/covsummary.html
# =====
```

Hình 4.31: Kết quả tổng hợp từ các bài kiểm tra

Bảng 4. 5: Tổng hợp kết quả kiểm tra từ môi trường UVM

STT	Bài kiểm tra	Nội dung	Vượt qua	Không vượt qua	Kết quả
1	riscv_rtype_test	Kiểm tra các lệnh loại R như add, sub, slt, xor, or, and	16	0	Đạt
2	riscv_itype_test	Kiểm tra các lệnh loại I	12	0	Đạt
3	riscv_mext_test	Kiểm tra các lệnh mở rộng M như mul, div, rem	20	0	Đạt
4	riscv_fext_test	Kiểm tra các lệnh số thực thuộc phần mở rộng F	22	0	Đạt
5	riscv_hazard_test	Kiểm tra cơ chế xử lý xung đột	12	0	Đạt

Nhìn chung, tất cả các bài kiểm tra đều có số lượng không vượt qua bằng 0 và không xuất hiện lỗi trong phần báo cáo tổng kết. Điều này chứng minh môi trường UVM đã kiểm tra đúng quá trình hoạt động của DUT, đồng thời xác nhận CPU có thể thực thi chính xác các nhóm lệnh chính thuộc RV32I, phần mở rộng M, phần mở rộng F và cơ chế xử lý xung đột trong đường ống. Kết quả này cho thấy thiết kế đáp ứng đúng các chức năng đã đề ra và môi trường UVM hỗ trợ hiệu quả cho quá trình xác minh tự động.

Questa Coverage Report

Number of tests run:	5
Passed:	5
Warning:	0
Error:	0
Fatal:	0

[List of tests included in report...](#)

[List of global attributes included in report...](#)

[List of Design Units included in report...](#)

Coverage Summary by Structure:			Coverage Summary by Type:						
Design Scope	Hits %	Coverage %	Total Coverage:		100.00%	100.00%			
riscv_pkg	100.00%	100.00%	Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
riscv_coverage	100.00%	100.00%	Covergroups	46	46	0	1	100.00%	100.00%

Hình 4.32: Kết quả độ bao phủ từ QuestaSim

Hình 4.32 trình bày báo cáo độ bao phủ sau khi chạy bằng môi trường UVM trên QuestaSim. Kết quả cho thấy tổng cộng có 5 bài kiểm tra được thực thi, trong đó cả 5 đều hoàn thành thành công, không xuất hiện lỗi. Bên cạnh đó, phần tổng kết độ bao phủ cho thấy độ bao phủ đạt 100%. Điều này chứng minh rằng các kịch bản kiểm tra trong môi trường UVM đã bao phủ đầy đủ các trường hợp chức năng được xây dựng, đồng thời xác nhận CPU hoạt động đúng với các nhóm lệnh và cơ chế xử lý đã kiểm tra.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN

Đề tài đã thực hiện thiết kế và kiểm tra CPU RISC-V 32 bit sử dụng kiến trúc đường ống 5 tầng. CPU được xây dựng với đầy đủ các khối chức năng chính bao gồm khối nạp lệnh, giải mã lệnh, thực thi, truy cập bộ nhớ, ghi kết quả và khối xử lý xung đột. Thiết kế hỗ trợ các nhóm lệnh cơ bản thuộc RV32I, một số lệnh mở rộng M và các lệnh số thực thuộc RV32F.

Kết quả mô phỏng cho thấy CPU thực thi đúng các chương trình kiểm tra đã xây dựng. Các lệnh tính toán số nguyên, lệnh rẽ nhánh, lệnh truy cập bộ nhớ, lệnh nhân chia và lệnh số thực đều cho kết quả phù hợp với giá trị kỳ vọng. Ngoài ra, môi trường kiểm tra UVM cũng được xây dựng để tự động hóa quá trình xác minh. Kết quả chạy cho thấy các kịch bản kiểm tra đều đạt, không xuất hiện lỗi và độ bao phủ đạt 100%. Điều này chứng minh thiết kế CPU hoạt động đúng trong phạm vi các chức năng đã kiểm tra.

5.2. HƯỚNG PHÁT TRIỂN

Mặc dù đề tài đã hoàn thành cơ bản các mục tiêu đặt ra trong việc thiết kế và kiểm tra CPU RISC-V 32 bit đường ống 5 tầng, thiết kế vẫn còn một số hướng có thể tiếp tục phát triển trong tương lai. CPU có thể được mở rộng để hỗ trợ đầy đủ và đa dạng hơn các lệnh thuộc tập lệnh RV32I, RV32M và RV32F, đồng thời bổ sung thêm các trường hợp kiểm tra nhằm nâng cao độ tin cậy của quá trình xác minh. Ngoài ra, thiết kế có thể được phát triển thêm các bus giao tiếp như AHB và APB để kết nối CPU với bộ nhớ và các ngoại vi bên ngoài như UART, SPI, I2C. Việc bổ sung các giao tiếp này sẽ giúp hệ thống phù hợp hơn cho quá trình triển khai trên kit FPGA.

TÀI LIỆU THAM KHẢO

- [1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware/Software Interface, 2nd ed. Cambridge, MA: Morgan Kaufmann, 2021.
- [2] A. Sharma, "How NVIDIA shipped one billion RISC-V cores in 2024," *RISC-V International Blog*, Feb. 25, 2025. [Online]. Available: <https://riscv.org/blog/how-nvidia-shipped-one-billion-risc-v-cores-in-2024/>. [Accessed: 2026].
- [3] Synopsys, "What is RISC-V?," Synopsys Glossary. [Online]. Available: <https://www.synopsys.com/glossary/what-is-risc-v.html>. [Accessed: Apr. 11, 2026].
- [4] J. Lu, "Single-Cycle vs. Five-Stage Pipelined RISC-V CPU Designs: Architecture and Performance Comparison," in 2025 7th International Conference on Electronic Engineering and Informatics (EEI), Yangzhou, China, 2025.
- [5] A. Parsons, "Design of a 32-bit RISC Microprocessor with Floating Point Unit," Carleton University, Ottawa, Canada, 2010.
- [6] L. Bertaccini, M. Perotti, S. Mach et al., "Tiny-FPU: Low-Cost Floating-Point Support for Small RISC-V MCU Cores," in Proc. IEEE Int. Symp. Circuits and Systems (ISCAS), 2021.
- [7] E. M. Hamed, K. Salah, and A. H. Madian, "An Automated Lightweight UVM Tool," in Proc. IEEE Int. Conf. ..., 2018.